

BNCmatmul ユーザーズガイド

Tomonori Kouya

<https://github.com/tkouya/bncmatmul>

Version 0.23: March 16, 2026

目次

第 1 章	BNCmatmul とは何か	1
1.1	著作権と配布条件	2
1.2	BNCmatmul のコンパイル方法	2
1.3	SIMD バックエンド: 実装範囲・ISA 要件・実測性能	3
1.4	数学的記法	5
1.5	なぜ多倍長精度浮動小数点演算ライブラリが必要なのか	6
1.6	最適化された行列乗算のアルゴリズムと性能	7
1.7	BNCmatmul のソフトウェア層	9
1.8	バージョン履歴と Todo リスト	10
第 2 章	基本データ型と算術演算	12
2.1	c_dd_*算術層 (rdd.h)	12
2.2	誤差なし変換 (error-free transformation)	13
2.3	double-double 精度算術演算	13
2.4	quadruple-double 精度算術演算	15
2.5	triple-double 精度算術演算	16
2.6	Branch-free 融合積和演算 (bncfma.h)	17
2.7	FMA ベース初等関数 (bncelem.h)	19
2.8	DLL / Python 向け関数エクスポート API (rdtq_func.h)	21
2.9	rdd.h	22
2.10	AVX2	25
2.11	その他の関数	28
2.12	ベンチマーク用ツール関数	30
第 3 章	基本的な線形計算	33
3.1	D, DD, TD, QD および MPFR のベクトルと行列のデータ型	33
3.2	ベクトル演算	34
3.3	行列演算	37
3.4	行列とベクトルのファイル入出力	41
3.5	テスト行列の生成	42
3.6	単純法・ブロック法・Strassen 法による行列乗算と関連関数	43
3.7	ベクトルと行列の相対誤差の取得	45

3.8	尾崎スキームと関連関数	46
第 4 章	LU 分解	49
4.1	関数一覧	49
第 5 章	配列・多項式と代数方程式のソルバ	51
5.1	配列と多項式のデータ型	51
5.2	関数一覧	53
第 6 章	疎行列および関連関数	66
6.1	CRS 行列の構造体	66
6.2	クリロフ部分空間反復解法	75
第 7 章	単精度系および複素数系の精度ファミリ	88
7.1	単精度系マルチコンポーネント型 (DS, TS, QS)	88
7.2	複素数型 (CD およびマルチコンポーネント複素数)	89
7.3	利用可能なファミリの一覧	90
第 8 章	CUDA による GPU 計算	91
8.1	命名規則とデータ型	91
8.2	デバイスメモリとホスト・デバイス間転送	92
8.3	ベクトルと行列の演算	93
8.4	行列乗算と行列ベクトル積	93
8.5	GPU 上の LU 分解と線形ソルバ	94
8.6	疎行列ベクトル乗算 (SpMV)	94
8.7	任意精度行列乗算 (CUDA MPFR / MPC)	95
8.8	GPU スイートのビルドと実行	95
第 9 章	Python からの BNCmatmul の利用	96
9.1	共有ライブラリのビルド	96
9.2	ライブラリのロードとフル精度表示	96
9.3	算術演算・FMA・初等関数	97
9.4	NumPy と要素毎の一括関数	98
9.5	注意事項	99
参考文献		100

第 1 章

BNCmatmul とは何か

BNCmatmul は、double(binary64)、DD(double-double)、TD(triple-double)、QD(quadruple-double)、および MPFR による任意精度浮動小数点演算といった多倍長精度演算をサポートする最適化 BLAS(Basic Linear Algebra Subprograms) ライブラリの一つです。

拡張多倍長精度線形計算のためのライブラリである MPLAPACK/MPBLAS が標準的な多倍長精度数値計算環境の基盤を提供するようになった現在、より高速な基本線形計算を提供する最適化ライブラリへの需要が高まっています。標準的な最適化手法としては SIMD 命令の利用や OpenMP による並列化が挙げられますが、多倍長精度行列乗算においては、分割統治法や Ozaki scheme といったアルゴリズムによる最適化も利用可能です。私たちは、これらすべての最適化手法を利用でき、multi-component 方式による固定精度計算と multi-digit 方式による任意精度計算の両方をサポートする新しいバージョンの BNCmatmul を開発しました。本稿では、そのソフトウェア構造を説明し、最適化された行列ベクトル乗算および行列乗算の性能評価結果を示します。

私たちは、Xeon および EPYC という以下の環境で BNCmatmul を開発しています。Intel OneAPI または GNU Compiler Collection を備えた他の x86_64 linux 環境においても、本ライブラリはおそらく利用可能です。

EPYC AMD EPYC 7402P 24 cores, Ubuntu 18.04.6 LTS, Intel Compiler version 2021.4.0, MPLAPACK 1.0.1, MPFR 4.1.0

Xeon Intel Xeon W-2295 3.0GHz 18 cores, Ubuntu 20.04.3 LTS, Intel Compiler version 2021.5.0, MPLAPACK 1.0.1, MPFR 4.1.0

BNCmatmul バージョン 0.23 から、AArch64 の Arm Neon を用いて本ライブラリを高速化できるようになりました。私たちは、以下の Arm CPU 環境でテストを行っています。

Raspberrypi Raspberry Pi 5 Model B Rev 1.0, Broadcom BCM2712 2.4GHz Coretex-A76 4 cores, Debian GNU/Linux 12 ,GCC 12.2.0, MPLAPACK 2.0.1

Snapdragon Qualcomm Snapdragon X Plus 3.24GHz 8 cores, Ubuntu 24.04.3 LTS on Windows 11, GCC 13.3.0, MPLAPACK 2.0.1

1.1 著作権と配布条件

BNCmatmul は幸谷智紀 (<https://na-inet.jp/>) によって作成され、本ドキュメントを含めて [GPL] バージョン 2 以降の下で配布されています。

[GPL]: <https://www.gnu.org/copyleft/gpl.html>

1.2 BNCmatmul のコンパイル方法

BNCmatmul は、以下のように Intel OneAPI Compiler (ICC) または GNU Compiler Collection (GCC) でコンパイルできます。

1. <https://github.com/tkouya/bncmatmul> からホームディレクトリに git clone します。
2. `./configure` を実行して、コンパイラおよび GMP/MPFR/MPC/QD のインストールを検出し、`bncmatmul.inc` を生成します。このファイルは生成物なので、`bncmatmul.inc` 自体ではなく `bncmatmul.inc.in` を編集する（または `configure` オプションを渡す）ようにしてください。
3. `libbncmatmul*.a` と `python/libbncmatmul*.so` をビルドするために "make all" を実行します。
4. "test" ディレクトリ内の反復改良 C++ プログラムを、BNCmatmul および LAPACK/MPBLAS とともにコンパイルします。

GMP・MPFR・MPC・QD が既定の prefix にない場合は、`configure` にその場所を指定します。

```
./configure --with-gmp=$HOME/local --with-mpfr=$HOME/local \
            --with-mpc=$HOME/local --with-qd=$HOME/local \
            --enable-omp --enable-avx2 --enable-avx512
```

`-enable-avx2` と `-enable-avx512` の既定値は `auto` で、ホストが x86-64 であれば両方が有効になります (AArch64 では `-enable-neon` と `-enable-sve2` が同様に働きます)。そのバックエンドを外すには `-disable-avx512` を指定します。有効になったバックエンドとフラグは `configure` 末尾のサマリに表示されます。なお、`configure` が判定するのは**ホストアーキテクチャ**であって、その CPU が実際に AVX-512 を実行できるかどうかではありません。AVX-512 非対応のマシンでは `avx512` ターゲットを明示的に無効化してください。

`make` には SIMD バックエンドごとのターゲットがあります。いずれもそのバックエンドの SIMD フラグで**ライブラリ全体を再コンパイル**し、専用の名前でアーカイブします。したがって各バックエンドは共存し、選択はリンク時にのみ行われます。

ターゲット	SIMD フラグ	生成されるライブラリ
make	—	<code>libbncmatmul-0.24.a</code> , ... <code>_winograd.a</code>
make avx2	<code>-mavx2 -mfma</code>	<code>libbncmatmul-0.24_avx2.a</code> , <code>-omp_avx2.a</code> , <code>_winograd_a</code>
make avx512	<code>-mavx512f -mfma</code>	<code>libbncmatmul-0.24_avx512.a</code> と同様の 3 種
make neon	<code>-mcpu=cortex-a76 -DBNC_ENABLE_NEON</code>	<code>libbncmatmul-0.24_neon.a</code> , <code>-omp_neon.a</code>
make sve2	<code>-mcpu=neoverse-v2 -DBNC_ENABLE_SVE2</code>	<code>libbncmatmul-0.24_sve2.a</code> , <code>-omp_sve2.a</code>

プログラムのバックエンド変更は -l フラグの差し替えだけで済みます。ヘッダ内のインライン SIMD コードが同じ分岐を選ぶよう、呼び出し側も対応する -mavx2 -mfma または -mavx512f -mfma でコンパイルしてください。

```
g++ -O3 -ffp-contract=off -mavx512f -mfma -Iinclude prog.cc \
-L. -lbncmatmul-0.24_avx512 -lmpc -lmpfr -lgmp -lqd -lstdc++ -lm
```

インクルードパスに関する注意が2点あります。第一に、-Iinclude (ソースツリー) は GMP/MPFR/QD の prefix より前に置いてください。以前の make install により同じ prefix の下に BNCmatmul のヘッダが残っていると、古いコピーがライブラリのビルドに使われたヘッダを覆い隠してしまいます。第二に、-ffp-contract=off は必須です。誤差なし変換は、コンパイラが乗算と加算を勝手に FMA へ融合しないことを前提としているためです。

1.3 SIMD バックエンド: 実装範囲・ISA 要件・実測性能

4 つの SIMD バックエンドは同一のソース内でプリプロセッサにより、次の順序で選択されます。#if __AVX512F__ ... #elif __AVX2__ ... #elif __ARM_SVE2__ ... #elif __ARM_NEON__ ... #else という順序で判定され、利用可能な最も広いバックエンドが選ばれます。最後の #else が移植性のあるスカラーパスです。GCC は AVX-512 ビルドでも __AVX2__ を定義するため、AVX2 側のガードは #if defined(__AVX2__) && !defined(__AVX512F__) と書かれています。したがって #elif __AVX512F__ の腕を持たない構文は、-mavx512f では AVX2 側ではなく **スカラー**パスを通ります。0.24 では、AVX2 ブロックを持つソースはすべて AVX-512 ブロックも備えており、ライブラリのビルド対象 154 ファイル中 426 個の AVX2 構文のうち、AVX-512 の腕を欠くのは 15 個のみです: norm2_{ds,ts,qs}vector、poly.c・dd_poly.c・cdd_poly.c の ceval_diff_ 系多項式評価、7 個の_bncavx2*_estrin カーネル (引数シグネチャ __m256d ret[] が AVX2 固有のため。AVX-512 ビルドではスカラーのディスパッチャを呼びます)、および OpenMP 版 LU の rank-1 更新 _bncomp_{TD,QD}LUdecompPM です。この 15 個は avx512 ビルドではスカラー実行になります。

ISA 要件。 AVX2 ビルドには AVX2 と FMA (-mavx2 -mfma、Haswell 以降) が必要です。AVX-512 ビルドに必要なのは基盤サブセット AVX-512F と FMA のみ (-mavx512f -mfma) で、AVX-512DQ/BW/VL はどのカーネルでも使っていません。したがって生成されるライブラリは AVX-512F 対応の任意の CPU — Skylake-SP、Ice Lake、Sapphire/Emerald Rapids、さらに Knights Landing — で動作します。両ビルドとも -ffp-contract=off でコンパイルされます。誤差なし変換は、コンパイラが乗算と加算を勝手に融合しないことに依存しているためです。

バックエンド間の再現性。 各 SIMD カーネルはスカラーの誤差なし変換をレーン方向に書き直したもので、要素ごとの演算 — AXPY (add_cmul)・スカラー倍・加算、および疎行列の SpMV・SpMV^T — は 6 種の EFT 型すべてについて、serial・AVX2・AVX-512 の各ビルドで **ビット単位に一致**します。bench/mds/mds_verify.cc がこれを検査し、 $n = 96 \cdot 257 \cdot 512 \cdot 1000$ で 3 つのビルドが同一のチェックサムを出力します。

リダクションを伴うカーネルには但し書きが必要です。内積 — したがって GEMV と GEMM — は SIMD レーン上で W 個の部分積を蓄積して最後にまとめるため、スカラーループとは加算順序が異なり、multi-component 加算は結合的ではありません。乱数被演算数ではこれが dd と ds で観測され、GEMV と GEMM

の結果は serial・AVX2・AVX-512 の各ビルドで異なります。差は最下位 limb の水準 (dd で相対 10^{-16} 未満、約 48 ビットの ds で 10^{-14})、すなわちその型自身の丸め水準です。同じ試験で td・qd・ts・qs の GEMV と GEMM は 3 バックエンドすべてでビット単位に一致しました (bench/mds/mds_verify_rand.cc で再現できます)。バックエンドをまたいだ再現性が速度より重要な場合は、常に同一のライブラリにリンクして実行してください。

実測性能。以下の数値は bench/mds/run_x86.sh で測定しました。同一の bench/mds/mds_bench パイナリを serial・AVX2・AVX-512 の各ライブラリにリンクし直して実行するものです。測定環境は Intel Xeon Gold 6526Y (Emerald Rapids、2.8/3.9 GHz、GCC 13.3、GMP 6.3.0、MPFR 4.2.2) の 1 コアで、taskset でピン留めし、 $n = 128, 256, 512, 1024$ の結果を用いています。各欄はその 4 サイズにわたる serial ビルドに対する MFLOPS 比の幾何平均です。演算は AXPY (add_cmul)・GEMV・GEMM・半帯幅 3 の帯行列 SpMV です。

型	AXPY		GEMV		GEMM		SpMV	
	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512
float	1.32	2.13	6.37	7.83	1.21	1.22	0.27	0.14
double	1.23	1.31	3.14	4.57	1.11	1.09	1.02	0.96
dd	5.80	8.81	6.25	6.52	3.67	3.74	1.68	0.76
td	4.68	6.83	4.63	5.21	3.17	3.62	1.47	0.64
qd	4.55	6.97	3.93	5.62	2.96	3.81	1.71	0.95
ds	9.31	13.36	10.45	7.07	4.36	3.33	0.45	0.17
ts	3.43	27.83	2.18	8.60	1.91	5.43	0.69	0.41
qs	4.72	29.69	4.39	15.56	3.92	9.07	0.99	0.59

表から読み取るべき点が 3 つあります。第一に、SIMD の効果はプレーンな float/double より multi-component 型ではるかに大きいことです。DD の加算は十数段の依存した two_sum の連鎖であり、 W 要素方向へベクトル化するとこの依存連鎖が解消されます。一方、プレーンな double の GEMM はすでにメモリバウンドです。第二に、密カーネルではほぼ全面的に AVX-512 が AVX2 をさらに上回ります。例外は ds で、GEMV と GEMM は AVX2 の方がやや高速です。第三に — ビルドを選ぶ際にはここが重要です — 帯の狭い疎行列の SpMV は SIMD で遅くなり、AVX-512 では特に顕著です。

SpMV の性能低下は構造的なものであり、バグではありません。drsmatrix とその多倍長版は、行ループに端数処理を不要にするため、各行を SIMD 幅 (double 基の型は _BNC_D_WIDTH、float 基の型は _BNC_S_WIDTH) の倍数にパディングします。半帯幅 3 の帯行列は 1 行あたり非零要素が 7 個で、AVX2 の double および AVX-512 の double では 8 にパディングされますが、AVX-512 の float では 16 になり、有効な演算の 2 倍以上になります。しかも x の不規則な gather には再利用がなく、この分を償却できません。このような行列では serial または AVX2 のライブラリの方が適しています。行の非零要素数がベクトル幅を十分に上回るようになって初めて、幅の広いバックエンドが効いてきます。bench/mds/sp_perf.cc による dd の測定 ($n = 8192$ 、serial ビルドに対する速度比) を次に示します。

半帯幅	非零要素/行	AVX2	AVX-512
1	3	0.92	0.29
3	7	1.70	0.77
7	15	2.82	1.62
15	31	3.60	2.28
31	63	4.62	3.47
63	127	4.98	4.12

AVX-512 が損益分岐に達するのは 1 行あたり非零要素 7~15 個の間で、127 個でも AVX2 には追いつきません。疎行列計算では AVX2 版のライブラリを既定にするのが安全です。複素の疎行列型は挙動が異なります。cdd・ctd・cqđ の SpMV は半帯幅 3 でもどちらのバックエンドでも 2.9~4.1 倍高速化されます。複素の非零要素 1 個あたり実数乗算が 4 回発生し、パディングされたレーンを償却できるだけの演算量があるためです。

ts と qs の行については注意が必要です。AVX-512 側のヘッダ `avx2/_bncavx_{ts,qs}_avx512.h` はビルド時スイッチ `USE_TS_BF / USE_QS_BF` を解釈して branch-free の加算・乗算カーネルを選びますが、AVX2 側のヘッダ `avx2/_bncavx_{ts,qs}.h` はこれらを実装しておらず、常に分岐を含む `addq/mult` カーネルを使います (dd と ds では非対称性が逆向きです)。試験した被演算数では両カーネルはビット単位に一致するため結果は変わりませんが、AVX2 側の分岐が、ts/qs で AVX-512 が 28~30 倍に達する一方 AVX2 が 4 倍前後に留まる主因です。この差の大部分はベクトル幅ではなく選択されたカーネルに由来します。AVX2 向けに float 基の branch-free カーネルを (および dd/ds 向けに AVX-512 版を) 実装すれば解消できます。

複素型も `bench/mds/mds_bench_cplx` で同様に測定しました。密カーネルでの向上幅は控えめです (複素の積 1 回は実数の積 4 回と加算 2 回であり、有効な FLOP あたりのデータ移動が多くなるためです)。一方、前述のとおり SpMV については複素型が SIMD の恩恵を受ける唯一の疎行列ケースです。

型	AXPY		GEMV		GEMM		SpMV	
	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512
cd	1.16	1.37	1.08	1.08	1.00	1.13	1.89	1.15
cdd	2.15	3.20	2.79	2.89	1.84	2.03	3.57	4.14
ctd	3.50	4.90	3.05	3.28	2.35	2.80	3.17	2.86
cqd	3.98	6.08	3.18	4.06	2.38	3.14	3.19	3.18

両方の掃引は `bench/mds/run_x86.sh` で再現でき、サイズごとの生の測定値は `bench/mds/results_x86.csv` と `results_x86_cplx.csv` に残ります。個々の値は実行ごとに数パーセント変動します (測定ホストは `schedutil` ガバナかつ `turbo` 有効のため)。float/double の一部の点は作業セットがキャッシュ階層をまたぐ際にそれ以上に振れます。表が単一の値ではなく 4 サイズの幾何平均を掲げているのはこのためです。

1.4 数学的記法

ここで、 $\mathbb{F}_{bS}$ と $\mathbb{F}_{bL}$ を、それぞれ S ビットおよび L ビットの仮数を持つ浮動小数点数の集合として定義します。例えば、 $\mathbb{F}_{b24}$ と $\mathbb{F}_{b53}$ は IEEE754-1985 binary32 および binary64 浮動小数点数の集合を指し、一方で $\mathbb{F}_{b106}$ 、 $\mathbb{F}_{b159}$ 、 $\mathbb{F}_{b212}$ はそれぞれ DD、TD、QD 精度浮動小数点数の例を表します。MPFR 演算では任意の仮数長を選択できますが、MPFR 数の集合は $\mathbb{F}_{bM}$ として表され、これは主に `mpfr_set_default_prec` 関

数を用いて M ビットとして定義されます。

さらに、 n 次元ベクトル $\mathbf{x} = [x_i]_{i=1,2,\dots,n} \in \mathbb{R}^n$ の第 i 要素として $(\mathbf{x})_i (= x_i)$ を、 $A = [a_{ij}]_{i=1,2,\dots,m,j=1,2,\dots,n} \in \mathbb{R}^{m \times n}$ の (i, j) 要素として $(A)_{ij} (= a_{ij})$ を用います。

1.5 なぜ多倍長精度浮動小数点演算ライブラリが必要なのか

以下に、binary64 を超える仮数を持つ多倍長精度計算の必要性について簡単に説明します。まず、ユーザは浮動小数点数で表現された入力値 x から値 $F(x)$ を得るために浮動小数点演算を用いたいと考える場合があります。最終的に、 $F(x)$ は少なくとも U 桁の有効数字を保持しなければなりません。直感的には、数値計算において丸めによる初期誤差は入力値に含まれるものと仮定されます。浮動小数点演算の仮数桁数を $L(> U)$ とします。すると、 $F(x)$ の有効桁数は $U-R$ となり、ここで R は $F(x)$ を計算する過程で失われる桁数を表します。さらに、 L 桁の計算で精度を保証するようにアルゴリズムを変更できないものと仮定します。

$F(x)$ の計算アルゴリズムを変更しない限り、誤差伝播は仮数桁数に関わらずほとんど変化しません。したがって、仮数桁数を、 R をわずかに超える α 桁の余裕を持たせて $L + R + \alpha$ 桁まで増やすことができます。初期誤差を十分に小さくできれば、その結果として U 桁以上の有効数字を持つ $F(x)$ を得ることができます。この過程は、精度を保証するために多倍長精度計算がどのように用いられるかを示しています。

これに対して、Rump と Ogita によって提案された multi-fold 計算法 [9] [10] は、誤差なし変換 (error-free transformation) 技術を用いて算術演算で生じる丸め誤差を下位の桁に抑えることで、初期誤差の増加を防ぎます。誤差なし変換処理の計算様式は、多倍長精度計算とほぼ同じです。さらに、この処理では L 桁以上の浮動小数点演算は用いられません。したがって、再正規化手続きを行う必要がありません。これにより、誤差なし変換技術を用いる multi-component 方式と比較して計算量を削減できるという利点があります。

前述の2つのアプローチの概念図を図 1.1 に示します。

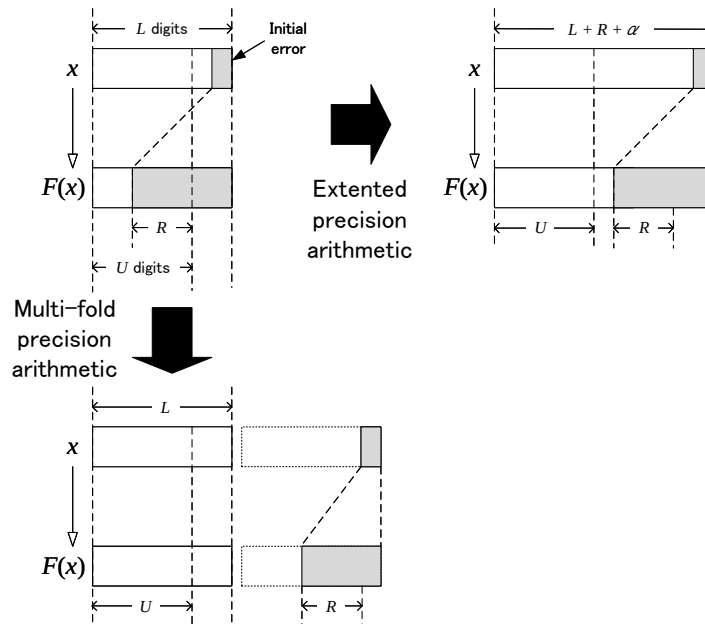


図 1.1 Extended precision and multi-fold precision arithmetics

現在のところ、数値計算で用いられるすべてのアルゴリズム、特に非線形計算を multi-fold 演算で適用できるわけではありません。したがって、multi-fold と多倍長精度計算を組み合わせる必要があり、常微分方程式の初期値問題を解く陽的補外法のような基本線形計算部分に multi-fold 計算を用いることで計算時間を削減します。

私たちがライブラリに組み込んだ行列乗算アルゴリズムである Ozaki scheme は、multi-fold 計算アプローチに基づく技術です。既存の binary32 および binary64 の xGEMM アルゴリズムの速度を活用することで、私たちのベンチマークおよび先行研究 [7][11] で明らかにされているように、特定の条件下で多倍長精度行列乗算を大幅に最適化します。私たちは、多倍長精度数値アルゴリズムの発展動向に従い、多倍長精度線形計算ライブラリの高速度化が、より広範な数値計算アルゴリズムの精度保証において引き続き重要なテーマであり続けると確信しています。

1.6 最適化された行列乗算のアルゴリズムと性能

私たちは、任意精度数値計算のための MPI サポート [4] を皮切りに、多倍長精度行列乗算の最適化に注力してきました。その後、現在のマルチコア CPU の普及を背景に、OpenMP を用いた並列化サポートによって多倍長精度行列乗算を高速化しました。多倍長精度計算に関する限り、Strassen や Winograd アルゴリズムのような分割統治法を組み込んだ他の同様の成果は得られていません。multi-component 方式に限られるものの、私たちの提案手法は AVX2 をサポートし、高速化のために OpenMP 化された唯一のアプローチです。しかしながら、現在のコードは OpenMP 高速化において 8 スレッドを超えると性能が向上せず、並列化性能をさらに改善するには抜本的な再構成が必要です。Algorithm 1 は Strassen 行列乗算です。

Algorithm 1 Strassen algorithm for matrix multiplication

Input: $A = [A_{ij}]_{i,j=1,2} \in \mathbb{R}^{m \times l}$, $A_{ij} \in \mathbb{R}^{m/2 \times l/2}$, $B = [A_{ij}]_{i,j=1,2} \in \mathbb{R}^{l \times n}$, $B_{ij} \in \mathbb{R}^{l/2 \times n/2}$

Output: $C := \text{Strassen}(A, B) = AB \in \mathbb{R}^{m \times n}$

if $m < m_0$ && $n < n_0$ **then**

$C := AB$

end if

$P_1 := \text{Strassen}(A_{11} + A_{22}, B_{11} + B_{22})$

$P_2 := \text{Strassen}(A_{21} + A_{22}, B_{11})$

$P_3 := \text{Strassen}(A_{11}, B_{12} - B_{22})$

$P_4 := \text{Strassen}(A_{22}, B_{21} - B_{11})$

$P_5 := \text{Strassen}(A_{11} + A_{12}, B_{22})$

$P_6 := \text{Strassen}(A_{21} - A_{11}, B_{11} + B_{12})$

$P_7 := \text{Strassen}(A_{12} - A_{22}, B_{21} + B_{22})$

$C_{11} := P_1 + P_4 - P_5 + P_7$; $C_{12} := P_3 + P_5$

$C_{21} := P_2 + P_4$; $C_{22} := P_1 + P_3 - P_2 + P_6$

$C := [C_{ij}]_{i,j=1,2}$

Ozaki scheme の有用性は、Mukunoki らによる float128 精度行列乗算の高速化の成功 [7] により、多倍長精度レベルにおいても明らかです。GCC がサポートする float128 演算は、加算と乗算において TD から QD 精度の性能を備えており、同様にこの精度範囲に対して十分であると期待されます。

Ozaki scheme は、行列を、より短い桁で表現される要素を持つ多数の行列に分割することで、性能の高速化と精度の向上を同時に達成することを目指すアルゴリズムです。誤差なし変換技術で用いられる “Split” 法

と同様に、このアプローチは最適化された短精度行列乗算 (xGEMM) 関数の速度を活用します。与えられた行列 $A \in \mathbb{R}^{m \times l}$ および $B \in \mathbb{R}^{l \times n}$ に対して、長い L ビット精度の行列積 $C := AB \in \mathbb{R}^{m \times n}$ を得るために、Algorithm 2 に示すように、 $D \in \mathbb{N}$ を短い S ビット精度行列 ($S \ll L$) の最大分割数として、 A と B は Ozaki scheme を用いて分割されます。特に記述のない計算には S ビット演算が用いられ、 L ビット演算は高精度演算が必要な箇所でのみ用いられます。

Algorithm 2 Ozaki scheme for multiple-precision matrix multiplication

Input: $A \in \mathbb{F}_{bL}^{m \times l}, B \in \mathbb{F}_{bL}^{l \times n}$
Output: $C \in \mathbb{F}_{bL}^{m \times n}$
 $A^{(S)} := A, B^{(S)} := B : A^{(S)} \in \mathbb{F}_{bS}^{m \times l}, B^{(S)} \in \mathbb{F}_{bS}^{l \times n}$
 $\mathbf{e} := [1 \ 1 \ \dots \ 1]^T \in \mathbb{F}_{bS}^l$
 $\alpha := 1$
while $\alpha < D$ **do**
 $\mu_A := [\max_{1 \leq p \leq l} |(A^{(S)})_{ip}|]_{i=1,2,\dots,m} \in \mathbb{F}_{bS}^m$
 $\mu_B := [\max_{1 \leq q \leq l} |(B^{(S)})_{qj}|]_{j=1,2,\dots,n} \in \mathbb{F}_{bS}^n$
 $\tau_A := [2^{\lceil \log_2((\mu_A)_i) \rceil + \lceil (S + \log_2(l))/2 \rceil}]_{i=1,2,\dots,m} \in \mathbb{F}_{bS}^m$
 $\tau_B := [2^{\lceil \log_2((\mu_B)_j) \rceil + \lceil (S + \log_2(l))/2 \rceil}]_{j=1,2,\dots,n} \in \mathbb{F}_{bS}^n$
 $S_A := \tau_A \mathbf{e}^T$
 $S_B := \mathbf{e} \tau_B^T$
 $A_\alpha := (A^{(S)} + S_A) - S_A : A_\alpha \in \mathbb{F}_{bS}^{m \times l}$
 $B_\alpha := (B^{(S)} + S_B) - S_B : B_\alpha \in \mathbb{F}_{bS}^{l \times n}$
 $A := A - A_\alpha, B := B - B_\alpha : L\text{-bit FP arithmetic}$
 $A^{(S)} := A, B^{(S)} := B$
 $\alpha := \alpha + 1$
end while
 $A_D := A^{(S)}, B_D := B^{(S)}$
 $C := O$
for $\alpha = 1, 2, \dots, D$ **do**
 for $\beta = 1, 2, \dots, D - \alpha + 1$ **do**
 $C_{\alpha\beta} := A_\alpha B_\beta$
 end for
 $C := C + \sum_{\beta=1}^{D-\alpha+1} C_{\alpha\beta} : L\text{-bit FP arithmetic}$
end for

図 1.2 は、 A と $B \in \mathbb{R}^{3 \times 3}$ を 3 つの短桁行列に分割した場合の Ozaki scheme の例を示しています。Ozaki scheme の最も重要な特徴は、行列 A と B が短精度に収まるように $A_1, A_2, A_3, B_1, B_2, B_3$ に分割され、それによって高速な低精度行列乗算における丸め誤差を回避できる点です。その結果として誤差なし行列積 $C_{ij} := A_i B_j (i, j = 1, 2, 3)$ を得ることができ、 $C := \sum_{i,j} C_{ij}$ という多倍長精度行列加算演算を用いて高精度な行列積 C を得ることができます。 A と B の分割数は有限ですが、ある精度のしきい値を保証するために必要な最小分割数を決定することは困難です。その結果、このアルゴリズムが他の乗算アルゴリズムよりも高速に実行できるかどうかを判断するには、ベンチマークテストを行う必要があります。

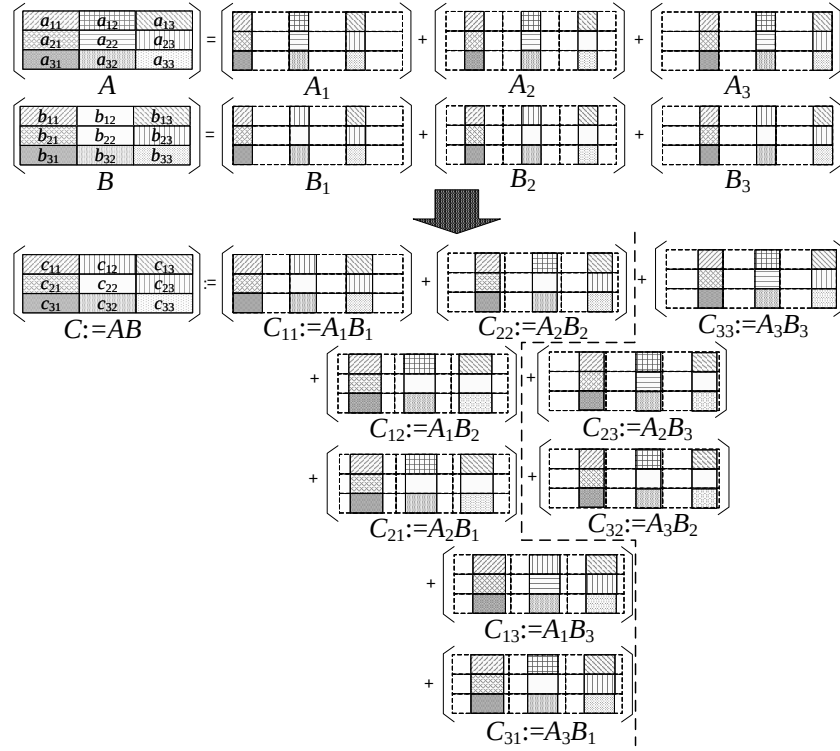


図 1.2 Matrix multiplication based on Ozaki scheme when the matrices are divided into three components

1.7 BNCmatmul のソフトウェア層

私たちはこれまで、multi-component 型固定精度 C++ クラスライブラリとして QD [1]、multi-component 型任意精度ライブラリとして CAMPARY [3]、そして GNU MP [2] の MPN カーネルに基づく MPFR [8] を用いてきました。MPLAPACK/MPBLAS [6] は、これらすべての信頼できる多倍長精度ライブラリを取り込み、LAPACK/BLAS [5] に対応する API を提供しています。私たちが開発した BNCmatmul は、ATLAS、OpenBLAS、Intel Math Kernel のように MPBLAS よりも優れた性能を得ることを目指しており、以下の特徴を持ちます。

- 中核となる関数とマクロは、C++ ではなく従来の ANSI C の流儀で書かれています。
- ネイティブな multi-component 型の DD、TD、QD 精度演算をサポートし、さらに x86-64 では AVX2 と AVX-512、AArch64 では NEON と SVE2 を用いて BLAS のようにベクトルおよび行列計算を高速化します。0.24 の時点で AVX-512 バックエンドは AVX2 バックエンドと同一のカーネルをすべて備え、スカラー版とのビット一致が検証されています (1.3 節)。
- mpreal クラスライブラリによるオーバーヘッドを回避するために、MPFR 関数を直接利用します。
- OpenMP に基づく共有メモリ型並列化を部分的にサポートします。
- 行列乗算の 3 つのアルゴリズム、すなわち単純な三重ループ、ブロック化 (タイリング)、および Strassen または Winograd アルゴリズムを実装しています。Ozaki scheme は試験的な最適化として利用でき

ます。

並列化を除いた BNCmatmul のソフトウェア層を 図 1.3 に示します。

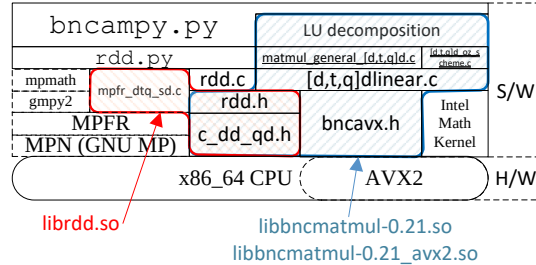


図 1.3 Software layer of BNCmatmul library

基本的な DD、TD、QD 精度演算はすべて `rdd.h` 内で定義されています。実装層となる `c_dd_*`/`c_td_*`/`c_qd_*` の C の inline 関数と、その上に構築されライブラリ全体から利用されるインタフェースである `rdd_*`/`RDD_*` マクロ・inline 関数から成ります (バージョン 0.23 までは inline 層は独立したヘッダ `c_dd_qd.h` にありましたが、`rdd.h` に吸収されました)。Python 環境を利用するために、`rdd.h` に基づく `rdd.c` が `librdd.so` としてコンパイルされます。Python 上の DD、TD、QD クラスは、`librdd.so` 内で定義された関数を用います。

同じく `rdd.h` 上に構築された BNCmatmul には、単純な三重ループ行列乗算よりも効率的な性能を提供するブロック行列乗算および Strassen 行列乗算が含まれています。これらは、`include/avx2/` 以下で定義された SIMD 化関数 — AVX2 用の `_bncavx_*.h` (`_m256d/_m256`) と AVX-512 用の `_bncavx*_avx512.h` (`_m512d/_m512`) で、いずれもアンブレラヘッダ `avx2/bncavx.h` 経由で取り込まれます — および、AArch64 では `include/neon/`・`include/sve2/` 以下の対応する関数を用いて高速化されています。LU 分解も同様に高速化されています。

BNCmatmul (図 1.3) で定義されたこれらの関数は、SIMD 化技術を用いない `libbncmatmul-0.xx.a/.so`、および AVX2・AVX-512・Arm NEON・Armv9-A SVE2 による SIMD 化をそれぞれ用いた `libbncmatmul-0.xx_avx2*_avx512*_neon*_sve2` から利用できます。これらのライブラリは同じ関数名を持つため、ユーザは BNCmatmul のライブラリを選択する際にコードを変更する必要はありません。SIMD バックエンドはリンク時にのみ選択されます。

1.8 バージョン履歴と Todo リスト

■バージョン履歴

Version 0.24: 2026-08-15 dtq-0.0.3 から純粋な ANSI C で移植した、証明付き branch-free FMA と FMA ベース初等関数:

1. 機械証明された誤差限界を持つ branch-free 融合積和カーネル `bnc_{dw,tw,qw}fma[f]` (`bncfma.h`)。NEON・SVE2 に加え、新たに AVX2/AVX-512 版 (`avx2/_bncavx_fma.h`) を追加。NEON 版・AVX2 版はスカラー認証版とのビット一致を検証済み (2.6 節)
2. DD/TD/QD 向け FMA 融合初等関数 `exp`・`expm1`・`log`・`log10`・`sin`・`cos`・`sincos` (DS/TS/QS は委譲)。従来の `c_dd_sin`・`c_td_*`・`c_qd_*` 等の空スタブ・未実装を置き換え。MPFR 基準で

精度検証済み、平均誤差は各型の 1ε 未満 (2.7 節)

3. limb-planar 配列に対する一括関数 `bnc_{dd,td,qd}_{exp,log,sin,cos}_array`。ビルドの SIMD 幅でブロック化され、スカラー関数とビット単位で一致 (2.7 節)
4. バグ修正: `const_dd_pi` が $\pi/16$ 、`const_td_pi/const_qd_pi` が $\pi/1024$ 、float 基の π 定数がゼロ、`c_dd_nint` が空スタブだった問題
5. div/sqrt-safe 版 FMA `bnc_{dw,tw,qw}fma[f]_safe` と FMA 駆動除算 `bnc_{dd,td,qd,ds,ts,qs}_div_fma` (`dtq` の `fma_div` 移植。BNC_USE_FMA_DIV で既定除算を切替可能)。double 基の除算は同精度のまま 1.2 倍 / 2.8 倍 / 2.2 倍 (DD/TD/QD) 高速 (2.7.4 節)
6. AVX-512 バックエンドを AVX2 と同等に整備: ビルド対象ソース中 426 個の AVX2 構文のうち 411 個が対応する AVX-512 の腕を備え、`avx512` ビルドに必要な ISA は AVX-512F + FMA のみ。serial・AVX2・AVX-512 の各ビルドが要素毎カーネルおよび疎行列カーネルでビット一致することを検証済み (1.3 節)
7. Python/ctypes 向けの DLL 対応関数エクスポート API `rdd_*/rtd_*/rqd_*/rds_*/rts_*/rqs_*` (99 シンボル、`src/rdtq_func.c` + `include/rdtq_func.h`)。librdtq ビルドターゲットと `python/test_rdtq.py` ドライバ付き (2.8 節)

Version 0.23: 2026-03-16 Arm Neon CPU 上で高速化された BNCmatmul がリリースされました。

Version 0.22: 2024-05-31 いくつかの関数が追加されました。

1. 複素数演算と線形計算
2. 各種関数 (アクセサ、初等関数、乱数関数)
3. 疎行列と SpMV

Version 0.21: 2023-05-31 DD、TD、QD、MPFR 精度の実数 BLAS 関数を含むソースを <https://github.com/tkouya/bncmatmul/blob/main/bncmatmul-0.21.tar.bz2> で 2 回目に公開しました。

Version 0.20: 2016-06-08 MPFR に基づく任意精度行列乗算のみを提供するソースを <https://na-inet.jp/na/bnc/bncmatmul-0.2.tar.bz2> で初めて公開しました。

■ Todo リスト

1. 本マニュアルにおいて、BNCmatmul と MPLAPACK/BLAS を用いたサンプルソースをより多く示すこと
2. 本ライブラリの使用方法と、さまざまな状況への適用方法に関する説明を完成させること

第 2 章

基本データ型と算術演算

本章では、BNCmatmul で定義されているすべての基本データ型と関数を提示・説明し、多倍長精度計算を含むプログラムを構築するためのそれらの使い方を示します。

第 1 章で示したとおり、すべての基本データ型と関数は C++ 流ではなく ANSI C で記述されています。

2.1 c_dd_*算術層 (rdd.h)

バージョン 0.23 まで、本節のマクロと inline 関数は独立したヘッダ `c_dd_qd.h` (単精度系は `c_ds_qs.h`) で提供されていました。0.24 からは両ヘッダはそれぞれ `rdd.h`・`rds.h` に吸収され、多倍長成分ファミリの算術ヘッダはこの 2 つに一本化されています。`rdd.h` を include すれば以下のすべてが利用できます。

(Macro) `DFMA(a, b, c)` 倍精度で $a \times b + c$ を返します

(Macro) `SFMA(a, b, c)` 単精度で $a \times b + c$ を返します

(Macro) `QD_FMA(a, b, c)` 次と同じです: `DFMA(a, b, c)`

(Macro) `QD_FMS(a, b, c)` 倍精度で $a \times b - c$ を返します

(Macro) `DDSIZE` は 2 で、DD 内の binary64 (double) 変数の個数です

(Macro) `TDSIZE` は 3 で、TD 内の binary64 変数の個数です

(Macro) `QDSIZE` は 4 で、QD 内の binary64 変数の個数です

2.1.1 DD (double-double) : 106 ビット (53 ビット × 2) 精度浮動小数点数

(Macro) `DD_HI(a)` is `a[0]`.

(Macro) `DD_LOW(a)` is `a[1]`.

(Macro) `DD__TRUE` is 1UL.

(Macro) `TD__TRUE`

(Macro) `QD__TRUE`

(Macro) `DD__FALSE` is 0UL.

(Macro) `TD__FALSE`

(Macro) `QD__FALSE`

(Macro) `DD_ISNAN(a)` `a` が NaN を含むかどうかを判定します。

(Macro) TD_ISNAN(a)
 (Macro) QD_ISNAN(a)
 (Macro) DD_ISINF(a) a が INF を含むかどうかを判定します。
 (Macro) TD_ISINF(a)
 (Macro) QD_ISINF(a)
 (Macro) DD_ISZERO(a) a がゼロかどうかを判定します。
 (Macro) TD_ISZERO(a)
 (Macro) QD_ISZERO(a)
 (Macro) DD_ISONE(a) a が 1 かどうかを判定します。
 (Macro) TD_ISONE(a)
 (Macro) QD_ISONE(a)
 (Macro) DD_ISNEGATIVE(a) $a < 0$ かどうかを判定します。
 (Macro) TD_ISNEGATIVE(a)
 (Macro) QD_ISNEGATIVE(a)
 (Macro) DD_NAN is FP_NAN.
 (Macro) TD_NAN
 (Macro) QD_NAN

2.2 誤差なし変換 (error-free transformation)

- `double quick_two_sum(double a, double b, double * err) ...` $\text{fl}(a + b)$ と $\text{err}(a + b)$ を計算します。
 $|a| \geq |b|$ を仮定します。
- `double quick_two_diff(double a, double b, double * err) ...` $\text{fl}(a - b)$ と $\text{err}(a - b)$ を計算します。
 $|a| \geq |b|$ を仮定します。
- `double two_sum(double a, double b, double * err) ...` $\text{fl}(a + b)$ と $\text{err}(a + b)$ を計算します。
- `double two_diff(double a, double b, double * err) ...` $\text{fl}(a - b)$ と $\text{err}(a - b)$ を計算します。
- `void split(double a, double *hi, double *lo) ...` a を誤差なしで hi と lo に分割します。
- `double two_prod(double a, double b, double * err) ...` $\text{fl}(ab)$ と $\text{err}(ab)$ を計算します。
- `double two_sqr(double a, double * err) ...` $\text{fl}(a^2)$ と $\text{err}(a^2)$ を計算します。上記の方法より高速です。

2.3 double-double 精度算術演算

- `dd_bool dd_is_zero(const double * x) ...` $*x$ がゼロかどうかを判定します。
- `dd_bool dd_is_one(const double * x) ...` $*x$ が 1 かどうかを判定します。
- `void c_dd_set(const double * x, double *ret) ...` $ret := x$
- `void c_dd_set_dd_d(const double x, double * ret) ...` $ret := (\text{double})x$
- `void c_dd_set0(double * ret) ...` $ret := 0$
- `void c_dd_setnan(double * ret) ...` $ret := NaN$

- `void c_dd_neg(const double * a, double * b) ...  $b := -a$`
- `dd_bool dd_is_positive(const double * x) ...  $*x > 0$  ?`
- `dd_bool dd_is_negative(const double * x) ...  $*x < 0$  ?`
- `void c_dd_comp(const double * a, const double * b, int * result) ... a と b を比較し、 $a == b$  なら 0、 $a > b$  なら +1、 $a < b$  なら -1 を格納します。`
- `void c_dd_comp_dd_d(const double * a, double b, int * result)`
- `void c_dd_comp_d_dd(double a, const double * b, int * result)`
- `void c_dd_pi(double * a)`
- `void c_d_add(double a, double b, double * c) ... double-double := double + double`
- `void c_dd_add(const double * a, const double * b, double * c) ...  $c := a + b$`
- `void c_dd_add_sloppy(const double * a, const double * b, double * c)`
- `void c_dd_add_dd_d(const double * a, double b, double * c)`
- `void c_dd_add_d_dd(double a, const double * b, double * c)`
- `void c_d_sub(double a, double b, double * c) ... double-double = double - double`
- `void c_dd_sub(const double * a, const double * b, double * c) ...  $c := a - b$`
- `void c_dd_sub_sloppy(const double * a, const double * b, double * c)`
- `void c_dd_sub_dd_d(const double * a, double b, double * c)`
- `void c_dd_sub_d_dd(double a, const double * b, double * c)`
- `void c_d_mul(double a, double b, double * c) ... double-double := double * double`
- `void c_dd_mul(const double * a, const double * b, double * c)`
- `void c_dd_mul_dd_d(const double * a, double b, double * c)`
- `void c_dd_mul_d_dd(double a, const double * b, double * c)`
- `void c_d_div(double a, double b, double * c) ...  $c := a/b$`
- `void c_dd_div(const double * a, const double * b, double * c) ...  $c := a/b$`
- `void c_dd_sloppy_div(double * a, double * b, double * c)`
- `void c_dd_div_dd_d(const double * a, double b, double * c)`
- `void c_dd_div_d_dd(double a, const double * b, double * c)`
- `void c_dd_copy(const double * a, double * b)  $b := a$`
- `void c_dd_copy_d(double a, double * b)`
- `void c_d_sqr(double a, double * b) ...  $b := a^2$`
- `void c_dd_sqr_d(double a, double * ret)`
- `void c_dd_sqr(const double * a, double * b)`
- `void c_dd_sqrt(const double * a, double * b) ...  $b := \sqrt{a}$`
- `void c_dd_abs(const double * a, double * b) ...  $b := |a|$`
- `void c_dd_floor(const double * a, double * b) ...  $b := \text{floor}(a)$`
- `void c_dd_ceil(const double * a, double * b) ...  $b := \text{ceil}(a)$`

2.4 quadruple-double 精度算術演算

- `void c_qd_copy(const double * a, double * b) ...  $b := a$`
- `void c_qd_copy_dd(const double * a, double * b)`
- `void c_qd_copy_d(double a, double * b)`
- `void quick_renorm(double * c0, double * c1, double * c2, double * c3, double * c4) ... Renormalize c0 ...`
- `void renorm(double * c0, double * c1, double * c2, double * c3)`
- `void renorm4(double * c0, double * c1, double * c2, double * c3, double * c4)`
- `void three_sum(double * a, double * b, double * c)`
- `void three_sum2(double * a, double * b, double * c)`
- `void c_qd_add_qd_dd(const double * a, const double * b, double * c) ...  $c := a + b$`
- `void c_qd_add(const double * a, const double * b, double * c)`
- `void c_qd_add_sloppy(const double * a, const double * b, double * c) ... 精度はやや劣るものの高速な加算  $c := a + b$`
- `void c_td_addq(const double * a, const double * b, double * c) ... quad-double 方式に基づく triple double 加算`
- `void c_qd_selfadd(const double * a, double * b) ...  $b := b + a$`
- `void c_qd_selfadd_dd(const double * a, double * b)`
- `void c_qd_selfadd_d(double a, double * b)`
- `void c_qd_neg(const double * a, double * b) ...  $b := -a$`
- `void c_qd_neg_dd(const double * a, double * b)`
- `void c_qd_neg_d(const double a, double * b)`
- `void c_qd_sub(const double * a, const double * b, double * c) ...  $c := a - b$`
- `void c_qd_sub_qd_dd(const double * a, const double * b, double * c)`
- `void c_qd_sub_dd_qd(const double * a, const double * b, double * c)`
- `void c_qd_sub_qd_d(const double * a, double b, double * c)`
- `void c_qd_sub_d_qd(double a, const double * b, double * c)`
- `void c_qd_selfsub(const double * a, double * b) ...  $b := b + (-a) = b - a$`
- `void c_qd_selfsub_dd(const double * a, double * b)`
- `void c_qd_selfsub_d(double a, double * b)`
- `void c_qd_mul(const double * a, const double * b, double * c) ...  $c := a \times b$`
- `void c_qd_mul_qd_dd(const double * a, const double * b, double * c)`
- `void c_qd_mul_dd_qd(const double * a, const double * b, double * c)`
- `void c_qd_mul_qd_d(const double * a, double b, double * c)`
- `void c_qd_mul_d_qd(double a, const double * b, double * c)`
- `void c_qd_mul_sloppy(const double * a, const double * b, double * c)`
- `void c_qd_sqr(const double * a, double * c) ...  $c = a^2$`
- `void c_qd_selfmul(const double * a, double * b) ...  $b := a * b$`

- void c_qd_selfmul_dd(const double * a, double * b)
- void c_qd_selfmul_d(double a, double * b)
- void c_qd_div_accurate(const double * a, const double * b, double * c) ... $c := a/b$
- void c_qd_div_sloppy(const double * a, const double * b, double * c) ... 精度はやや劣るものの高速な除算 (デフォルト)

(Macro) USE_QD_DIV_ACCURATE() ... 真の場合、高精度な qd 除算を使用します

- void c_qd_div_qd_dd(const double * a, const double * b, double * c)
- void c_qd_div_qd_d(const double * a, double b, double * c)
- void c_qd_div_d_qd(double a, const double * b, double * c)
- void c_qd_selfdiv(const double * a, double * b) ... $b := b/a$
- void c_qd_selfdiv_dd(const double * a, double * b)
- void c_qd_selfdiv_d(double a, double * b)
- void c_qd_mul_pwr2(const double * a, double b, double * c) ... $c := (a[0]*b, a[1]*b, a[2]*b, a[3]*b)$
- void c_qd_set(const double * x, double * qdval) ... $qdval := c$
- void c_qd_set0double *qdval() ... $qdval := 0$
- void c_qd_sqrt(const double * a, double * b) ... $b := \sqrt{a}$
- void c_qd_abs(const double * a, double * b) ... $b := |a|$

(Macro) nint(a) ... a を整数に丸めます

- void c_qd_nint(const double * a, double * b)
- void c_qd_floor(const double * a, double * b) ... $b := \text{floor}(a)$
- void c_qd_ceil(const double * a, double * b) ... $b := \text{ceil}(a)$
- void c_qd_comp(const double * a, const double * b, int * result) ... $a < b$ なら -1、 $a == b$ なら 0、 $a > b$ なら +1 を返します。
- void c_qd_comp_qd_d(const double * a, double b, int * result)
- void c_qd_comp_d_qd(double a, const double * b, int * result)

2.5 triple-double 精度算術演算

- void c_td_copy(const double * a, double * b) ... $b := a$
- void c_qd_copy_td(const double * a, double * c)
- void c_td_copy_qd(const double * a, double * b)
- void c_td_copy_dd(const double * a, double * b)
- void c_td_copy_d(double a, double * b)
- void vec_sum(double *e, const double * x, int n) ... $e[n] := \text{vec_sum}(x[n])$
- void vseb(double * y, int ny, const double * e, int ne) ... $y[n] := \text{vec_sum_err_branch}(vseb)(k)(e[n])$
- void c_to_td(double * r, double a, double b, double c) ... $r[3] := \text{to_td}(a, b, c)$
- void merge(double * c, double * a, int na, double * b, int nb) ... $a[na]$ と $b[nb]$ を $c[na + nb]$ にマージします
- void c_td_add(double * a, double * b, double * c) ... $c := a + b$

- void c_td_add_td_d(double * a, double b, double * c)
- void c_td_neg(const double * a, double * c) ... $c := -a$
//
- void c_td_sub(double * c, double * a, double * b) ... $c := a - b$
- void c_td_sub(double * a, double * b, double * c)
- void c_td_subq(const double * a, const double * b, double * c)
- void c_td_sub_d_td(double a, double * b, double * c) //
- void c_td_sub_td_d(double * c, double * a, double b)
- void c_td_sub_td_d(double * a, double b, double * c)
- void c_td_mul_accurate(double * a, double * b, double * c) ... $c := a \times b$
- void c_td_mul_sloppy(double * a, double * b, double * c) ... 精度はやや劣るものの高速 $c := a \times b$

(Macro) USE_TD_MUL_ACCURATE() ... 真の場合、高精度な td 乗算を使用します

- void c_td_mul_dd_td_sloppy(double * a, double * b, double * c)
- void c_td_mul_dd_td_accurate(double * a, double * b, double * c)

(Macro) USE_TD_MUL_DD_TD_ACCURATE() ... 真の場合、高精度な td-dd 乗算を使用します

- void c_td_mul_d_td(const double a, const double * b, double * c)
- void c_td_mul_td_d(const double * a, const double b, double * c)
- void c_td_abs(const double * a, double * b) ... $b := |a|$
- void c_td_comp(const double * a, const double * b, int * result) ... $a < b$ なら -1、 $a == b$ なら 0、 $a > b$ なら +1 を返します。
- void c_td_comp_td_d(const double * a, double b, int * result)
- void c_td_comp_d_td(double a, const double * b, int * result)
- void c_td_2mtw_dd_td(double a[DDSIZE], double b[TDSIZE], double c[TDSIZE])

(Macro) ONE_P_2DBL_EPS() ... $1 + 2 \times \text{DBL_EPSILON} = (1.00000000000000044e + 00)$

(Macro) ONE_M_2DBL_EPS() ... $1 - 2 \times \text{DBL_EPSILON} = (9.99999999999999556e - 01)$

- void c_td_reci(double * a, double * c)
- void c_td_divt(double * a, double * b, double * c)

(Macro) c_td_div() ... 次と同じです: c_td_divtq

- void c_td_divq(const double * a, const double * b, double * c) ... quad-double 除算に基づく td 除算です。
- void c_td_div_td_d(double * a, double b, double * c)
- void c_td_sqrt(double * a, double * c) ... $c := \sqrt{a}$
- void c_td_sqrt_d(double a, double * c)
- void c_td_sqr(double * a, double * c) ... $c := a^2$

2.6 Branch-free 融合積和演算 (bncfma.h)

バージョン 0.24 では、DD/TD/QD (double 基) および DS/TS/QS (float 基) 算術向けの、証明付き branch-free 融合積和演算 (FMA) カーネル $z := x \times y + c$ を提供します。T. Kouya, “Performance evaluation

of branch-free fused multiply-add algorithms for multi-component-type multiple-precision floating-point arithmetic” (arXiv:2607.11391) の参照実装から演算単位で忠実に移植したもので、各カーネルは FPAN-Verifier で認証されたネットリストに一致し、機械証明された誤差限界 ($u = 2^{-p}$) を持ちます。

	演算数	認証誤差限界
DW (DD/DS)	17	$ z - (xy + c) \leq 34u^2(xy + c)$
TW (TD/TS)	66	$ z - (xy + c) \leq 184u^3(xy + c)$
QW (QD/QS)	146	$ z - (xy + c) \leq 812u^4(xy + c)$

- `void bnc_dwfma(double z[2], const double x[2], const double y[2], const double c[2]) ... z := x × y + c (DD)`
- `void bnc_twfma(double z[3], const double x[3], const double y[3], const double c[3]) ... z := x × y + c (TD)`
- `void bnc_qwfma(double z[4], const double x[4], const double y[4], const double c[4]) ... z := x × y + c (QD)`
- `void bnc_dwfmaf(float z[2], const float x[2], const float y[2], const float c[2]) ... z := x × y + c (DS)`
- `void bnc_twfmaf(float z[3], const float x[3], const float y[3], const float c[3]) ... z := x × y + c (TS)`
- `void bnc_qwfmaf(float z[4], const float x[4], const float y[4], const float c[4]) ... z := x × y + c (QS)`

スカラーカーネルは `bncfma_d.h` / `bncfma_f.h` (アンブレラヘッダ `bncfma.h`) にあります。同一仕様の SIMD 版を全バックエンドに提供します: `neon/_bncneon_fma.h` (`_bncneon_{dw,tw,qw}fma[f]`、`float64x2_t/float32x4_t`)、`sve2/_bncsve2_fma.h`、および 0.24 で新規追加の `avx2/_bncavx_fma.h` (AVX2 版 `_bncavx2_{dw,tw,qw}fma[f]` (`_m256d/_m256`) と AVX-512 版 `_bncavx512_{dw,tw,qw}fma[f]` (`_m512d/_m512`))。NEON 版と AVX2 版はランダムな被演算数に対してスカラー認証版と全 limb ビット一致することを確認済みです。AVX-512 ビルドについては、代わりに 1.3 節のライブラリ全体のビット一致検査でカーネルレベルの一致を確認しています。いずれも `-ffp-contract=off` が必須です (BNCmatmul の全ビルドで設定済み)。コンパイラが $a \times b + c$ を勝手に縮約すると、内部の誤差なし変換が誤差なしでなくなるためです。

■**div/sqrt-safe 変種** 上の認証誤差限界は、被演算数が正規化された (non-overlapping な) 多倍長展開であることを前提に機械証明されています。除算・平方根の Newton 反復に現れる残差は non-overlapping とは限らないため、どのゲートについても FastTwoSum (`quick_two_sum`) の前提条件を主張できません — 前提条件が破れた FastTwoSum は $s + e = a + b$ すら満たしません。このような被演算数向けに、*safe* 変種 (dtq の `dw_fma_safe` / `tw_fma_safe` / `qw_fma_safe` の移植。第 2 被乗数はスカラー) はすべての FastTwoSum を完全な TwoSum に置き換えます (DW: 20 flops)。safe 変種が標準版より速くなることはなく、また精度が落ちることもありません。FMA 駆動除算 (2.7 節) から使用されます。

- `void bnc_dwfma_safe(double z[2], const double x[2], double y, const double c[2]) ... z := x × y + c (DD、スカラー y)`

- `void bnc_twffma_safe(double z[3], const double x[3], double y, const double c[3]) ... (TD)`
- `void bnc_qwffma_safe(double z[4], const double x[4], double y, const double c[4]) ... (QD)`
- `void bnc_dwffmaf_safe(float z[2], const float x[2], float y, const float c[2]) ... (DS)`
- `void bnc_twffmaf_safe(float z[3], const float x[3], float y, const float c[3]) ... (TS)`
- `void bnc_qwffmaf_safe(float z[4], const float x[4], float y, const float c[4]) ... (QS)`

ライブラリのビルド時に `BNC_USE_NEW_FMA` を定義すると、`rdd_fma/rtd_fma/rqd_fma`（および `rds/rts/rqs` 系）と、`serial`・`NEON`・`SVE2` パスの `AXPY`・`内積`・`GEMV`・`GEMM`・`多項式評価`の呼び出し箇所がこれらのカーネルに切り替わります。マクロ未定義の場合、ライブラリはビット単位で従来と同一です。

2.7 FMA ベース初等関数 (bncelem.h)

バージョン 0.24 では、`dtq-0.0.3` の FMA 融合初等関数を純粋な ANSI C に移植しました。適切な引数簡約 ($\ln 2$ 簡約+反復自乗、または 2 段の $\exp$ 表；三角関数は $2\pi \rightarrow \pi/2 \rightarrow \pi/16$ または $\pi/1024$ 簡約+256 エントリ表) の後、Taylor 級数の各項を乗算・加算の分離実行ではなく、証明付き branch-free FMA（2.6 節）1 回で積算します。係数表 (`bncelem_tables.h`) は `dtq-0.0.3` のソースから機械的に抽出したもので、TD 用の表は QD 表に対する `dtq` の `to_td_real()` 折り畳みを厳密に再現しています。精度は MPFR (512 ビット) を基準に検証済みで、平均相対誤差は各型の 1ε 未満 ($\varepsilon_{DD} = 2^{-104}$ 、 $\varepsilon_{TD} = 2^{-157}$ 、 $\varepsilon_{QD} = 2^{-209}$) であり、`dtq` 本体と同等です。

2.7.1 スカラー関数

各 `double` 基型に対して以下の関数を提供します (`include/bncelem_dd.h` / `_td.h` / `_qd.h`、アンブレラヘッダは `bncelem.h`)。接頭辞は `bnc_dd_`、`bnc_td_`、`bnc_qd_`、limb 数 K はそれぞれ 2、3、4 です：

- `void bnc_dd_exp(const double * x, double * ret) ...  $ret := \exp(x)$`
- `void bnc_dd_expm1(const double * x, double * ret) ...  $ret := \exp(x) - 1$  (小さい  $|x|$  で高精度)`
- `void bnc_dd_log(const double * x, double * ret) ...  $ret := \log_e(x)$  ( $\exp$  に対する Newton 法 1 回)`
- `void bnc_dd_log10(const double * x, double * ret) ...  $ret := \log_{10}(x)$`
- `void bnc_dd_sin(const double * x, double * ret) ...  $ret := \sin(x)$`
- `void bnc_dd_cos(const double * x, double * ret) ...  $ret := \cos(x)$`
- `void bnc_dd_sincos(const double * x, double * sin_x, double * cos_x) ...  $\sin(x)$  と  $\cos(x)$  を同時に計算します`
- `void bnc_dd_nint(const double * x, double * ret) ... 最近接整数`

同じ関数群が 3-limb/4-limb 配列版の `bnc_td_*`・`bnc_qd_*` として存在します。`float` 基型は `dtq` と同様に `double` 基実装へ委譲します (`bncelem_f.h`) : `bnc_ds_*` $\rightarrow$ DD、`bnc_ts_*` $\rightarrow$ TD、`bnc_qs_*` $\rightarrow$ QD。limb 変換は双方向とも厳密です。

2.7.2 c_dd_*/c_td_*/c_qd_*インタフェースへの接続

バージョン 0.23 までは、c_dd_qd.h と c_ds_qs.h で宣言されている初等関数の大半は本体が #if 0 で無効化された「空スタブ」でした (c_dd_sin・c_dd_cos・c_dd_tan・c_dd_sincos・c_dd_nint、および c_qd_*・c_qs_*・c_ds_* の全初等関数)。また rdd.h/rds.h から参照される c_td_*/c_ts_* 初等関数は実装自体が存在しませんでした。0.24 ではこれらすべてが対応する bnc_* 実装へ委譲されます。従来の動作していた実装は比較用に c_dd_exp_orig (引数簡約なしの素朴 Taylor 級数)、c_dd_log_orig、c_dd_log10_orig の名前が残してあります。新しい c_dd_exp は c_dd_exp_orig の約 10 倍、c_dd_log は c_dd_log_orig の約 5.7 倍高速です (Arm Cortex-X925、gcc 13.3)。

バージョン 0.24 では、スタブに隠れていた定数の誤りも修正しました: const_dd_pi は $\pi/16$ 、const_td_pi と const_qd_pi は $\pi/1024$ の値を保持しており、float 基の const_ds_pi/const_ts_pi/const_qs_pi はすべてゼロでした。c_dd_pi() 等は正しく丸められた π の limb 値を返すようになりました。

2.7.3 要素毎の一括関数 (bncelem_vector.h)

ベクトルの全要素へ初等関数を適用するために、src/elem_vector.c は limb-planar (SoA) 配列に対する一括版を提供します。レイアウトは ddvector/tdvector/qdvector と同一のため、vec->element をそのまま渡せます。配列は _BNC_D_WIDTH 要素のブロック単位で走査されます。この値は AVX-512 ビルドで 8、AVX2 ビルドで 4、NEON/SVE2 で 2、それ以外で 1 であり、各要素まわりの limb の gather/scatter がベクトル化され、ステージング用バッファがレジスタに留まります。要素ごとのカーネル自体はスカラーの bnc_dd_*/bnc_td_*/bnc_qd_* 関数のままです。+、-、×、÷ の誤差なし変換カーネルとは異なり、これらは引数の縮約・テーブル参照・Newton 補正が分岐を含むため、レーン一様な SIMD 形に置き換えられません。したがって結果は各要素にスカラー関数を適用した場合とビット単位で一致し、python/test_rdtq.py がこれを検査します。

- void bnc_dd_exp_array(long n, double * const ret[2], double * const x[2]) $\dots ret_i := \exp(x_i), i = 0, \dots, n-1$
- void bnc_dd_log_array(long n, double * const ret[2], double * const x[2])
- void bnc_dd_sin_array(long n, double * const ret[2], double * const x[2])
- void bnc_dd_cos_array(long n, double * const ret[2], double * const x[2])
- void bnc_td_exp_array(long n, double * const ret[3], double * const x[3]) (log/sin/cos も同様)
- void bnc_qd_exp_array(long n, double * const ret[4], double * const x[4]) (log/sin/cos も同様)

2.7.4 FMA 駆動除算

dtq-0.0.3 の除算 (fma_div) も移植しました。従来の accurate 除算と同じ筆算型の補正列ですが、各残差 $r \leftarrow r - q \cdot b$ を乗算+減算の代わりに 1 回の融合 safe FMA (2.6 節) で計算するため、ステップごとに正規化が 1 回減ります。MPFR (512 ビット) 基準での実測では精度クラスは不変 (DD/TD/QD で

$\leq 0.5\varepsilon$ 、DS/TS/QS で $\leq 2.4\varepsilon$) のまま、double 基の除算は Arm Cortex-X925 上で `c_dd_div / c_td_divq / c_qd_div_sloppy` に比べ 1.2 倍 (DD)・2.8 倍 (TD)・2.2 倍 (QD) 高速です (`bench/fma/test_fma_safe.c` と `bench/fma/results/test_fma_safe.txt` を参照)。

- `void bnc_dd_div_fma(const double * a, const double * b, double * ret) ... ret := a/b (DD)`
- `void bnc_td_div_fma(const double * a, const double * b, double * ret) ... (TD)`
- `void bnc_qd_div_fma(const double * a, const double * b, double * ret) ... (QD)`
- `void bnc_ds_div_fma(const float * a, const float * b, float * ret) ... (DS。bnc_ts_div_fma / bnc_qs_div_fma も同様)`

ビルド時に `BNC_USE_FMA_DIV` を定義すると、既定の除算エントリポイント `c_dd_div・c_td_div・c_qd_div・c_ds_div・c_ts_div・c_qs_div` (および `rdd.h / rds.h` の `RTD_DIV / RTS_DIV` マクロ) がこれらの関数に切り替わります。従来の DD/DS 版の本体は `c_dd_div_orig / c_ds_div_orig` として残ります。マクロ未定義ならライブラリはビット単位で従来と同一です。なお、dtq 本体で safe FMA を呼ぶのは double 基の `fma_div` のみですが、BNCmatmul の float 基移植では、dtq が double 基について記した根拠に従い safe 変種を使用しています。

2.7.5 精度・性能ベンチマーク

精度・性能ベンチマーク (MPFR 基準) は `bench/elem/` にあり、移植作業時の実測結果は `bench/elem/results/` に保存されています。

2.8 DLL / Python 向け関数エクスポート API (`rdtq_func.h`)

`rdd.h / rds.h` の `rdd_* / rtd_* / rqd_*` (および `rds_* / rts_* / rqs_*`) インタフェースはマクロと static inline 関数であり、C の翻訳単位内にのみ存在してシンボルをエクスポートしません。Python/ctypes のような共有ライブラリ (DLL) 利用者のために、`src/rdtq_func.c` が dtq-0.0.3 移植と 1 対 1 に対応する実体シンボル (99 関数) をコンパイルします (プロトタイプは `include/rdtq_func.h`。Windows の `__declspec(dllexport)` に備えた `BNC_API` 修飾マクロ付き)。引数順は `rdd.h` の慣習に従い、結果が先頭です。

各型 $XX \in \{rdd, rtd, rqd\}$ (double の limb) および $\{rds, rts, rqs\}$ (float の limb) について、以下がエクスポートされます:

- `void rdd_fma(double * ret, const double * a, const double * b, const double * c) ... ret := a × b + c` (証明付き branch-free FMA)
- `void rdd_fma_safe(double * ret, const double * a, double b, const double * c) ... div/sqrt-safe FMA (スカラー b)`
- `void rdd_div_fma(double * ret, const double * a, const double * b) ... ret := a/b`
- `void rdd_exp(double * ret, const double * x) ... expm1・log・log10・sin・cos・tan も同様`
- `void rdd_sincos(double * sin_ret, double * cos_ret, const double * x)`
- `void rdd_nint(double * ret, const double * x)` (double 基の型のみ)

SIMD ベクトル化された一括関数 `bnc_XX_yy_array` (2.7 節) はもともと実体シンボルであり、同様に Python から呼び出せます。Python 向け共有ライブラリはレガシーターゲット `make -C src -f Makefile.legacy librdtq` (または `librdtq_neon`) でビルドされ、`rdtq_func.c + elem_vector.c` から `python/librdtq.so` が生成されます。`python/test_rdtq.py` は全 6 精度ファミリと一括関数をカバーする ctypes テストドライバです。C 利用者への注意: `USE_RDD_FUNCTIONS` 等を先に定義しない限り、`rdtq_func.h` を `rdd.h / rds.h` と同一翻訳単位で include しないでください。小文字のマクロ別名がプロトタイプを壊すためです (ヘッダは `#error` でこれを強制します)。

2.9 rdd.h

```
// ddfloat, tdfloat, qdfloat
typedef struct { double val[DDSIZE]; } ddfloat; // 53 * 2 = 106
typedef struct { double val[TDSIZE]; } tdfloat; // 53 * 3 = 159
typedef struct { double val[QDSIZE]; } qdfloat; // 53 * 4 = 212
```

(Macro) `SET0_DD(val) ... val := 0 val[0] = (double)0.0; val[1] = (double)0.0;`

(Macro) `SET0_TD(val)`

(Macro) `SET0_QD(val)`

- `void rdd_out_str_base(FILE *fp, int base, int length, double val[DDSIZE]) ... DD の出力 (改行を付加しません)`
- `int rdd_cmp(double a[DDSIZE], double b[DDSIZE]) ... a < b なら -1、a == b なら 0、a < b なら -1 を返します。`
- `int rdd_cmp_d(double a[DDSIZE], double b)`
- `void rdd_sqrt_d(double ret[DDSIZE], double a) ... return  $\sqrt{ret}$`
- `void rdd_fma(double ret[DDSIZE], double a[DDSIZE], double b[DDSIZE], double c[DDSIZE]) ...  $ret := a \times b + c$`
- `void rdd_pow(double ret[DDSIZE], double base[DDSIZE], double power[DDSIZE]) ...  $ret := base^{power}$`
- `void rqd_out_str_base(FILE *fp, int base, int length, double val[QDSIZE]) ... 出力します (改行を付加しません)`
- `int rqd_cmp(double a[QDSIZE], double b[QDSIZE]) ... a < b なら -1、a == b なら 0、a < b なら -1 を返します。`
- `int rqd_cmp_d(double a[QDSIZE], double b)`
- `void rqd_sqrt_d(double ret[QDSIZE], double a) ... return  $\sqrt{ret}$`
- `void rqd_fma(double ret[QDSIZE], double a[QDSIZE], double b[QDSIZE], double c[QDSIZE]) ...  $ret := a \times b + c$`
- `void rqd_pow(double ret[QDSIZE], double base[QDSIZE], double power[QDSIZE]) ...  $ret := base^{power}$`
- `void rtd_out_str_base(FILE *fp, int base, int length, double val[TDSIZE])`
- `int rtd_cmp(double a[TDSIZE], double b[TDSIZE]) ... a < b なら -1、a == b なら 0、a < b なら`

-1 を返します。

- `int rtd_cmp_d(double a[TDSIZE], double b)`
- `void rtd_sqrt_d(double ret[TDSIZE], double a) ... return  $\sqrt{ret}$`
- `void rtd_fma(double ret[TDSIZE], double a[TDSIZE], double b[TDSIZE], double c[TDSIZE]) ...`
 $ret := a \times b + c$
- `void rtd_pow(double ret[TDSIZE], double base[TDSIZE], double power[TDSIZE]) ...`
 $ret := base^{\text{power}}$

(Macro) `set0_dd(val) val := 0`

(Macro) `rdd_set0(val)`

(Macro) `rdd_add(ret, a, b) ret := a + b`

(Macro) `rdd_sub(ret, a, b) ret := a - b`

(Macro) `rdd_mul(ret, a, b) ret := a  $\times$  b`

(Macro) `rdd_div(ret, a, b) ret := a/b`

(Macro) `rdd_sqrt(ret, a) ret :=  $\sqrt{a}$`

(Macro) `rdd_sqrt_d(ret, a)`

(Macro) `rdd_sqrt_ui(ret, a)`

(Macro) `rdd_get_d(a)`

(Macro) `rdd_set_d(ret, d) ret := d`

(Macro) `rdd_set_ui(ret, d)`

(Macro) `rdd_set(ret, d)`

(Macro) `rdd_neg(ret, a) ret :=  $-a$`

(Macro) `rdd_abs(ret, a) ret :=  $|a|$`

(Macro) `rdd_cmp_ui(a, b) a > b` なら -1、`a == b` なら 0、`a < b` なら -1 を返します

(Macro) `rdd_ui_div(ret, a, b) ret := a/b`

(Macro) `rdd_ui_sub(ret, a, b) ret := a - b`

(Macro) `rdd_div_d(ret, a, b) ret := a/b`

(Macro) `rdd_add_d(ret, a, b) ret := a + b`

(Macro) `rdd_sub_d(ret, a, b) ret := a - b`

(Macro) `rdd_mul_d(ret, a, b) ret := a  $\times$  b`

(Macro) `rdd_div_ui(ret, a, b) ret := a/b`

(Macro) `rdd_add_ui(ret, a, b) ret := a + b`

(Macro) `rdd_sub_ui(ret, a, b) ret := a - b`

(Macro) `rdd_mul_ui(ret, a, b) ret := a  $\times$  b`

(Macro) `set0_td(val) SET0_TD(val)`

(Macro) `rtd_set0(val) SET0_TD(val)`

(Macro) `rtd_add(ret, a, b) ret := a + b`

(Macro) `rtd_addt(ret, a, b)`

(Macro) `rtd_addq(ret, a, b)`

(Macro) `rtd_sub(ret, a, b) ret := a - b`

(Macro) `rtd_subt(ret, a, b) ret := a - b`

```

(Macro) rtd_subq(ret, a, b) ret :=  $a - b$ 
(Macro) rtd_mul(ret, a, b) ret :=  $a \times b$ 
(Macro) rtd_divt(ret, a, b) ret :=  $a/b$ 
(Macro) rtd_divtq(ret, a, b)
(Macro) rtd_divq(ret, a, b)
(Macro) rtd_div(ret, a, b)
(Macro) rtd_sqrt(ret, a) ret :=  $\sqrt{a}$ 
(Macro) rtd_sqrt_d(ret, a)
(Macro) rtd_sqrt_ui(ret, a)
(Macro) rtd_get_d(a) return (double)a
(Macro) rtd_set_d(ret, d) ret :=  $d$ 
(Macro) rtd_set_ui(ret, d)
(Macro) rtd_set(ret, d)
(Macro) rtd_neg(ret, a) ret :=  $-a$ 
(Macro) rtd_abs(ret, a) ret :=  $|a|$ 
(Macro) rtd_cmp_ui(a, b)  $a > b$  なら  $-1$ 、 $a == b$  なら  $0$ 、 $a < b$  なら  $-1$  を返します
(Macro) rtd_ui_div(ret, a, b) ret :=  $a/b$ 
(Macro) rtd_ui_sub(ret, a, b) ret :=  $a - b$ 
(Macro) rtd_div_d(ret, a, b) ...  $c := a/b$ 
(Macro) rtd_add_d(ret, a, b) ...  $c := a + b$ 
(Macro) rtd_sub_d(ret, a, b) ...  $c := a - b$ 
(Macro) rtd_mul_d(ret, a, b) ...  $c := a \times b$ 
(Macro) rtd_div_ui(ret, a, b) ...  $c := a/b$ 
(Macro) rtd_add_ui(ret, a, b) ...  $c := a + b$ 
(Macro) rtd_sub_ui(ret, a, b) ...  $c := a - b$ 
(Macro) rtd_mul_ui(ret, a, b) ...  $c := a \times b$ 
(Macro) set0_qd(val) ...  $val := 0$ 
(Macro) rqd_set0(val)
(Macro) rqd_add(ret, a, b) ...  $c := a + b$ 
(Macro) rqd_sub(ret, a, b) ...  $c := a - b$ 
(Macro) rqd_mul(ret, a, b) ...  $c := a \times b$ 
(Macro) rqd_div(ret, a, b) ...  $c := a/b$ 
(Macro) rqd_sqrt(ret, a) ... ret :=  $\sqrt{a}$ 
(Macro) rqd_sqrt_d(ret, a) ret :=  $\sqrt{a}$ 
(Macro) rqd_sqrt_ui(ret, a)
(Macro) rqd_get_d(a) return (double)a
(Macro) rqd_set_d(ret, d) ret :=  $d$ 
(Macro) rqd_set_ui(ret, d)
(Macro) rqd_set(ret, d)
(Macro) rqd_neg(ret, a) ret :=  $-a$ 

```

(Macro) `rqd_abs(ret, a)` $ret := |a|$
 (Macro) `rqd_cmp_ui(a, b)` $a > b$ なら -1 、 $a == b$ なら 0 、 $a < b$ なら -1 を返します
 (Macro) `rqd_ui_div(ret, a, b)` $ret := a/b$
 (Macro) `rqd_ui_sub(ret, a, b)` $ret := a - b$
 (Macro) `rqd_div_d(ret, a, b)` $\dots c := a/b$
 (Macro) `rqd_add_d(ret, a, b)` $\dots c := a + b$
 (Macro) `rqd_sub_d(ret, a, b)` $\dots c := a - b$
 (Macro) `rqd_mul_d(ret, a, b)` $\dots c := a \times b$
 (Macro) `rqd_div_ui(ret, a, b)` $\dots c := a/b$
 (Macro) `rqd_add_ui(ret, a, b)` $\dots c := a + b$
 (Macro) `rqd_sub_ui(ret, a, b)` $\dots c := a - b$
 (Macro) `rqd_mul_ui(ret, a, b)` $\dots c := a \times b$

2.10 AVX2

2.10.1 float

- `__m256 _bncavx2_fabsf(__m256 val8)` $\dots val8 := \max val8, 0 - val8$
- `__m256 _bncavx2_fneg(__m256 a)` $\dots \text{return } -a$
- `void _bncavx2_ffma(float ret[], float a[], float b[], float c[], int dim)` $\dots ret := a \times b + c$
- `void _bncavx2_fmulp(float ret[], float a[], float b[], int dim)` $\dots ret := a \times b$

2.10.2 double

- `__m256d _bncavx2_fabs(__m256d val4)` $\dots val4 := \max val4, -val4$
- `__m256d _bncavx2_dneg(__m256d a)` $\dots -a$
- `void _bncavx2_dfma(double ret[], double a[], double b[], double c[], int dim)` $\dots ret := a \times b + c$
- `void _bncavx2_dmul(double ret[], double a[], double b[], int dim)` $\dots ret := a \times b$
- `void _bncavx2_ddiv(double ret[], double a[], double b[], int dim)` $\dots ret := a/b$
- `void _bncavx2_dadd(double ret[], double a[], double b[], int dim)` $\dots ret := a + b$
- `void _bncavx2_dsub(double ret[], double a[], double b[], int dim)` $\dots ret := a - b$
- `double _bncavx2_ddotp(double a[], double b[], int dim)` $\dots ret := \sum_{i=0}^{\text{dim}-1} a[i] \times b[i]$

2.10.3 誤差なし変換 (error-free transformation)

- `__m256d _bncavx2_dquick_two_sum(__m256d a, __m256d b, __m256d * err)` $\dots fl(a + b)$ と $err(a + b)$ を計算します。 $|a| \geq |b|$ を仮定します。
- `__m256d _bncavx2_dquick_two_diff(__m256d a, __m256d b, __m256d * err)` $\dots fl(a - b)$ と $err(a - b)$ を計算します。 $|a| \geq |b|$ を仮定します。
- `__m256d _bncavx2_dtwo_sum(__m256d a, __m256d b, __m256d * err)` $\dots fl(a + b)$ と $err(a + b)$

を計算します。

- `__m256d _bncavx2_dtwo_diff(__m256d a, __m256d b, __m256d * err) ... fl( $a-b$ ) と  $err(a-b)$  を計算します。`
- `__m256d _bncavx2_dtwo_prod(__m256d a, __m256d b, __m256d * err) ... fl( $a \times b$ ) と  $err(a \times b)$  を計算します。`

2.10.4 double-double 精度算術演算

- `void _bncavx2_set0_dd(__m256d) ret[DDSIZE] ... ret := 0`
- (Macro) `_bncavx2_rdd_set0(ret) ...` 次と同じです: `_bncavx2_set0_dd(ret)`
- `void _bncavx2_get_dd_m256d_i(ddfloat *ret, __m256d ret4[DDSIZE], int avx_index) ... ret := ret4[avx_index]`
 - `void _bncavx2_rdd_sum256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := ret4[0] + ret4[1] + ret4[2] + ret4[3]`
 - `void _bncavx2_rdd_abssum256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := |ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|`
 - `void _bncavx2_rdd_absmax256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)`
 - `void _bncavx2_rdd_norm256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := ||ret4[0]2 + ret4[1]2 + ret4[2]2 + ret4[3]2||2`
 - `void _bncavx2_rdd_add(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret := a + b`
 - `void _bncavx2_rdd_sub(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret := a - b`
 - `void _bncavx2_rdd_mul(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret := a × b`
 - `void _bncavx2_rdd_div(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret := a/b`
 - `void _bncavx2_rdd_abs(__m256d ret[DDSIZE], __m256d a[DDSIZE]) ... ret := |a|`

2.10.5 triple-double 精度算術演算

- `void _bncavx2_set0_td(__m256d ret[TDSIZE]) ... ret := 0`
- `void _bncavx2_get_td_m256d_i(tdfloat *ret, __m256d ret4[TDSIZE], int avx_index) ... ret := ret4[avx_index]`
- `void _bncavx2_rtd_sum256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) ... ret := ret4[0] + ret4[1] + ret4[2]`
- `void _bncavx2_rtd_abs(__m256d ret[TDSIZE], __m256d a[TDSIZE]) ... ret := |a|`
- `void _bncavx2_rtd_abssum256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) ... ret :=`

- $|ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|$
- void _bncavx2_rtd_absmax256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) $\cdots ret := \max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)$
- void _bncavx2_rtd_norm256d(double ret[TDSIZE], __m256d ret4[TDSIZE])
- void _bncavx2_vec_sum(__m256d e[], const __m256d x[], int n) $\cdots e := \text{vec_sum}(x)$
- void _bncavx2_vseb(__m256d y[], int ny, const __m256d e[], int ne) $\cdots y := \text{vec_sum_err_branch}(\text{vseb}(k))(e)$
- void _bncavx2_merge(__m256d c[], __m256d a[], int na, __m256d b[], int nb) $\cdots a[na]$ と $b[nb]$ を $c[na + nb]$ にマージします

(Macro) _bncavx2_rtd_add() _bncavx2_rtd_addq

- void _bncavx2_rtd_addq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
 $\cdots ret := a + b$
- void _bncavx2_rtd_addt(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
- void _bncavx2_rtd_mulq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
 $\cdots ret := a \times b$
- void _bncavx2_rtd_mul(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
- void _bncavx2_rtd_neg(__m256d c[TDSIZE], __m256d a[TDSIZE]) $\cdots c := -a$
- void _bncavx2_rtd_sub(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots c := a - b$
- void _bncavx2_rtd_subq(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
- void _bncavx2_rtd_divq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
 $\cdots c := a/b$
- void _bncavx2_to_td(__m256d r[TDSIZE], __m256d a, __m256d b, __m256d c) $\cdots r := \text{TD}(a, b, c)$
- void _bncavx2_rtd_mul_d(__m256d c[TDSIZE], __m256d a, __m256d b[TDSIZE]) $\cdots c := a \times b$
- void _bncavx2_rtd_mul_dd(__m256d c[TDSIZE], __m256d a[DDSIZE], __m256d b[TDSIZE])
 $\cdots c := a \times b$

(Macro) _bncavx2_rtd_div() $\cdots$ _bncavx2_rtd_divtq

- void _bncavx2_rtd_divt(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
 $\cdots c := a/sb$
- void _bncavx2_rtd_divtq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])

2.10.6 quadruple-double 精度算術演算

- void _bncavx2_set0_qd(__m256d ret[QDSIZE]) $\cdots ret := 0$
- void _bncavx2_get_qd_m256d_i(qdfloat * ret, __m256d ret4[QDSIZE], int avx_index) $ret := ret4[avx_index]$
- void _bncavx2_rqd_sum256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) $\cdots ret := \text{mmval}[0] + \dots + \text{mmval}[7]$
- void _bncavx2_rqd_abs __m256d ret[QDSIZE], __m256d a[QDSIZE] $\cdots ret := |a|$

- `void _bncavx2_rqd_abssum256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := |ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|`
- `void _bncavx2_rqd_absmax256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)`
- `void _bncavx2_rqd_norm256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := ||ret4[0]2 + ret4[1]2 + ret4[2]2 + ret4[3]2||2`
- `void _bncavx2_renorm(__m256d * c0, __m256d * c1, __m256d * c2, __m256d * c3) ... renorm(double *c0, double *c1, double *c2, double *c3)`
- `void _bncavx2_renorm4(__m256d * c0, __m256d * c1, __m256d * c2, __m256d * c3, __m256d * c4) ... renorm4(double *c0, double *c1, double *c2, double *c3, double *c4)`
- `void _bncavx2_three_sum(__m256d * a, __m256d * b, __m256d * c) ... three_sum(double *a, double *b, double *c)`
- `void _bncavx2_three_sum2(__m256d * a, __m256d * b, __m256d * c) ... void three_sum2(double *a, double *b, double *c)`
- `void _bncavx2_rqd_add(__m256d ret[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... ret := a + b`
- `void _bncavx2_rtd_addq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... ret := a + b`
- `void _bncavx2_rtd_mulq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... ret := a × b`
- `void _bncavx2_rqd_mul(__m256d ret[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... ret := a × b`
- `void _bncavx2_rqd_sub(__m256d c[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... ret := a - b`
- `void _bncavx2_rqd_mul_d(__m256d c[QDSIZE], const __m256d a[QDSIZE], __m256d b) ... ret := a × b`
- `void _bncavx2_rtd_divq(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... c := a/b quad-double 除算の方式で`
- `void _bncavx2_rqd_div(__m256d c[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... c := a/b`

2.11 その他の関数

以下の関数は `bnc_common.h` で定義されており、`bnc_common_func.c` と `algebraic_eq.c` に実装されています。

2.11.1 共通関数

- `void bnc_print_env_all(void) ... bncmatmul` ライブラリのすべての環境変数を出力します。
- `double mylog2(double x) ... log2(x)` を返します。

- `double matmul_gflops(double comp_sec, int dim) ...` `comp_sec` (秒単位の計算時間) から、`dim` 次元の密な正方行列積を計算する際のギガフロップス値を返します。
- `int byte_double_sqmat(int dim) ...` `dim` 次元の正方行列を格納するのに必要なバイト数を返します。
- `void set_bnc_default_prec(unsigned long precision)`
- `void set_bnc_default_prec_decimal(unsigned long precision) ...` デフォルト精度をビット単位または十進桁数で設定します。
- `mp_rnd_t get_bnc_default_rounding_mode(void) ...` 現在の丸めモードを返します。
- `void set_bnc_rounding_mode(mp_rnd_t rounding_mode) ...` `rounding_mode` をデフォルトの丸めモードとして設定します。
- `unsigned long int get_bnc_default_prec(void) ...` 現在のデフォルト精度をビット単位で返します。
- `void mpf_fma(mpf_t rop, mpf_t op1, mpf_t op2, mpf_t op3) ...` `rop := op1 × op2 + op3`
- `void mpf_log10(mpf_t ret, mpf_t x) ...` $\log_{10}(x)$
- `void mpf_exp(mpf_t ret, mpf_t x) ...` $\exp(x)$
- `void mpf_log(mpf_t ret, mpf_t x) ...` $\log_e(x)$
- `void mpf_sin(mpf_t ret, mpf_t x) ...` $\sin(x)$
- `void mpf_cos(mpf_t ret, mpf_t x) ...` $\cos(x)$
- `void mpf_tan(mpf_t ret, mpf_t x) ...` $\tan(x)$
- `void mpf_atan(mpf_t ret, mpf_t x) ...` $\arctan(x)$

2.11.2 乱数関数

- `void rdd_srand(unsigned long seed)`
- `void rtd_srand(unsigned long seed)`
- `void rqd_srand(unsigned long seed)`
- `void mpf_srand(unsigned long seed) ...` 乱数シードを設定し、各精度について現在の乱数状態を初期化します。
- `void rdd_urand(double ret[DDSIZE])`
- `void rtd_urand(double ret[TDSIZE])`
- `void rqd_urand(double ret[QDSIZE])`
- `void mpf_urand(mpf_t ret) ...` $[0, 1]$ 上の乱数を返します。
- `void rdd_nrand(double ret[DDSIZE])`
- `void rtd_nrand(double ret[TDSIZE])`
- `void rqd_nrand(double ret[QDSIZE])`
- `void mpf_nrand(mpf_t ret) ...` 標準正規分布 $[-1, 1]$ に従う乱数を返します。

2.11.3 代数方程式ソルバ

- `int dquadratic_eq(double ans_re[2], double ans_im[2], double coef[3])`
- `int mpfquadratic_eq(mpf_t ans_re[2], mpf_t ans_im[2], mpf_t coef[3]) ...` $\text{coef}[2] \cdot x^2 +$

$\text{coef}[1] \cdot x + \text{coef}[0] = 0$ を解き、根を $x = \text{ans_re}[0] + \text{ans_im}[0] \cdot i$ および $\text{ans_re}[1] + \text{ans_im}[1] \cdot i$ として格納します。

2.12 ベンチマーク用ツール関数

BNCmatmul には、ベンチマークに役立つ関数がいくつか用意されています。

2.12.1 時間計測

- `float fget_sec(int flag)`
- `double get_sec(int flag) ...` `flag ≠ 0` の場合は現在の実時間・ユーザ時間・システム時間を倍精度で秒単位として返し、`flag = 0` の場合は現在のユーザ時間とシステム時間の合計を秒単位で返します。
- `double get_secv(void)`
- `float fget_secv(void) ...` `[f]get_sec(0)` と同じです。
- `double get_real_sec(int flag)`
- `float fget_real_sec(int flag) ...` `flag ≠ 0` の場合は現在の `tv_sec` (秒単位) と `tv_usec` (マイクロ秒単位) を倍精度で秒単位として返し、それ以外の場合は `tv_sec + tv_usec/106` の合計を返します。
- `double get_real_secv(void)`
- `float fget_real_secv(void) ...` `[f]get_real_sec(0)` と同じです。

2.12.2 ベクトルおよび行列の相対誤差

■ベクトルの相対誤差

- `void relerr3_dvector(double *max_relerr, double *min_relerr, double *norm_relerr, DVector vec, DVector vec_true, int kind_of_norm)`
- `void relerr3_ddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector vec, DDVector vec_true, int kind_of_norm)`
- `void relerr3_tdvector(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, TDVector vec_true, int kind_of_norm)`
- `void relerr3_tdvector_qdvec(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, QDVector vec_true, int kind_of_norm)`
- `void relerr3_qdvector(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDVector vec, QDVector vec_true, int kind_of_norm)`
- `void relerr3_cddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double max_real_relerr[DDSIZE], double min_real_relerr[DDSIZE], double max_image_relerr[DDSIZE], double min_image_relerr[DDSIZE], double norm_relerr[DDSIZE], CDDVector vec, CDDVector vec_true, int kind_of_norm)`
- `void relerr3_ctdvector(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double max_real_relerr[TDSIZE], double min_real_relerr[TDSIZE], double max_image_relerr[TDSIZE],`

- double min_image_relerr[TDSIZE], double norm_relerr[TDSIZE], CTDVector vec, CTDVector vec_true, int kind_of_norm)
- void relerr3_cqdvectord(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double max_real_relerr[QDSIZE], double min_real_relerr[QDSIZE], double max_image_relerr[QDSIZE], double min_image_relerr[QDSIZE], double norm_relerr[QDSIZE], CQDVector vec, CQDVector vec_true, int kind_of_norm)
- void relerr3_mpfvector(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFVector vec, MPFVector vec_true, int kind_of_norm)
- void relerr3_cmpfvector(mpf_t max_abs_relerr, mpf_t min_abs_relerr, mpf_t max_real_relerr, mpf_t min_real_relerr, mpf_t max_image_relerr, mpf_t min_image_relerr, mpf_t norm_relerr, CMPFVector vec, CMPFVector vec_true, int kind_of_norm)

■行列の相対誤差

kind_of_norm = 0 $\|A\|_{\infty}$

kind_of_norm = 1 $\|A\|_1$

kind_of_norm = 2 $\|A\|_F$

- void relerr3_dmatrix(double * max_relerr, double * min_relerr, double * norm_relerr, DMatrix mat, DMatrix mat_true, int kind_of_norm)
- void relerr3_ddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDMatrix mat, DDMatrix mat_true, int kind_of_norm)
- void relerr3_tdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDMatrix mat, TDMatrix mat_true, int kind_of_norm)
- void relerr3_qdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDMatrix mat, QDMatrix mat_true, int kind_of_norm)
- void relerr3_mpfmatrix(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFMatrix mat, MPFMatrix mat_true, int kind_of_norm)
- void relerr3_cddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double max_real_relerr[DDSIZE], double min_real_relerr[DDSIZE], double max_image_relerr[DDSIZE], double min_image_relerr[DDSIZE], double norm_relerr[DDSIZE], CDDMatrix mat, CDDMatrix mat_true, int kind_of_norm)
- void relerr3_ctdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double max_real_relerr[TDSIZE], double min_real_relerr[TDSIZE], double max_image_relerr[TDSIZE], double min_image_relerr[TDSIZE], double norm_relerr[TDSIZE], CTDMatrix mat, CTDMatrix mat_true, int kind_of_norm)
- void relerr3_cqdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double max_real_relerr[QDSIZE], double min_real_relerr[QDSIZE], double max_image_relerr[QDSIZE], double min_image_relerr[QDSIZE], double norm_relerr[QDSIZE], CQDMatrix mat, CQDMatrix mat_true, int kind_of_norm)
- void relerr3_cmpfmatrix(mpf_t max_abs_relerr, mpf_t min_abs_relerr, mpf_t max_real_relerr,

```
mpf_t min_real_relerr, mpf_t max_image_relerr, mpf_t min_image_relerr, mpf_t  
norm_relerr, CMPFMatrix mat, CMPFMatrix mat_true, int kind_of_norm)
```

第 3 章

基本的な線形計算

3.1 D, DD, TD, QD および MPFR のベクトルと行列のデータ型

DD, TD, QD および MPFR 精度のベクトルのデータ型は次のとおりです。

DDVector, TDVector, QDVector

```
typedef struct
{
    long int dim; // dim <= real_dim
    long int real_dim; // multiplier of _BNC_D_WIDTH
    double *element[DDSIZE]; // [TDSIZE] and [QDSIZE] used as TDVector and
    ↪ QDVector
} ddvector;

typedef *DDVector; // DDVector, TDVector, QDVector are pointers.
```

MPFVector

```
typedef struct{
    unsigned long int prec; // default precision of element
    mpf_t *element;
    long int dim;
    long int real_dim;
} mpfvector;

typedef mpfvector *MPFVector; // MPFVector is a pointer.
```

DD, TD, QD および MPFR 精度の行列のデータ型は次のとおりです。

DDMatrix, TDMatrix, QDMatrix

```
// DD matrix
```

```
typedef struct{
    long int row_dim, col_dim;
    long int real_row_dim, real_col_dim; // multiplier of _BNC_D_WIDTH
    double *element[DDSIZE];
} ddmatrix;

typedef *DDMatrix;
```

MPFMatrix

```
typedef struct{
    unsigned long int prec;
    mpf_t *element;
    long int row_dim, col_dim;
    long int real_row_dim, real_col_dim;
    void *element_block; // mantissa block
} mpfmatrix;

typedef mpfmatrix *MPFMatrix;
```

3.2 ベクトル演算

- `ddfloat get_ddvector_i_ddfloat(DDVector vec, long int index)` ... `vec` の `index` 番目の要素を `ddfloat` 型として取得します。
- `tdfloat get_tdvector_i_tdfloat(TDVector vec, long int index)`
- `qdfloat get_qdvector_i_qdfloat(QDVector vec, long int index)`

(Macro) `GET_DDVECTOR_I(vec, index)` ... `index` 番目の要素を `ddfloat` へのポインタとして取得します。

(Macro) `get_ddvector_i(vec, index)`

(Macro) `GET_TDVECTOR_I(vec, index)` ... `index` 番目の要素を `tdfloat` へのポインタとして取得します。

(Macro) `get_tdvector_i(vec, index)`

(Macro) `GET_QDVECTOR_I(vec, index)` ... `index` 番目の要素を `qdfloat` へのポインタとして取得します。

(Macro) `get_qdvector_i(vec, index)`

(Macro) `GET_MPFVECTOR_I(vec, index)` ... `index` 番目の要素を `mpf_t` (`mpfr_t`) へのポインタとして取得します。

(Macro) `get_mpfvector_i(vec, index)`

- `void set_ddvector_i(DDVector vec, long int index, double * val)` ... `val` を `vec` の `index` 番目の要素に格納します。

(Macro) `SET_DDVECTOR_I(vec, index, val)`

- `void set_tdvector_i(TDVector vec, long int index, double * val)`

(Macro) SET_TDVECTOR_I(vec, index, val)

- void set_qdvector_i(QDVector vec, long int index, double * val)

(Macro) SET_QDVECTOR_I(vec, index, val)

- void set_qdvector_i(QDVector vec, long int index, double * val)

(Macro) SET_QDVECTOR_I(vec, index, val)

- void set_ddvector_i_d(DDVector vec, long int index, double val) ... val を double として vec の index 番目の要素に格納します。

(Macro) SET_DDVECTOR_I_D(vec, index, val)

- void set_tdvector_i_d(TDVector vec, long int index, double val)

(Macro) SET_TDVECTOR_I_D(vec, index, val)

- void set_qdvector_i_d(QDVector vec, long int index, double val)

(Macro) SET_QDVECTOR_I_D(vec, index, val)

- void set0_ddvector_i(DDVector vec, long int index) ... vec の index 番目の要素に 0 を格納します。

(Macro) SET0_DDVECTOR_I(vec, index)

- void set0_tdvector_i(TDVector vec, long int index)

(Macro) SET0_TDVECTOR_I(vec, index)

- void set0_qdvector_i(QDVector vec, long int index)

(Macro) SET0_QDVECTOR_I(vec, index)

- DDVector init_ddvector(in dimension) ... ベクトルを確保・初期化し、零ベクトルとして設定します。
- TDVector init_tdvector(in dimension)
- QDVector init_qdvector(in dimension)
- MPFVector init_mpfvector(in dimension) ... bnc_set_default_prec によるビット単位の既定精度で MPFVector を確保・初期化し、零ベクトルとして設定します。
- MPFVector init2_mpfvector(int dimension, unsigned long prec) ... prec ビットで MPFVector を初期化し、零ベクトルとして設定します。
- void free_ddvector(DDVector vec) ... vec を解放します。
- void free_tdvector(TDVector vec)
- void free_qdvector(QDVector vec)
- void free_mpfvector(MPFVector vec)
- void set_ddfloat_ddvec(ddfloat ret[], int ret_dim, DDVector vec) ... vec を ddfloat 配列に変換します。
- void set_tdfloat_tdvec(tdfloat ret[], int ret_dim, TDVector vec) ... vec を tdfloat 配列に変換します。
- void set_qdfloat_qdvec(qdfloat ret[], int ret_dim, QDVector vec) ... vec を qdfloat 配列に変換します。
- void set_ddvector_ddfloat(DDVector ret, ddfloat array[], int array_dim) ... ddfloat 配列を DDVector ret に変換します。
- void set_tdvector_tdfloat(TDVector ret, tdfloat array[], int array_dim) ... tdfloat 配列を

TDVector ret に変換します。

- void set_qdvector_qdfloat(QDVector ret, qdfloat array[], int array_dim) ... qdfloat 配列を QDVector ret に変換します。
- void print_ddvector(DDVector vec) ... vec を表示します。
- void print_tdvector(TDVector vec)
- void print_qdvector(QDVector vec)
- void print_mpfvector(MPFVector vec)
- void set0_ddvector(DDVector vec) ... vec を零ベクトルに設定します。
- void set0_tdvector(TDVector vec)
- void set0_qdvector(QDVector vec)
- void set0_mpfvector(MPFVector vec)
- void set_ddvector_i_str(DDVector vec, long int index, const char * str) ... 10 進数で表現された *str を vec の index 番目の要素に設定します。
- void set_tdvector_i_str(TDVector vec, long int index, const char * str)
- void set_qdvector_i_str(QDVector vec, long int index, const char * str)
- void set_mpfvector_i_str(MPFVector vec, long int index, const char * str)
- void add_ddvector(DDVector c, DDVector a, DDVector b) ... $c := a + b$
- void add_tdvector(TDVector c, TDVector a, TDVector b)
- void add_qdvector(QDVector c, QDVector a, QDVector b)
- void add_mpfvector(MPFVector c, MPFVector a, MPFVector b)
- void add2_ddvector(DDVector c, DDVector a) *cdots* $c := c + a$
- void add2_tdvector(TDVector c, TDVector a)
- void add2_qdvector(TDVector c, QDVector a)
- void add2_mpfvector(MPFVector c, MPFVector a)
- void sub_ddvector(DDVector c, DDVector a, DDVector b) ... $c := a - b$
- void sub_tdvector(TDVector c, TDVector a, TDVector b)
- void sub_qdvector(QDVector c, QDVector a, QDVector b)
- void sub_mpfvector(MPFVector c, MPFVector a, MPFVector b)
- void sub2_ddvector(DDVector c, DDVector a) ... $c - = a$
- void sub2_tdvector(TDVector c, TDVector a)
- void sub2_qdvector(QDVector c, QDVector a)
- void sub2_mpfvector(MPFVector c, MPFVector a)
- void cmul_ddvector(DDVector c, double val[DDSIZE], DDVector a) ... $c := val \times a$
- void cmul_tdvector(TDVector c, double val[TDSIZE], TDVector a)
- void cmul_qdvector(QDVector c, double val[QDSIZE], QDVector a)
- void cmul_mpfvector(MPFVector c, mpf_t val, MPFVector a)
- void cmul2_ddvector(DDVector c, double val[DDSIZE]) ... $c := val \times c$
- void cmul2_tdvector(TDVector c, double val[TDSIZE])
- void cmul2_qdvector(QDVector c, double val[QDSIZE])
- void cmul2_mpfvector(MPFVector c, mpf_t val)

- void add_cmul_ddvector(DDVector c, DDVector a, double val[DDSIZE], DDVector b) ... $c := a + val * b$
- void add_cmul_tdvector(TDVector c, TDVector a, double val[TDSIZE], TDVector b)
- void add_cmul_qdvector(QDVector c, QDVector a, double val[QDSIZE], QDVector b)
- void add_cmul_mpfvector(MPFVector c, MPFVector a, mpf_t val, MPFVector b)
- void ip_ddvector(double ret[DDSIZE], DDVector a, DDVector b) ... a と b の内積を計算します。
- void ip_tdvector(double ret[TDSIZE], TDVector a, TDVector b)
- void ip_qdvector(double ret[QDSIZE], QDVector a, QDVector b)
- void ip_mpfvector(mpf_t ret, MPFVector a, MPFVector b)
- void neg_ddvector(DDVector c, DDVector a) ... $c := -a$
- void neg_tdvector(TDVector c, TDVector a)
- void neg_qdvector(QDVector c, QDVector a)
- void neg_mpfvector(MPFVector c, MPFVector a)
- void norm1_ddvector(double ret[DDSIZE], DDVector a) ... $\|a\|_1$
- void norm1_tdvector(double ret[TDSIZE], TDVector a)
- void norm1_qdvector(double ret[QDSIZE], QDVector a)
- void norm1_mpfvector(mpf_t ret, MPFVector a)
- void normi_ddvector(double ret[DDSIZE], DDVector a) ... $\|a\|_\infty$
- void normi_tdvector(double ret[TDSIZE], TDVector a)
- void normi_qdvector(double ret[QDSIZE], QDVector a)
- void normi_mpfvector(mpf_t ret, MPFVector a)
- void norm2_ddvector(double ret[DDSIZE], DDVector vec) ... $\|a\|_2$
- void norm2_tdvector(double ret[TDSIZE], TDVector vec)
- void norm2_qdvector(double ret[QDSIZE], QDVector vec)
- void norm2_mpfvector(mpf_t ret, MPFVector vec)

3.3 行列演算

- ddfloat get_ddmatrix_ij_ddfloat(DDMatrix mat, long int i, long int j) ... mat の (i, j) 番目の要素を取得します。

(Macro) GET_DDMATRIX_IJ(mat, i, j) ((get_ddmatrix_ij_ddfloat((mat), (i), (j)).val))

(Macro) get_ddmatrix_ij(mat, i, j) ((get_ddmatrix_ij_ddfloat((mat), (i), (j)).val))

- tdfloat get_tdmatrix_ij_tdfloat(TDMatrix mat, long int i, long int j)

(Macro) GET_TDMATRIX_IJ(mat, i, j) ((get_tdmatrix_ij_tdfloat((mat), (i), (j)).val))

(Macro) get_tdmatrix_ij(mat, i, j) ((get_tdmatrix_ij_tdfloat((mat), (i), (j)).val))

- qdfloat get_qdmatrix_ij_qdfloat(QDMatrix mat, long int i, long int j)

(Macro) GET_QDMATRIX_IJ(mat, i, j) ((get_qdmatrix_ij_qdfloat((mat), (i), (j)).val))

(Macro) get_qdmatrix_ij(mat, i, j) ((get_qdmatrix_ij_qdfloat((mat), (i), (j)).val))

(Macro) GET_MPFMATRIX_IJ(mat, i, j) ((get_mpfmatrix_ij((mat), (i), (j))))

- `mpf_ptr get_mpfmatrix_ij(MPFMatrix mat, long int i, long int j)`
- `void set_ddmatrix_ij(DDMatrix mat, long int i, long int j, double * val)`
- (Macro) `SET_DDMatrix_IJ(mat, i, j, val) set_ddmatrix_ij((mat), (i), (j), (val))`
- `void set_tdmatrix_ij(TDMatrix mat, long int i, long int j, double * val)`
- (Macro) `SET_TDMatrix_IJ(mat, i, j, val) set_tdmatrix_ij((mat), (i), (j), (val))`
- `void set_qdmatrix_ij(QDMatrix mat, long int i, long int j, double * val)`
- (Macro) `SET_QDMatrix_IJ(mat, i, j, val) set_qdmatrix_ij((mat), (i), (j), (val))`
- `void set_mpfmatrix_ij(MPFMatrix mat, long int i, long int j, mpf_t val)`
- (Macro) `SET_MPFMatrix_IJ(mat, i, j, val) set_mpfmatrix_ij((mat), (i), (j), (val))`
- `void set_ddmatrix_ij_d(DDMatrix mat, long int i, long int j, double val)`
- (Macro) `SET_DDMatrix_IJ_D(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (val))`
- (Macro) `SET_DDMatrix_IJ_UI(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (double)(val))`
- (Macro) `set_ddmatrix_ij_ui(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (double)(val))`
- `void set0_ddmatrix_ij(DDMatrix mat, long int i, long int j)`
- (Macro) `SET0_DDMatrix_IJ(mat, i, j) set0_ddmatrix_ij((mat), (i), (j))`
- `void set0_ddmatrix(DDMatrix mat) ... mat を零行列に設定します。`
- `void set0_tdmatrix(TDMatrix mat)`
- `void set0_qdmatrix(QDMatrix mat)`
- `void set0_mpfmatrix(MPFMatrix mat)`
- `DDMatrix init_ddmatrix(long int row_dim, long int col_dim) ... DDMatrix を初期化します。`
- `TDMatrix init_tdmatrix(long int row_dim, long int col_dim) ... TDMatrix を初期化します。`
- `QDMatrix init_qdmatrix(long int row_dim, long int col_dim) ... QDMatrix を初期化します。`
- `MPFMatrix init_mpfmatrix(long int row_dim, long int col_dim) ... MPFMatrix を初期化します。`
- `MPFMatrix init2_mpfmatrix(long int row_dim, long int col_dim, unsigned long prec) ... MPF-Matrix を prec ビット精度で初期化します。`
- `void free_ddmatrix(DDMatrix mat) ... mat を解放します。`
- `void free_tdmatrix(TDMatrix mat)`
- `void free_qdmatrix(QDMatrix mat)`
- `void free_mpfmatrix(MPFMatrix mat)`
- `void print_ddmatrix(DDMatrix mat) ... mat を表示します。`
- `void print_tdmatrix(TDMatrix mat)`
- `void print_qdmatrix(QDMatrix mat)`
- `void print_mpfmatrix(MPFMatrix mat)`
- `void set_ddfloat_ddmat(ddfloat ret[], int ret_dim, DDMatrix mat) ... DDMatrix mat を ddfloat 配列に変換します。`
- `void set_tdfloat_tdmat(tdfloat ret[], int ret_dim, DDMatrix mat) ... TDMatrix mat を tdfloat 配列に変換します。`
- `void set_qdfloat_qdmat(qdfloat ret[], int ret_dim, DDMatrix mat) ... QDMatrix mat を qdfloat 配列に変換します。`
- `void set_ddmatrix_ddfloat(DDMatrix ret, ddfloat array[], int array_dim) ... ddfloat 配列を`

DDMatrix に変換します。

- void set_tdmatrix_tdfloat(TDMatrix ret, tdfloat array[], int array_dim) ... tdfloat 配列を TDMatrix に変換します。
- void set_qdmatrix_qdfloat(QDMatrix ret, qdfloat array[], int array_dim) ... qdfloat 配列を QDMatrix に変換します。
- void mul_ddmatrix(DDMatrix ret, DDMatrix a, DDMatrix b) ... 単純な行列乗算です。
- void mul_tdmatrix(TDMatrix ret, TDMatrix a, TDMatrix b)
- void mul_qdmatrix(QDMatrix ret, QDMatrix a, QDMatrix b)
- void mul_mpfmatrix(MPFMatrix ret, MPFMatrix a, MPFMatrix b)
- void normf_ddmatrix(double ret[DDSIZE], DDMatrix mat) ... mat のフロベニウスノルムの値を取得します。
- void normf_tdmatrix(double ret[TDSIZE], TDMatrix mat)
- void normf_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void normf_mpfmatrix(mpf_t ret, MPFMatrix mat)
- void print_normf_ddmatrix(const char * str, DDMatrix mat) ... mat のフロベニウスノルムを表示します。
- void normi_ddmatrix(double ret[DDSIZE], DDMatrix mat) ... 行列の無限大ノルムです。
- void normi_tdmatrix(double ret[TDSIZE], TDMatrix mat)
- void normi_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void normi_mpfmatrix(mpf_t ret, TDMatrix mat)
- void norm1_ddmatrix(double ret[DDSIZE], DDMatrix mat) ... 行列の 1 ノルムです。
- void norm1_tdmatrix(double ret[TDSIZE], TDMatrix mat)
- void norm1_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void norm1_mpfmatrix(mpf_t ret, MPFMatrix mat)
- void add_ddmatrix(DDMatrix c, DDMatrix a, DDMatrix b) ... $c := a + b$
- void add_tdmatrix(TDMatrix c, TDMatrix a, TDMatrix b)
- void add_qdmatrix(QDMatrix c, QDMatrix a, QDMatrix b)
- void add_mpfmatrix(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void sub_ddmatrix(DDMatrix c, DDMatrix a, DDMatrix b) ... $c := a - b$
- void sub_tdmatrix(TDMatrix c, TDMatrix a, TDMatrix b)
- void sub_qdmatrix(QDMatrix c, QDMatrix a, QDMatrix b)
- void sub_mpfmatrix(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void cmul_ddmatrix(DDMatrix c, double sc[DDSIZE], DDMatrix a) ... $c := sc \times a$
- void cmul_tdmatrix(TDMatrix c, double sc[TDSIZE], TDMatrix a)
- void cmul_qdmatrix(QDMatrix c, double sc[QDSIZE], QDMatrix a)
- void cmul_mpfmatrix(MPFMatrix c, mpf_t sc, MPFMatrix a)
- void transpose_ddmatrix(DDMatrix c, DDMatrix a) ... $c := a^T$
- void transpose_tdmatrix(TDMatrix c, TDMatrix a)
- void transpose_qdmatrix(QDMatrix c, QDMatrix a)
- void transpose_mpfmatrix(MPFMatrix c, MPFMatrix a)

- void subst_ddmatrix(DDMatrix c, DDMatrix a) ... $c := a$
- void subst_tdmatrix(TDMatrix c, TDMatrix a)
- void subst_qdmatrix(QDMatrix c, QDMatrix a)
- void subst_mpfmatrix(MPFMatrix c, MPFMatrix a)
- void setI_ddmatrix(DDMatrix c) ... $c := I$
- void setI_tdmatrix(TDMatrix c)
- void setI_qdmatrix(QDMatrix c)
- void setI_mpfmatrix(MPFMatrix c)
- void mul_ddmatrix_ddvec(DDVector v, DDMatrix a, DDVector vb) ... $v := a * vb$
- void mul_tdmatrix_tdvec(TDVector v, TDMatrix a, TDVector vb)
- void mul_qdmatrix_qdvec(QDVector v, QDMatrix a, QDVector vb)
- void mul_mpfmatrix_mpfvec(MPFVector v, MPFMatrix a, MPFVector vb)
- void mul_ddmatrixt_ddvec(DDVector v, DDMatrix a, DDVector vb) ... $v := a^T * vb$
- void mul_tdmatrixt_tdvec(TDVector v, TDMatrix a, TDVector vb)
- void mul_qdmatrixt_qdvec(QDVector v, QDMatrix a, QDVector vb)
- void mul_mpfmatrixt_mpfvec(MPFVector v, MPFMatrix a, MPFVector vb)
- void inv_ddmatrix(DDMatrix a) ... $a := a^{-1}$ (only for square matrix)
- void inv_tdmatrix(TDMatrix a)
- void inv_qdmatrix(QDMatrix a)
- void inv_mpfmatrix(MPFMatrix a)
- void subst_mpfvector_ddvec(MPFVector c, DDVector a) ... $c := (mpf)a$
- void subst_ddvector_mpfvec(DDVector c, MPFVector MPFVector a) ... $c := (dd)a$
- void subst_mpfmatrix_ddmat(MPFMatrix c, DDMatrix a) ... $c := (mpf)a$
- void subst_ddmatrix_mpfmat(DDMatrix c, MPFMatrix a) ... $c := (dd)a$
- void relerr_ddvector_mpfvec(double relerr[DDSIZE], DDVector approx_vec, MPFVector true_vec, int norm_type) ... ベクトルのノルム相対誤差です。
- void relerr_element_ddvector_mpf(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector approx_vec, MPFVector true_vec, int norm_type) ... ベクトルの要素ごとの相対誤差です。
/* c := (dd)a */
- void subst_ddvector_dvec(DDVector c, DVector sa) ... $c := (dd)a$
/* c := (d)a */
- void subst_dvector_ddvec(DVector c, DDVector a) ... $c := (double)a$
/* c := (dd)a */
- void subst_ddmatrix_dmat(DDMatrix c, DMatrix a) ... $c := (dd)a$
/* c := (d)a */
- void subst_dmatrix_ddmat(DMatrix c, DDMatrix a) ... $c := (double)a$
- void relerr_ddvector(double relerr[DDSIZE], DDVector approx_vec, DDVector true_vec, int norm_type) ... ベクトルのノルム相対誤差です。

- `void relerr_element_ddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector approx_vec, DDVector true_vec, int norm_type) ...` ベクトルの要素ごとの相対誤差です。
- `void row_swap_ddmatrix(DDMatrix mat, long int row_index0, long int row_index1, long int col_start, long int col_end) ...` `a(row_index0, col_start:col_end)` と `a(row_index1, col_start:col_end)` を入れ替えます。

3.4 行列とベクトルのファイル入出力

- `void fread_ddmatrix(FILE * fp, DDMatrix mat) ...` `fp` から行列の要素を読み込み、`mat` に格納します。
- `void fread_tdmatrix(FILE * fp, TDMatrix mat)`
- `void fread_qdmatrix(FILE * fp, QDMatrix mat)`
- `void fread_mpfmatrix(FILE * fp, MPFMatrix mat)`
- `void fread_ddmatrix_fname(const char * fname, DDMatrix mat) ...` `fname` から行列の要素を読み込み、`mat` に格納します。
- `void fread_tdmatrix_fname(const char * fname, TDMatrix mat)`
- `void fread_qdmatrix_fname(const char * fname, QDMatrix mat)`
- `void fread_mpfmatrix_fname(const char * fname, MPFMatrix mat)`
- `void fwrite_ddmatrix(FILE * fp, DDMatrix mat) ...` `mat` の行列要素を `fp` に書き込みます。
- `void fwrite_tdmatrix(FILE * fp, TDMatrix mat)`
- `void fwrite_qdmatrix(FILE * fp, QDMatrix mat)`
- `void fwrite_mpfmatrix(FILE * fp, MPFMatrix mat)`
- `void fwrite_ddmatrix_fname(const char * fname, DDMatrix mat) ...` `mat` の行列要素を `fname` に書き込みます。
- `void fwrite_tdmatrix_fname(const char * fname, TDMatrix mat)`
- `void fwrite_qdmatrix_fname(const char * fname, QDMatrix mat)`
- `void fwrite_mpfmatrix_fname(const char * fname, MPFMatrix mat)`
- `void fread_ddvector(FILE * fp, DDVector vec) ...` `fp` からベクトルの要素を読み込み、`vec` に格納します。
- `void fread_tdvector(FILE * fp, TDVector vec)`
- `void fread_qdvector(FILE * fp, QDVector vec)`
- `void fread_mpfvector(FILE * fp, MPFVector vec)`
- `void fread_ddvector_fname(const char * fname, DDVector vec) ...` `fname` からベクトルの要素を読み込み、`vec` に格納します。
- `void fread_tdvector_fname(const char * fname, TDVector vec)`
- `void fread_qdvector_fname(const char * fname, QDVector vec)`
- `void fread_mpfvector_fname(const char * fname, MPFVector vec)`
- `void fwrite_ddvector(FILE * fp, DDVector vec) ...` `vec` のベクトル要素を `fp` に書き込みます。

- void fwrite_tdvector(FILE * fp, TDVector vec)
- void fwrite_qdvector(FILE * fp, QDVector vec)
- void fwrite_mpfvector(FILE * fp, MPFVector vec)
- void fwrite_ddvector_fname(const char * fname, DDVector vec) ... vec のベクトル要素を fname に書き込みます。
- void fwrite_tdvector_fname(const char * fname, TDVector vec)
- void fwrite_qdvector_fname(const char * fname, QDVector vec)
- void fwrite_mpfvector_fname(const char * fname, MPFVector vec)
- void read_test_linear_eq_dd(DDMatrix A, DDVector true_x, DDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b) ... 係数行列 A を fname_A から、true_x を fname_true_x から、ベクトル b を fname_b から読み込みます。
- void read_test_linear_eq_td(TDMatrix A, TDVector true_x, TDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)
- void read_test_linear_eq_qd(QDMatrix A, QDVector true_x, QDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)
- void read_test_linear_eq_mpf(MPFMatrix A, MPFVector true_x, MPFVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)

3.5 テスト行列の生成

以下の関数は、ベンチマークテスト用にさまざまなテスト行列を提供します。

- void hilbert_ddmatrix(DDMatrix a, long int dim) ... ヒルベルト行列です。
- void hilbert_tdmatrix(TDMatrix a, long int dim)
- void hilbert_qdmatrix(QDMatrix a, long int dim)
- void hilbert_mpfmatrix(MPFMatrix a, long int dim)
- void lotkin_ddmatrix(DDMatrix a, long int dim) ... Lotkin 行列です。
- void lotkin_tdmatrix(TDMatrix a, long int dim)
- void lotkin_qdmatrix(QDMatrix a, long int dim)
- void lotkin_mpfmatrix(MPFMatrix a, long int dim)
- void frank_ddmatrix(DDMatrix a, long int dim) ... 対称 Frank 行列です。
- void frank_tdmatrix(TDMatrix a, long int dim)
- void frank_qdmatrix(QDMatrix a, long int dim)
- void frank_mpfmatrix(MPFMatrix a, long int dim)
- void tridiag_ddmatrix(DDMatrix a, DDVector low_subdiag, DDVector diag, DDVector up_subdiag, long int dim) ... 三重対角行列です。
- void tridiag_tdmatrix(DDMatrix a, TDVector low_subdiag, TDVector diag, TDVector up_subdiag, long int dim)
- void tridiag_qdmatrix(DDMatrix a, QDVector low_subdiag, QDVector diag, QDVector up_subdiag, long int dim)

- `void tridiag_mpfmatrix(DDMatrix a, MPFVector low_subdiag, MPFVector diag, MPFVector up_subdiag, long int dim)`
- `void int_sym_rand_ddmatrix(DDMatrix mat, long int max, long int seed, long int dim) ...` 整数対称ランダム行列です。
- `void int_sym_rand_tdmatrix(TDMatrix mat, long int max, long int seed, long int dim)`
- `void int_sym_rand_qdmatrix(QDMatrix mat, long int max, long int seed, long int dim)`
- `void int_sym_rand_mpfmatrix(MPFMatrix mat, long int max, long int seed, long int dim)`
- `void int_unsym_rand_ddmatrix(DDMatrix mat, long int max, long int seed, long int dim) ...` 整数非対称ランダム行列です。
- `void int_unsym_rand_tdmatrix(TDMatrix mat, long int max, long int seed, long int dim) ...` 整数非対称ランダム行列です。
- `void int_unsym_rand_qdmatrix(QDMatrix mat, long int max, long int seed, long int dim) ...` 整数非対称ランダム行列です。
- `void int_unsym_rand_mpfmatrix(MPFMatrix mat, long int max, long int seed, long int dim) ...` 整数非対称ランダム行列です。
- `void diag_ddmatrix(DDMatrix mat, DDVector diag, long int dim) ...` 実対角行列です。
- `void diag_tdmatrix(TDMatrix mat, TDVector diag, long int dim)`
- `void diag_qdmatrix(QDMatrix mat, QDVector diag, long int dim)`
- `void diag_mpfmatrix(MPFMatrix mat, MPFVector diag, long int dim)`
- `void toeplitz_ddmatrix(DDMatrix mat, double gamma_param[DDSIZE], long int dim) ...` Toeplitz 行列です。
- `void toeplitz_tdmatrix(TDMatrix mat, double gamma_param[TDSIZE], long int dim)`
- `void toeplitz_qdmatrix(QDMatrix mat, double gamma_param[QDSIZE], long int dim)`
- `void toeplitz_mpfmatrix(MPFMatrix mat, mpf_t gamma_param, long int dim)`
- `void pascal_ddmatrix(DDMatrix ret, long int dim) ...` Pascal 行列です。
- `void pascal_tdmatrix(TDMatrix ret, long int dim)`
- `void pascal_qdmatrix(QDMatrix ret, long int dim)`
- `void pascal_mpfmatrix(MPFMatrix ret, long int dim)`
- `void im_rand_ddmatrix(DDMatrix ret, unsigned long seed) ...` $I - random$
- `void im_rand_tdmatrix(TDMatrix ret, unsigned long seed)`
- `void im_rand_qdmatrix(QDMatrix ret, unsigned long seed)`
- `void im_rand_mpfmatrix(MPFMatrix ret, unsigned long seed)`

3.6 単純法・ブロック法・Strassen 法による行列乗算と関連関数

- (Macro) `mul_ddmatrix_simple(c, a, b) ... mul_ddmatrixc, a, b ...` 単純な三重ループ方式による行列乗算です。
- (Macro) `mul_tdmatrix_simple(c, a, b) ... mul_tdmatrixc, a, b`
- (Macro) `mul_qdmatrix_simple(c, a, b) ... mul_qdmatrixc, a, b`

- void mul_mpfmatrix_simple(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void mul_dmatrix_strassen(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Strassen 法または Winograd 法に基づく行列乗算です。
- void mul_ddmatrix_strassen(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_strassen(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_strassen(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_strassen(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void mul_dmatrix_block(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... ブロック行列乗算です。
- void mul_ddmatrix_block(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_block(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_block(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_block(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void mul_dmatrix_strassen_even(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Strassen 法 (偶数次元) です。
- void mul_ddmatrix_strassen_even(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_strassen_even(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_strassen_even(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_strassen_even(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void mul_dmatrix_winograd_even(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Strassen 法の Winograd 変種です。
- void mul_ddmatrix_winograd_even(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_winograd_even(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_winograd_even(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_winograd_even(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void inv_ddmatrix_strassen_even(DDMatrix ret, DDMatrix mat_a, long int min_dim) ... Strassen 法を用いた逆行列の計算です。

- void inv_tdmatrix_strassen_even(TDMatrix ret, TDMatrix mat_a, long int min_dim)
- void inv_qdmatrix_strassen_even(QDMatrix ret, QDMatrix mat_a, long int min_dim)

3.6.1 OpenMP

(Macro) `_bncomp_mul_ddmatrix_simple(c, a, b)` `_bncomp_mul_ddmatrix(c, a, b)` ... 並列化された単純な行列乗算です。

(Macro) `_bncomp_mul_tdmatrix_simple(c, a, b)` `_bncomp_mul_tdmatrix(c, a, b)`

(Macro) `_bncomp_mul_qdmatrix_simple(c, a, b)` `_bncomp_mul_qdmatrix(c, a, b)`

(Macro) `_bncomp_mul_mpfmatrix_simple(c, a, b)` `_bncomp_mul_mpfmatrix(c, a, b)`

- void `_bncomp_mul_ddmatrix_block`(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim) ... 並列化されたブロック行列乗算です。
- void `_bncomp_mul_tdmatrix_block`(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_qdmatrix_block`(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_mpfmatrix_block`(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void `_bncomp_mul_ddmatrix_strassen`(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim) ... 並列化された Strassen 法および Winograd 法による行列乗算です (実装が未成熟なため性能は限定的です)。
- void `_bncomp_mul_tdmatrix_strassen`(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_qdmatrix_strassen`(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_mpfmatrix_strassen`(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)

3.7 ベクトルと行列の相対誤差の取得

- void `relerr3_dvector`(double *max_relerr, double *min_relerr, double *norm_relerr, DVector vec, DVector vec_true, int kind_of_norm) ... vec の相対誤差です。
- void `relerr3_ddvector`(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector vec, DDVector vec_true, int kind_of_norm)
- void `relerr3_tdvector`(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, TDVector vec_true, int kind_of_norm)
- void `relerr3_qdvector`(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDVector vec, QDVector vec_true, int kind_of_norm)
- void `relerr3_mpfvector`(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFVector

- vec, MPFVector vec_true, int kind_of_norm)
- void relerr3_dmatrix(double * max_relerr, double * min_relerr, double * norm_relerr, DMatrix mat, DMatrix mat_true, int kind_of_norm) ... mat の相対誤差です。
- void relerr3_ddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDMatrix mat, DDMatrix mat_true, int kind_of_norm)
- void relerr3_tdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDMatrix mat, TDMatrix mat_true, int kind_of_norm)
- void relerr3_qdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDMatrix mat, QDMatrix mat_true, int kind_of_norm)
- void relerr3_mpfmatrix(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFMatrix mat, MPFMatrix mat_true, int kind_of_norm)

3.8 尾崎スキームと関連関数

以下の関数は、尾崎スキームおよび多倍長精度演算に必要です。

- void extract_dvector(DVector ret_high_vec, DVector ret_low_vec, DVector org_vec, double num_bits) ... $ret_high + ret_low = org_vec$ となります。
- void split_dmatrix(DMatrix ret_high_mat, DMatrix ret_low_mat, DMatrix org_mat) ... org_mat を行方向に ret_high_mat と ret_low_mat に分割します。
- void split_dmatrix_t(DMatrix ret_high_mat, DMatrix ret_low_mat, DMatrix org_mat) ... org_mat を列方向に ret_high_mat と ret_low_mat に分割します。
- int split_ddvector_dvec(DVector ret_vec[], int num_div, DDVector org_vec) ... org_vec を $ret_vec[0] + ret_vec[1] + \dots + ret_vec[num_div - 1]$ に分割します。
- int split_tdvector_dvec(DVector ret_vec[], int num_div, TDVector org_vec)
- void absmax_ddvector(double ret[DDSIZE], long int * max_index, DDVector vec) ... vec の要素のうち絶対値が最大のものを取得します。
- void absmax_tdvector(double ret[TDSIZE], long int * max_index, TDVector vec)
- void absmax_qdvector(double ret[QDSIZE], long int * max_index, QDVector vec)
- void absmax_mpfvector(mpf_t ret, long int * max_index, MPFVector vec)
- void add_ddvector_dvec(DDVector c, DDVector a, DVector b) ... $c := a + (double)b$
- void add_tdvector_dvec(TDVector c, TDVector a, DVector b)
- void add_qdvector_dvec(QDVector c, QDVector a, DVector b)
- void add_mpfvector_dvec(MPFVector c, MPFVector a, DVector b) ;
- void sub_ddvector_dvec(DDVector c, DDVector a, DVector b) ... $c := a - (double)b$
- void sub_tdvector_dvec(TDVector c, TDVector a, DVector b)
- void sub_qdvector_dvec(QDVector c, QDVector a, DVector b)
- void sub_mpfvector_dvec(MPFVector c, MPFVector a, DVector b)
- void subst_dvector_qdvec(DVector c, QDVector a) ... $c := (DDVector)a$
- void absmax_row_ddmatrix(double mu[DDSIZE], long int * max_j, long int row_index, DDMatrix

- mat) ... max_j 番目の行で絶対値が最大の要素を返します。
- void absmax_row_tdmatrix(double mu[TDSIZE], long int * max_j, long int row_index, TDMatrix mat)
 - void absmax_row_qdmatrix(double mu[QDSIZE], long int * max_j, long int row_index, QDMatrix mat)
 - void absmax_row_mpfmatrix(mpf_t mu, long int * max_j, long int row_index, MPFMatrix mat)
 - void add_ddmatrix_dmat(DDMatrix c, DDMatrix a, DMatrix b) ... $c := a + (DMatrix)b$
 - void add_tdmatrix_dmat(TDMatrix c, TDMatrix a, DMatrix b)
 - void add_qdmatrix_dmat(QDMatrix c, QDMatrix a, DMatrix b)
 - void add_mpfmatrix_dmat(MPFMatrix c, MPFMatrix a, DMatrix b)
 - void sub_ddmatrix_dmat(DDMatrix c, DDMatrix a, DMatrix b) ... $c := a - (DMatrix)b$
 - void sub_tdmatrix_dmat(TDMatrix c, TDMatrix a, DMatrix b)
 - void sub_qdmatrix_dmat(QDMatrix c, QDMatrix a, DMatrix b)
 - void sub_mpfmatrix_dmat(MPFMatrix c, MPFMatrix a, DMatrix b)
 - int split_ddmatrix_dmat(DMatrix ret_mat[], int num_div, DDMatrix org_mat) *cdots* org_mat を行方向に ret_mat[] に分割し、実際の分割数を返します。
 - int split_tdmatrix_dmat(DMatrix ret_mat[], int num_div, TDMatrix org_mat)
 - int split_qdmatrix_dmat(DMatrix ret_mat[], int num_div, QDMatrix org_mat)
 - int split_mpfmatrix_dmat(DMatrix ret_mat[], int num_div, MPFMatrix org_mat)
 - void absmax_col_ddmatrix(double mu[DDSIZE], long int * max_i, long int col_index, DDMatrix mat) ... mat の各列について絶対値が最大の要素とその (i,j) インデックスを返します。
 - void absmax_col_tdmatrix(double mu[TDSIZE], long int * max_i, long int col_index, TDMatrix mat)
 - void absmax_col_qdmatrix(double mu[QDSIZE], long int * max_i, long int col_index, QDMatrix-mat)
 - void absmax_col_mpfmatrix(mpf_t mu, long int * max_i, long int col_index, MPFMatrix mat)
 - int split_ddmatrix_t_dmat(DMatrix ret_mat[], int num_div, DDMatrix org_mat) ... org_mat を列方向に ret_mat[] に分割し、実際の分割数を返します。
 - int split_tdmatrix_t_dmat(DMatrix ret_mat[], int num_div, TDMatrix org_mat)
 - int split_qdvector_dvec(DVector ret_vec[], int num_div, QDVector org_vec)
 - int split_qdmatrix_t_dmat(DMatrix ret_mat[], int num_div, QDMatrix org_mat)
 - int split_mpfmatrix_t_dmat(DMatrix ret_mat[], int num_div , MPFMatrix org_mat)
 - void subst_qdmatrix_dmat(QDMatrix c, DMatrix a)
 - void subst_dmatrix_qdmat(DMatrix c, QDMatrix a)
 - void mul_ddmatrix_oz(DDMatrix ret, DDMatrix a, int max_num_div_a, DDMatrix b, int max_num_div_b) *cdots* 尾崎スキームに基づく行列乗算です。
 - void mul_tdmatrix_oz(TDMatrix ret, TDMatrix a, int max_num_div_a, TDMatrix b, int max_num_div_b)
 - void mul_qdmatrix_oz(QDMatrix ret, QDMatrix a, int max_num_div_a, QDMatrix b, int max_num_div_b)

- `void mul_mpfmatrix_oz(MPFMatrix ret, MPFMatrix a, int max_num_div_a, MPFMatrix b, int max_num_div_b)`
- `void mul_ddmatrix_ddvec_oz(DDVector ret, DDMatrix a, int max_num_div_a, DDVector vb, int max_num_div_vb)` ... 尾崎スキームに基づく行列ベクトル乗算です。
- `void mul_tdmatrix_tdvec_oz(TDVector ret, TDMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)`
- `void mul_qdmatrix_qdvec_oz(QDVector ret, QDMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)`
- `void mul_mpfmatrix_dvec_oz(MPFVector ret, MPFMatrix a, int max_num_div_a, MPFVector vb, int max_num_div_vb)`

第 4 章

LU 分解

BNCmatmul は、各種の LU 分解と、分解された連立一次方程式を解くための対応するソルバを備えています。

4.1 関数一覧

- `int DLUdecomp(DMatrix A)` ... ピボット選択を行わずに行列 A を LU 分解します。
- `int DDLUdecomp(DDMatrix A)`
- `int TDLUdecomp(TDMatrix A)`
- `int QDLUdecomp(QDMatrix A)`
- `int MPFLUdecomp(MPFMatrix A)`
- `int SolveDLS(DVector x, DMatrix LU, DVector b)` ... $(LU)x = b$ を x について解きます。
- `int SolveDDLSP(DDVector x, DDMatrix LU, DDVector b)`
- `int SolveTDLSP(TDVector x, TDMatrix LU, TDVector b)`
- `int SolveQDLSP(QDVector x, QDMatrix LU, QDVector b)`
- `int SolveMPFLSP(MPFVector x, MPFMatrix LU, MPFVector b)`
- `int DLUdecompP(DMatrix A, long int ch[])` ... 部分ピボット選択を用いて行列 A を LU 分解します。行の順序は `ch[]` に格納されます。
- `int DDLUdecompP(DDMatrix A, long int ch[])`
- `int TDLUdecompP(TDMatrix A, long int ch[])`
- `int QDLUdecompP(QDMatrix A, long int ch[])`
- `int MPFLUdecompP(MPFMatrix A, long int ch[])`
- `int SolveDLSP(DVector x, DMatrix LU, DVector b, long int ch[])` ... `ch[]` を行番号として用いて、 $(LU)x = b$ を x について解きます。
- `int SolveDDLSP(DDVector x, DDMatrix LU, DDVector b, long int ch[])`
- `int SolveTDLSP(TDVector x, TDMatrix LU, TDVector b, long int ch[])`
- `int SolveQDLSP(QDVector x, QDMatrix LU, QDVector b, long int ch[])`
- `int SolveMPFLSP(MPFVector x, MPFMatrix LU, MPFVector b, long int ch[])`
- `int DLUdecompC(DMatrix A, long int row_ch[], long int col_ch[])` ... 完全ピボット選択を用いて行列 A を LU 分解します。行の順序は `row_ch[]` に、列の順序は `col_ch[]` に格納されます。

- `int DDLUdecompC(DDMatrix A, long int row_ch[], long int col_ch[])`
- `int TDLUdecompC(TDMatrix A, long int row_ch[], long int col_ch[])`
- `int QDLUdecompC(QDMatrix A, long int row_ch[], long int col_ch[])`
- `int MPFLUdecompC(MPFMatrix A, long int row_ch[], long int col_ch[])`
- `int SolveDLSC(DVector x, DMatrix LU, DVector b, long int row_ch[], long int col_ch[])`
 $\cdots$ `row_ch[]` と `col_ch[]` をそれぞれ行番号と列番号として用いて, $(LU)x = b$ を x について解きます.
- `int SolveDDLSC(DDVector x, DDMatrix LU, DDVector b, long int row_ch[], long int col_ch[])`
- `int SolveTDLSC(TDVector x, TDMatrix LU, TDVector b, long int row_ch[], long int col_ch[])`
- `int SolveQDLSC(QDVector x, QDMatrix LU, QDVector b, long int row_ch[], long int col_ch[])`
- `int SolveMPFLSC(MPFVector x, MPFMatrix LU, MPFVector b, long int row_ch[], long int col_ch[])`
- `int DLUdecomp_strassen(DMatrix A, long int min_dim)` $\cdots$ Strassen の行列乗算を用いて LU 分解します.
- `int DDLUdecomp_strassen(DDMatrix A, long int min_dim)`
- `int TDLUdecomp_strassen(TDMatrix A, long int min_dim)`
- `int QDLUdecomp_strassen(QDMatrix A, long int min_dim)`
- `int MPFLUdecomp_strassen(MPFMatrix A, long int min_dim)`
- `int DLUdecomp_strassenPM(DMatrix A, long int ch[], long int min_dim)` $\cdots$ Strassen の行列乗算と部分ピボット選択を用いて行列 A を LU 分解します. 行の順序は `ch[]` に格納されます.
- `int DDLUdecomp_strassenPM(DDMatrix A, long int ch[], long int min_dim)`
- `int TDLUdecomp_strassenPM(TDMatrix A, long int ch[], long int min_dim)`
- `int QDLUdecomp_strassenPM(QDMatrix A, long int ch[], long int min_dim)`
- `int MPFLUdecomp_strassenPM(MPFMatrix A, long int ch[], long int min_dim)`
- `int DLUdecomp_oz(DMatrix A, long int min_dim, int max_num_div)` $\cdots$ `max_num_div` を最大分割数として設定し, Ozaki スキームを用いて LU 分解します.
- `int DDLUdecomp_oz(DDMatrix A, long int min_dim, int max_num_div)`
- `int TDLUdecomp_oz(TDMatrix A, long int min_dim, int max_num_div)`
- `int QDLUdecomp_oz(QDMatrix A, long int min_dim, int max_num_div)`
- `int MPFLUdecomp_oz(MPFMatrix A, long int min_dim, int max_num_div)`
- `int DLUdecomp_ozPM(DMatrix A, long int ch[], long int min_dim, int max_num_div)` $\cdots$ `max_num_div` を最大分割数として設定し, 部分ピボット選択と Ozaki スキームを用いて LU 分解します.
- `int DDLUdecomp_ozPM(DDMatrix A, long int ch[], long int min_dim, int max_num_div)`
- `int TDLUdecomp_ozPM(TDMatrix A, long int ch[], long int min_dim, int max_num_div)`
- `int QDLUdecomp_ozPM(QDMatrix A, long int ch[], long int min_dim, int max_num_div)`
- `int MPFLUdecomp_ozPM(MPFMatrix A, long int ch[], long int min_dim, int max_num_div)`

第 5 章

配列・多項式と代数方程式のソルバ

前章までに述べた多倍長演算と線形計算の応用として、本章では高速化した代数方程式のソルバを紹介します。

5.1 配列と多項式のデータ型

多項式は式 (5.1) のように表現されます。

$$p_n(x) = \sum_{k=0}^n a_k x^k \quad (5.1)$$

本ライブラリは以下の多項式型と配列型を提供します。これらはすべて `poly.h` で宣言されています (copyright © 2002–2025 Tomonori Kouya、GNU LGPL v3 以降のもとで配布)。

5.1.1 多項式型

単精度 (FPoly) と単精度複素数 (CFPoly)

- `fpoly` — single-precision polynomial; pointer type `FPoly`. Members: `float *coef`, `long int deg`, `long int max_len`.
- `cfpoly` — single-precision complex polynomial; pointer type `CFPoly`. Members: `fcmplx *coef`, `long int deg`, `long int max_len`.

倍精度 (DPoly) と倍精度複素数 (CDPoly)

- `dpoly` — double-precision polynomial; pointer type `DPoly`. Members: `double *coef`, `long int deg`, `long int max_len`.
- `cdpoly` — double-precision complex polynomial; pointer type `CDPoly`. Members: `double _Complex *coef`, `long int deg`, `long int max_len`.

double-double (DDPoly) と複素 double-double (CDDPoly)

- `ddpoly` — double-double polynomial; pointer type `DDPoly`. Members: `ddfloat *coef`, `ddfloat zero`, `long int deg`, `long int max_len`.

- `cddpoly` — complex double-double polynomial; pointer type `CDDPoly`. Members: `cddfloat *coef`, `cddfloat zero`, `long int deg`, `long int max_len`.

triple-double (TDPoly) と複素 triple-double (CTDPoly)

- `tdpoly` — triple-double polynomial; pointer type `TDPoly`. Members: `tdfloat *coef`, `tdfloat zero`, `long int deg`, `long int max_len`.
- `ctdpoly` — complex triple-double polynomial; pointer type `CTDPoly`. Members: `ctdfloat *coef`, `ctdfloat zero`, `long int deg`, `long int max_len`.

quadruple-double (QDPoly) と複素 quadruple-double (CQDPoly)

- `qdpoly` — quadruple-double polynomial; pointer type `QDPoly`. Members: `qdfloat *coef`, `qdfloat zero`, `long int deg`, `long int max_len`.
- `cqdpoly` — complex quadruple-double polynomial; pointer type `CQDPoly`. Members: `cqdfloat *coef`, `cqdfloat zero`, `long int deg`, `long int max_len`.

多倍長精度 (MPFPoly) と複素多倍長精度 (CMPFPoly)

`USE_GMP` が定義されているときに利用できます。

- `mpfpoly` — GMP multiple-precision polynomial; pointer type `MPFPoly`. Members: `unsigned long int prec`, `mpf_t *coef`, `mpf_t zero`, `long int deg`, `long int max_len`.
- `cmpfpoly` — MPC complex multiple-precision polynomial; pointer type `CMPFPoly`. Members: `unsigned long int prec`, `mpc_t *coef`, `mpc_t zero`, `long int deg`, `long int max_len`.
- `_bncold_cmpfpoly` — legacy complex multiple-precision polynomial (old implementation); pointer type `_bncold_CMPFPoly`. Members: `unsigned long int prec`, `mpfcmplx *coef`, `long int deg`, `long int max_len`.

5.1.2 配列型

- `farray` — float array; pointer type `FArray`. Members: `float *array`, `long int size`.
- `cfarray` — float complex array; pointer type `CFArray`. Members: `fcmplx *array`, `long int size`.
- `darray` — double array; pointer type `DArray`. Members: `double *array`, `long int size`.
- `cdarray` — double complex array; pointer type `CDArray`. Members: `dcmplx *array`, `long int size`.
- `mpfarray` — GMP multiple-precision array (requires `USE_GMP`); pointer type `MPFArray`. Members: `unsigned long prec`, `mpf_t *array`, `long int size`.
- `cmpfarray` — MPC complex multiple-precision array (requires `USE_GMP`); pointer type `CMPFArray`. Members: `unsigned long prec`, `mpc_t *array`, `long int size`.
- `_bncold_cmpfarray` — legacy complex multiple-precision array (requires `USE_GMP`); pointer type

`_bncold_CMPFArray`. Members: `unsigned long prec`, `mpfcmplx *array`, `long int size`.

5.2 関数一覧

5.2.1 単精度多項式関数 (poly.c — FPoly)

- `FPoly init_fpoly(long int max_len)`
- `void free_fpoly(FPoly pol)`
- `float get_fpoly_i(FPoly pol, long int i)` (alias: `gfpi`)
- `void set_fpoly_i(FPoly pol, long int i, float val)` (alias: `sfpi`)
- `long int setdegree_fpoly(FPoly pol)`
- `void print_fpoly(FPoly pol)`
- `void add_fpoly(FPoly ret, FPoly a, FPoly b)`
- `void sub_fpoly(FPoly ret, FPoly a, FPoly b)`
- `void cmul_fpoly(FPoly ret, float c, FPoly a)`
- `void subst_fpoly(FPoly ret, FPoly a)`
- `void set0_fpoly(FPoly pol)`
- `long int max_abscoef_fpoly(FPoly pol)`
- `long int num_nonzero_fpoly(FPoly pol)`
- `void diff_fpoly(FPoly pol)`
- `float eval_fpoly(FPoly pol, float x)`
- `float eval_diff_fpoly(FPoly pol, float x)`
- `void ceval_fpoly(FCmplx ret, FPoly pol, FCmplx x)`
- `void ceval_diff_fpoly(FCmplx ret, FPoly pol, FCmplx x)`

5.2.2 倍精度多項式関数 (poly.c — DPoly)

- `DPoly init_dpoly(long int max_len)`
- `void free_dpoly(DPoly pol)`
- `DPoly init_set_dpoly(DPoly org)`
- `double get_dpoly_i(DPoly pol, long int i)` (alias: `gdpi`)
- `void set_dpoly_i(DPoly pol, long int i, double val)` (alias: `sdpi`)
- `long int setdegree_dpoly(DPoly pol)`
- `void print_dpoly(DPoly pol)`
- `void add_dpoly(DPoly ret, DPoly a, DPoly b)`
- `void sub_dpoly(DPoly ret, DPoly a, DPoly b)`
- `void add2_dpoly(DPoly ret, DPoly a)`
- `void sub2_dpoly(DPoly ret, DPoly a)`
- `void mul_dpoly(DPoly ret, DPoly a, DPoly b)`
- `void cmul_dpoly(DPoly ret, double c, DPoly a)`

- void subst_dpoly(DPoly ret, DPoly a)
- void set0_dpoly(DPoly pol)
- long int max_abscoef_dpoly(DPoly pol)
- long int num_nonzero_dpoly(DPoly pol)
- void diff_dpoly(DPoly pol)
- double eval_dpoly_horner(DPoly pol, double x) (also aliased as eval_dpoly)
- double eval_dpoly_estrin(DPoly pol, double x)
- double eval_diff_dpoly(DPoly pol, double x)
- void div_dpoly(DPoly quo, DPoly rem, DPoly a, DPoly b)
- void xpow_mul_dpoly(DPoly ret, long int n, DPoly a)
- void ceval_dpoly_horner(DCmplx ret, DPoly pol, DCmplx x) (also aliased as ceval_dpoly)
- void ceval_dpoly_estrin(DCmplx ret, DPoly pol, DCmplx x)
- void ceval_diff_dpoly(DCmplx ret, DPoly pol, DCmplx x)
- double _bncavx2_eval_dpoly_estrin(DPoly a, double x) (AVX2)
- void _bncavx2_ceval_dpoly_estrin(DCmplx ret, DPoly a, DCmplx x) (AVX2)

5.2.3 倍精度複素多項式関数 (poly.c — CDPoly)

- CDPoly init_cdpoly(long int max_len)
- void free_cdpoly(CDPoly pol)
- CDPoly init_set_cdpoly(CDPoly org)
- CDPoly init_set_cdpoly_dpoly(DPoly org)
- void set_cdpoly_dpoly(CDPoly ret, DPoly src)
- double _Complex get_cdpoly_i(CDPoly pol, long int i) (alias: gcdpi)
- void set_cdpoly_i(CDPoly pol, long int i, double _Complex val) (alias: scdpi)
- void set_cdpoly_i_d(CDPoly pol, long int i, double val)
- long int setdegree_cdpoly(CDPoly pol)
- void print_cdpoly(CDPoly pol)
- void add_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void sub_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void add2_cdpoly(CDPoly ret, CDPoly a)
- void sub2_cdpoly(CDPoly ret, CDPoly a)
- void mul_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void mul2_cdpoly(CDPoly ret, CDPoly a)
- void cmul_cdpoly(CDPoly ret, double _Complex c, CDPoly a)
- void subst_cdpoly(CDPoly ret, CDPoly a)
- void set0_cdpoly(CDPoly pol)
- long int max_abscoef_cdpoly(CDPoly pol)
- long int num_nonzero_cdpoly(CDPoly pol)

- void diff_cdpoly(CDPoly pol)
- double __Complex eval_cdpoly_horner(CDPoly a, double __Complex x) (also aliased as eval_cdpoly)
- double __Complex eval_cdpoly_estrin(CDPoly a, double __Complex x)
- double __Complex eval_diff_cdpoly(CDPoly a, double __Complex x)
- void div_cdpoly(CDPoly quo, CDPoly rem, CDPoly a, CDPoly b)
- void xpow_mul_cdpoly(CDPoly ret, long int n, CDPoly a)

5.2.4 double-double 多項式関数 (poly.c — DDPoly, CDDPoly)

- DDPoly init_ddpoly(long int max_len)
- void free_ddpoly(DDPoly pol)
- double * get_ddpoly_i(DDPoly pol, long int i) (alias: gddpi)
- void set_ddpoly_i(DDPoly pol, long int i, double * val) (alias: sddpi)
- void set_ddpoly_i_si(DDPoly pol, long int i, long val)
- void set_ddpoly_i_ui(DDPoly pol, long int i, unsigned long val)
- void set_ddpoly_i_d(DDPoly pol, long int i, double val)
- void set_ddpoly_i_str(DDPoly pol, long int i, const char * str, int base)
- long int setdegree_ddpoly(DDPoly pol)
- void print_ddpoly(DDPoly pol)
- void add_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void sub_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void mul_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void cmul_ddpoly(DDPoly ret, double * c, DDPoly a)
- void subst_ddpoly(DDPoly ret, DDPoly a)
- void set0_ddpoly(DDPoly pol)
- long int max_abscoef_ddpoly(DDPoly pol)
- long int num_nonzero_ddpoly(DDPoly pol)
- void diff_ddpoly(DDPoly pol)
- void eval_ddpoly_horner(double * ret, DDPoly a, double * x) (also aliased as eval_ddpoly)
- void eval_ddpoly_estrin(double * ret, DDPoly a, double * x)
- void eval_diff_ddpoly(double * ret, DDPoly a, double * x)
- void ceval_ddpoly_horner(cddfloat * ret, DDPoly a, cddfloat * x) (also aliased as ceval_ddpoly)
- void ceval_ddpoly_estrin(cddfloat * ret, DDPoly a, cddfloat * x)
- void ceval_diff_ddpoly(cddfloat * ret, DDPoly a, cddfloat * x)
- void _bncavx2_eval_ddpoly_estrin(double ret[DDSIZE], DDPoly a, double x[DDSIZE]) (AVX2)
- void _bncavx2_ceval_ddpoly_estrin(cddfloat * ret, DDPoly a, cddfloat * x) (AVX2)

複素 double-double 多項式 (CDDPoly) :

- CDDPoly init_cddpoly(long int max_len)
- void free_cddpoly(CDDPoly pol)
- CDDPoly init_set_cddpoly(CDDPoly org)
- CDDPoly init_set_cddpoly_ddpoly(DDPoly org)
- cddfloat * get_cddpoly_i(CDDPoly pol, long int i) (alias: gcddpi)
- void set_cddpoly_i(CDDPoly pol, long int i, cddfloat * val) (alias: scddpi)
- long int setdegree_cddpoly(CDDPoly pol)
- void print_cddpoly(CDDPoly pol)
- void add_cddpoly(CDDPoly ret, CDDPoly a, CDDPoly b)
- void sub_cddpoly(CDDPoly ret, CDDPoly a, CDDPoly b)
- void mul_cddpoly(CDDPoly ret, CDDPoly a, CDDPoly b)
- void subst_cddpoly(CDDPoly ret, CDDPoly a)
- void set0_cddpoly(CDDPoly pol)
- long int max_abscoef_cddpoly(CDDPoly pol)
- long int num_nonzero_cddpoly(CDDPoly pol)
- void diff_cddpoly(CDDPoly pol)
- void eval_cddpoly_horner(cddfloat * ret, CDDPoly a, cddfloat * x) (also aliased as eval_cddpoly)
- void eval_cddpoly_estrin(cddfloat * ret, CDDPoly a, cddfloat * x)
- void eval_diff_cddpoly(cddfloat * ret, CDDPoly a, cddfloat * x)
- void _bncavx2_eval_cddpoly_estrin(cddfloat * ret, CDDPoly a, cddfloat * x) (AVX2)

5.2.5 triple-double 多項式関数 (poly.c — TDPoly)

- TDPoly init_tdpoly(long int max_len)
- void free_tdpoly(TDPoly pol)
- tdfloat get_tdpoly_i_float(TDPoly pol, long int i)
- double * get_tdpoly_i(TDPoly pol, long int i) (alias: gtdpi)
- void set_tdpoly_i(TDPoly pol, long int i, double val[TDSIZE]) (alias: stdpi)
- void set_tdpoly_i_si(TDPoly pol, long int i, long val)
- void set_tdpoly_i_ui(TDPoly pol, long int i, unsigned long val)
- void set_tdpoly_i_d(TDPoly pol, long int i, double val)
- void set_tdpoly_i_str(TDPoly pol, long int i, const char * str, int base)
- long int setdegree_tdpoly(TDPoly pol)
- void print_tdpoly(TDPoly pol)
- void add_tdpoly(TDPoly c, TDPoly a, TDPoly b) $\cdots c = a + b$
- void add2_tdpoly(TDPoly c, TDPoly a) $\cdots c += a$
- void sub_tdpoly(TDPoly c, TDPoly a, TDPoly b) $\cdots c = a - b$
- void sub2_tdpoly(TDPoly c, TDPoly a) $\cdots c -= a$

- `void mul_tdpoly(TDPoly c, TDPoly a, TDPoly b) ...  $c = a \times b$`
- `void cmul_tdpoly(TDPoly c, double val[TDSIZE], TDPoly a) ...  $c = val \times a$`
- `void cmul2_tdpoly(TDPoly c, double val[TDSIZE]) ...  $c *= val$`
- `void subst_tdpoly(TDPoly c, TDPoly a) ...  $c := a$`
- `void set0_tdpoly(TDPoly c) ...  $c := 0$`
- `long int num_nonzero_tdpoly(TDPoly c)`
- `long int max_abscoef_tdpoly(TDPoly c)`
- `void diff_tdpoly(TDPoly a) ...  $a := a'(x)$`
- `void eval_tdpoly_horner(double ret[TDSIZE], TDPoly a, double x[TDSIZE])` (also aliased as `eval_tdpoly`)
- `void eval_tdpoly_estrin(double ret[TDSIZE], TDPoly a, double x[TDSIZE])`
- `void eval_diff_tdpoly(double ret[TDSIZE], TDPoly a, double x[TDSIZE])`
- `void ceval_tdpoly_horner(ctdfloat * ret, TDPoly a, ctdfloat * x)` (also aliased as `ceval_tdpoly`)
- `void ceval_tdpoly_estrin(ctdfloat * ret, TDPoly a, ctdfloat * x)`
- `void ceval_diff_tdpoly(ctdfloat * ret, TDPoly a, ctdfloat * x)`
- `void _bncavx2_eval_tdpoly_estrin(double ret[TDSIZE], TDPoly a, double x[TDSIZE])` (AVX2)
- `void _bncavx2_ceval_tdpoly_estrin(ctdfloat * ret, TDPoly a, ctdfloat * x)` (AVX2)

5.2.6 quadruple-double 多項式関数 (poly.c — QDPoly)

- `QDPoly init_qdpoly(long int max_len)`
- `void free_qdpoly(QDPoly pol)`
- `qdfloat get_qdpoly_i_float(QDPoly pol, long int i)`
- `double * get_qdpoly_i(QDPoly pol, long int i)` (alias: `gqdpi`)
- `void set_qdpoly_i(QDPoly pol, long int i, double val[QDSIZE])` (alias: `sqdpi`)
- `void set_qdpoly_i_si(QDPoly pol, long int i, long val)`
- `void set_qdpoly_i_ui(QDPoly pol, long int i, unsigned long val)`
- `void set_qdpoly_i_d(QDPoly pol, long int i, double val)`
- `void set_qdpoly_i_str(QDPoly pol, long int i, const char * str, int base)`
- `long int setdegree_qdpoly(QDPoly pol)`
- `void print_qdpoly(QDPoly pol)`
- `void add_qdpoly(QDPoly c, QDPoly a, QDPoly b) ...  $c = a + b$`
- `void add2_qdpoly(QDPoly c, QDPoly a) ...  $c += a$`
- `void sub_qdpoly(QDPoly c, QDPoly a, QDPoly b) ...  $c = a - b$`
- `void sub2_qdpoly(QDPoly c, QDPoly a) ...  $c -= a$`
- `void mul_qdpoly(QDPoly c, QDPoly a, QDPoly b) ...  $c = a \times b$`
- `void cmul_qdpoly(QDPoly c, double val[QDSIZE], QDPoly a) ...  $c = val \times a$`
- `void cmul2_qdpoly(QDPoly c, double val[QDSIZE]) ...  $c *= val$`
- `void subst_qdpoly(QDPoly c, QDPoly a) ...  $c := a$`

- void set0_qdpoly(QDPoly c) ... $c := 0$
- long int num_nonzero_qdpoly(QDPoly c)
- long int max_abscoef_qdpoly(QDPoly c)
- void diff_qdpoly(QDPoly a) ... $a := a'(x)$
- void eval_qdpoly_horner(double ret[QDSIZE], QDPoly a, double x[QDSIZE]) (also aliased as eval_qdpoly)
- void eval_qdpoly_estrin(double ret[QDSIZE], QDPoly a, double x[QDSIZE])
- void eval_diff_qdpoly(double ret[QDSIZE], QDPoly a, double x[QDSIZE])
- void ceval_qdpoly_horner(cqdfloat * ret, QDPoly a, cqdfloat * x) (also aliased as ceval_qdpoly)
- void ceval_qdpoly_estrin(cqdfloat * ret, QDPoly a, cqdfloat * x)
- void ceval_diff_qdpoly(cqdfloat * ret, QDPoly a, cqdfloat * x)
- void ceval_hes_qdmatrix(cqdfloat * ret, QDMatrix hes, cqdfloat * x)
- void _bncavx2_eval_qdpoly_estrin(double ret[QDSIZE], QDPoly a, double x[QDSIZE]) (AVX2)
- void _bncavx2_ceval_qdpoly_estrin(cqdfloat * ret, QDPoly a, cqdfloat * x) (AVX2)

多項式型の間の変換関数 (USE_GMP が必要) :

- void set_ddpoly_i_mpf(DDPoly pol, long int i, mpf_t val)
- void set_tdpoly_i_mpf(TDPoly pol, long int i, mpf_t val)
- void set_qdpoly_i_mpf(QDPoly pol, long int i, mpf_t val)
- void set_cddpoly_i_mpf(CDDPoly pol, long int i, mpc_t val)
- DDPoly init_set_ddpoly_mpfpoly(MPFPoly org_pol)
- TDPoly init_set_tdpoly_mpfpoly(MPFPoly org_pol)
- QDPoly init_set_qdpoly_mpfpoly(MPFPoly org_pol)
- CDDPoly init_set_cddpoly_cmpfpoly(CMPFPoly org_pol)

5.2.7 多倍長精度多項式関数 (poly.c — MPFPoly、USE_GMP が必要)

- MPFPoly init_mpfpoly(long int max_len)
- MPFPoly init2_mpfpoly(long int max_len, unsigned long prec)
- void free_mpfpoly(MPFPoly pol)
- mpf_ptr get_mpfpoly_i(MPFPoly pol, long int i) (alias: gmpfpi)
- void set_mpfpoly_i(MPFPoly pol, long int i, mpf_t val) (alias: smpfpi)
- void set_mpfpoly_i_d(MPFPoly pol, long int i, double val)
- void set_mpfpoly_i_si(MPFPoly pol, long int i, long val)
- void set_mpfpoly_i_ui(MPFPoly pol, long int i, unsigned long val)
- void set_mpfpoly_i_str(MPFPoly pol, long int i, const char * str, int base)
- void print_mpfpoly(MPFPoly pol)
- void print_fdmpfpoly(FPoly fp, DPoly dp, MPFPoly mp)
- void add_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)

- void sub_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)
- void add2_mpfpoly(MPFPoly ret, MPFPoly a)
- void sub2_mpfpoly(MPFPoly ret, MPFPoly a)
- void mul_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)
- void cmul_mpfpoly(MPFPoly ret, mpf_t c, MPFPoly a)
- void subst_mpfpoly(MPFPoly ret, MPFPoly a)
- void set0_mpfpoly(MPFPoly pol)
- long int max_abscoef_mpfpoly(MPFPoly pol)
- long int num_nonzero_mpfpoly(MPFPoly pol)
- long int setdegree_mpfpoly(MPFPoly pol)
- void diff_mpfpoly(MPFPoly pol)
- void eval_mpfpoly_horner(mpf_t ret, MPFPoly pol, mpf_t x) (also aliased as eval_mpfpoly)
- void eval_mpfpoly_estrin(mpf_t ret, MPFPoly pol, mpf_t x)
- void eval_diff_mpfpoly(mpf_t ret, MPFPoly pol, mpf_t x)
- void ceval_mpfpoly_horner(mpc_t ret, MPFPoly pol, mpc_t x) (also aliased as ceval_mpfpoly)
- void ceval_mpfpoly_estrin(mpc_t ret, MPFPoly pol, mpc_t x)
- void ceval_diff_mpfpoly(mpc_t ret, MPFPoly pol, mpc_t x)
- void ceval_hes_mpfmatrix(mpc_t ret, MPFMatrix hes, mpc_t x)

複素多倍長精度多項式 (CMPFPoly、USE_GMP が必要) :

- CMPFPoly init_cmpfpoly(long int max_len)
- CMPFPoly init2_cmpfpoly(long int max_len, unsigned long prec)
- void free_cmpfpoly(CMPFPoly pol)
- mpc_ptr get_cmpfpoly_i(CMPFPoly pol, long int i) (alias: gcmpfpi)
- void set_cmpfpoly_i(CMPFPoly pol, long int i, mpc_t val) (alias: scmpfpi)
- void set_cmpfpoly_i_cd(CMPFPoly pol, long int i, double _Complex val)
- void set_cmpfpoly_i_d(CMPFPoly pol, long int i, double val)
- void set_cmpfpoly_i_si_si(CMPFPoly pol, long int i, long re, long im)
- void set_cmpfpoly_i_si(CMPFPoly pol, long int i, long val)
- void set_cmpfpoly_i_ui(CMPFPoly pol, long int i, unsigned long val)
- void set_cmpfpoly_i_str(CMPFPoly pol, long int i, const char * str, int base)
- long int setdegree_cmpfpoly(CMPFPoly pol)
- void print_cmpfpoly(CMPFPoly pol)
- void subst_cmpfpoly_mpfpoly(CMPFPoly ret, MPFPoly pol)
- CMPFPoly init_set_cmpfpoly_mpfpoly(MPFPoly org)
- CMPFPoly init_set_cmpfpoly(CMPFPoly org)
- void add_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)
- void sub_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)
- void add2_cmpfpoly(CMPFPoly ret, CMPFPoly a)
- void sub2_cmpfpoly(CMPFPoly ret, CMPFPoly a)

- void mul_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)
- void cmul_cmpfpoly(CMPFPoly ret, mpc_t c, CMPFPoly a)
- void subst_cmpfpoly(CMPFPoly ret, CMPFPoly a)
- void set0_cmpfpoly(CMPFPoly pol)
- long int max_abscoef_cmpfpoly(CMPFPoly pol)
- long int num_nonzero_cmpfpoly(CMPFPoly pol)
- void diff_cmpfpoly(CMPFPoly pol)
- void eval_cmpfpoly_horner(mpc_t ret, CMPFPoly pol, mpc_t x) (also aliased as eval_cmpfpoly)
- void eval_cmpfpoly_estrin(mpc_t ret, CMPFPoly pol, mpc_t x)
- void eval_diff_cmpfpoly(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_cmpfpoly_horner(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_cmpfpoly_estrin(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_diff_cmpfpoly(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_hes_cmpfmatrix(mpc_t ret, CMPFMatrix hes, mpc_t x)

5.2.8 配列関数 (array.c)

- FArray init_farray(long int size)
- void free_farray(FArray a)
- float get_farray_i(FArray a, long int i) (alias: gfai)
- void set_farray_i(FArray a, long int i, float val) (alias: sfai)
- void subst_farray(FArray ret, FArray src)
- void print_farray(FArray a)
- CArray init_carray(long int size)
- void free_carray(CArray a)
- FCmplx get_carray_i(CArray a, long int i) (alias: gcgai)
- void set_carray_i(CArray a, long int i, FCmplx val) (alias: scgai)
- void subst_carray(CArray ret, CArray src)
- void print_carray(CArray a)
- DArray init_darray(long int size)
- void free_darray(DArray a)
- double get_darray_i(DArray a, long int i) (alias: gdai)
- void set_darray_i(DArray a, long int i, double val) (alias: sdai)
- void subst_darray(DArray ret, DArray src)
- void print_darray(DArray a)
- CArray init_cdarray(long int size)
- void free_cdarray(CArray a)
- DCmplx get_cdarray_i(CArray a, long int i) (alias: gcdai)
- void set_cdarray_i(CArray a, long int i, DCmplx val) (alias: scdai)

- void subst_cdarray(CDArray ret, CDArray src)
- void print_cdarray(CDArray a)

多倍長精度配列関数 (USE_GMP が必要) :

- MPFArray init_mpfarray(long int size)
- MPFArray init2_mpfarray(long int size, unsigned long prec)
- void free_mpfarray(MPFArray a)
- mpf_ptr get_mpfarray_i(MPFArray a, long int i) (alias: gmpfai)
- void set_mpfarray_i(MPFArray a, long int i, mpf_t val) (alias: smpfai)
- void set_mpfarray_i_d(MPFArray a, long int i, double val)
- void set_mpfarray_i_ui(MPFArray a, long int i, unsigned long val)
- void subst_mpfarray(MPFArray ret, MPFArray src)
- void print_mpfarray(MPFArray a)
- CMPFArray init_cmpfarray(long int size)
- CMPFArray init2_cmpfarray(long int size, unsigned long prec)
- void free_cmpfarray(CMPFArray a)
- mpc_ptr get_cmpfarray_i(CMPFArray a, long int i) (alias: gcmpfai)
- void set_cmpfarray_i(CMPFArray a, long int i, mpc_t val) (alias: scmpfai)
- void set_cmpfarray_i_d(CMPFArray a, long int i, double val)
- void set_cmpfarray_i_ui(CMPFArray a, long int i, unsigned long val)
- void set_cmpfarray_i_real(CMPFArray a, long int i, mpf_t val)
- void subst_cmpfarray(CMPFArray ret, CMPFArray src)
- void print_cmpfarray(CMPFArray a)

5.2.9 DKA 求根法 (dka.c)

Durand–Kerner–Aberth (DKA) 法は、多項式のすべての根を同時に求めます。

- float fdka_center(FPoly pol)
- float fdka_radius(FPoly pol)
- void fdka_init(CFArray x, FPoly pol)
- long int fdka(CFArray ans, CFArray x_init, FPoly pol, long int maxtimes, float aeaps, float reps)
- double ddka_center(DPoly pol)
- double _Complex cddka_center(CDPoly pol)
- double ddka_radius(DPoly pol)
- double cddka_radius(CDPoly pol)
- void ddka_init(CDArray x, DPoly pol)
- void cddka_init(double _Complex[] x, CDPoly pol)
- void cddka_init_cdarray(CDArray x, CDPoly pol)
- long int ddka(CDArray ans, CDArray x_init, DPoly pol, long int maxtimes, double aeaps, double

reps)

- long int cddka(double __Complex[] ans, double __Complex[] x_init, CDPoly pol, long int maxtimes, double aeps, double reps)

double-double DKA (dd_dka.c) :

- void dd_dka_center(double ret[DDSIZE], DDPoly pol)
- void dd_dka_radius(double ret[DDSIZE], DDPoly pol)
- void dd_dka_init(CDDVector x, DDPoly pol)
- long int dd_dka(CDDVector ans, CDDVector x_init, DDPoly pol, long int maxtimes, double * aeps, double * reps)

triple-double DKA (td_dka.c) :

- void td_dka_center(double ret[TDSIZE], TDPoly pol)
- void td_dka_radius(double ret[TDSIZE], TDPoly pol)
- void td_dka_init(CTDVector x, TDPoly pol)
- long int td_dka(CTDVector ans, CTDVector x_init, TDPoly pol, long int maxtimes, double * aeps, double * reps)

quadruple-double DKA (qd_dka.c) :

- void qd_dka_center(double ret[QDSIZE], QDPoly pol)
- void qd_dka_radius(double ret[QDSIZE], QDPoly pol)
- void qd_dka_init(CQDVector x, QDPoly pol)
- long int qd_dka(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double * aeps, double * reps)
- long int qd_dka_mod(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])
- long int qd_petckovic(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])
- long int qd_aberth(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])

多倍長精度 DKA (mpf_dka.c、USE_GMP が必要) :

- void mpf_dka_center(mpf_t ret, MPFPoly pol)
- void mpf_dka_radius(mpf_t ret, MPFPoly pol)
- void mpf_dka_ozawa_radius(mpf_t ret, MPFPoly pol) ... Ozawa の初期半径: $r := (p(\text{center})/a_n)^{1/n}$
- void mpf_dka_init(CMPFArray x, MPFPoly pol)
- void mpf_dka_init2(CMPFArray x_init, MPFPoly pol, void (*) get_radius, void (*) get_center)
- long int mpf_dka(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t

- `aeps, mpf_t reps)`
- `long int mpf_dka_mod(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)` ... DKA 法、2 次収束
- `long int mpf_petckovic(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)` ... Petkovic 法、3 次収束
- `long int mpf_aberth(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)` ... Aberth 法、3 次収束

MPC 複素多倍長精度 DKA (`mpc_dka.c`、`USE_GMP` が必要) :

- `void mpc_dka_center(mpc_t ret, CMPFPoly pol)`
- `void mpc_dka_radius(mpf_t ret, CMPFPoly pol)`
- `void mpc_dka_ozawa_radius(mpf_t ret, CMPFPoly pol)`
- `void mpc_dka_init(CMPFArray x, CMPFPoly pol)`
- `void mpc_dka_init2(CMPFArray x_init, CMPFPoly pol, void (*) get_radius, void (*) get_center)`
- `long int mpc_dka(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpc_dka_mod(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpc_petckovic(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpc_aberth(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`

5.2.10 平野法 (`hirano.c`, `dd_hirano.c`)

平野法は、 $|p(x)| < \max_j |a_j x^j| \cdot r_\epsilon + a_\epsilon$ を満たす根 x を求めます。

- `double _Complex dhorner(double _Complex x, DPoly pol)` ... $p(x)$ の Horner 法による評価
- `double _Complex cdhorner(double _Complex x, CDPoly pol)`
- `void dcoef_horner(double _Complex[] ret, double _Complex x, DPoly pol)` ... $p(x+d)$ の係数
- `void cdcoef_horner(double _Complex[] ret, double _Complex x, CDPoly pol)`
- `long int absmax_dpoly(double * absmax, double _Complex x, DPoly pol)`
- `long int absmax_cdpoly(double * absmax, double _Complex x, CDPoly pol)`
- `double dget_plus_arg(double _Complex x)` ... $\arg(x) \in [0, 2\pi]$
- `long int dget_nearest_int(double x)`
- `double _Complex dget_min_branch(double _Complex x, double mu, double _Complex[] coef, long int i_num, long int i_den)`
- `long int dhirano(double _Complex * ret, double _Complex init_x, DPoly pol, double reps, double aeps, long int maxtimes)`

- long int cdhirano(double _Complex * ret, double _Complex init_x, CDPoly pol, double reps, double aeps, long int maxtimes)

多倍長精度平野法 (hirano.c、USE_GMP が必要) :

- void mpf_coef_horner(CMPFArray ret, mpc_t x, MPFPoly pol)
- void mpc_coef_horner(CMPFArray ret, mpc_t x, CMPFPoly pol)
- long int absmax_mpfpoly(mpf_t absmax, mpc_t x, MPFPoly pol)
- long int absmax_cmpfpoly(mpf_t absmax, mpc_t x, CMPFPoly pol)
- void mpf_get_plus_arg(mpf_t ret, mpc_t x)
- void mpf_get_nearest_int(mpf_t ret, mpf_t x)
- void mpf_get_min_branch(mpc_t ret, mpc_t x, mpf_t mu, CMPFArray coef, long int i_num, long int i_den)
- long int mpf_hirano(mpc_t ret, mpc_t init_x, MPFPoly pol, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int mpc_hirano(mpc_t ret, mpc_t init_x, CMPFPoly pol, mpf_t reps, mpf_t aeps, long int maxtimes)
- void ono_poly(MPFPoly pol, long int deg) ... 小野の問題多項式
- void wilkinson_poly(MPFPoly ret, long int n) ... Wilkinson 多項式 $(x-1)(x-2)\cdots(x-n)$
- void deflation_cmpfpoly(CMPFPoly ret, CMPFPoly pol, mpc_t root) ... デフレーション $p(x)/(x-r)$

double-double 平野法 (dd_hirano.c) :

- void dd_horner(cddfloat * ret, cddfloat * x, DDPoly pol)
- void dd_coef_horner(CDDVector ret, cddfloat * x, DDPoly pol)
- void cdd_horner(cddfloat * ret, cddfloat * x, CDDPoly pol)
- void cdd_coef_horner(CDDVector ret, cddfloat * x, CDDPoly pol)
- long int absmax_ddpoly(double absmax[DDSIZE], cddfloat * x, DDPoly pol)
- long int absmax_cddpoly(double absmax[DDSIZE], cddfloat * x, CDDPoly pol)
- void dd_get_plus_arg(double ret[DDSIZE], cddfloat * x)
- void dd_get_nearest_int(double ret[DDSIZE], double x[DDSIZE])
- void dd_get_min_branch(cddfloat * ret, cddfloat * x, double mu[DDSIZE], CDDVector coef, long int i_num, long int i_den)
- long int dd_hirano(cddfloat * ret, cddfloat * init_x, DDPoly pol, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int cdd_hirano(cddfloat * ret, cddfloat * init_x, CDDPoly pol, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- void deflation_cddpoly(CDDPoly ret, CDDPoly pol, cddfloat * root)

5.2.11 ファイル入出力関数 (fread_write_complex.c、USE_GMP が必要)

- void fread_cmpfarray(FILE * fp, CMPFArray array)
- void fread_cmpfarray_fname(const char * fname, CMPFArray array)
- void fwrite_cmpfarray(FILE * fp, CMPFArray array)
- void fwrite_cmpfarray_fname(const char * fname, CMPFArray array)
- void fread_mpfpolycoef(FILE * fp, MPFPoly pol, long int maxdeg)

第 6 章

疎行列および関連関数

本章では、本ライブラリが提供する疎行列型とそれに付随する関数について説明します。疎行列は圧縮行格納 (Compressed Row Storage, CRS) 形式で格納されます。各行列型について、本ライブラリは倍精度、double-double (DD)、triple-double (TD)、quadruple-double (QD)、および多倍長精度 (MPF、GMP による) の実数版および複素数版を提供します。

6.1 CRS 行列の構造体

6.1.1 DRSMatrix — 倍精度疎行列

`drsmatrix` 構造体は、実数の倍精度疎行列を CRS 形式で格納します。そのポインタ型は `DRSMatrix` です。主なメンバは次のとおりです。

- `double *element` — 非零要素の配列
- `long int row_dim, col_dim` — 行および列の次元
- `long int **nzero_index` — 非零要素の列インデックス
- `long int *nzero_col_dim` — 各行における非零要素の個数
- `long int *nzero_row_dim` — 各列における非零要素の個数
- `long int nzero_total_num` — 非零要素の総数
- `long int real_nzero_total_num` — SIMD アラインされた総数。次に等しい: $\sum_i \text{real_nzero_col_dim}[i]$
- `long int *real_nzero_col_dim` — SIMD アラインされた各行の非零要素数 (`_BNC_D_WIDTH` にパディング)
- `double zero_element` — ゼロを表す番兵値

`_BNC_D_WIDTH` (float 基の型では `_BNC_S_WIDTH`) は、ライブラリをビルドした SIMD バックエンドに依存する点に注意してください。AVX2 では 4/8、AVX-512 では 8/16、NEON および SVE2 では 2/4、serial ビルドでは 1/1 です。したがってパディングのコストは幅の広いビルドほど大きくなり、行の非零要素数がベクトル幅より少ない行列ではパディングされたレーンが支配的になり得ます。半帯幅 3 (1 行あたり非零要素 7 個) の場合、AVX-512 の SpMV は serial より明確に遅くなります。測定値とリンクすべきライブラリの選び方については 1.3 節を参照してください。

`DRSMatrix` に対する関数：

- `DRSMatrix init_drsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_drsmatrix(DRSMatrix mat)`
- `void set_nzero_row_dim(DRSMatrix mat)`
- `void print_drsmatrix(DRSMatrix mat)`
- `void set_dmatrix_drsmatrix(DMatrix ret, DRSMatrix spmat) ... 密 := 疎`
- `DRSMatrix init_set_drsmatrix_dmatrix(DMatrix org)`
- `double get_drsmatrix_ij(DRSMatrix mat, long int i, long int j)`
- `void set_drsmatrix_ij(DRSMatrix mat, long int i, long int j, double val)`
- `DRSMatrix init_set_drsmatrix(DRSMatrix org)`
- `void set0_drsmatrix(DRSMatrix spmat)`
- `int get_vars_drsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)`
- `int fread_urilinkdat_fname(DRSMatrix mat, const char * fname)`
- `int mul_drsmatrix_dvec(DVector ret, DRSMatrix mat, DVector vec)`
- `int mul_drsmatrixt_dvec(DVector ret, DRSMatrix mat, DVector vec) ... 転置`
- `void mul_drsmatrix_dvec_oz(DVector ret, DRSMatrix a, int max_num_div_a, DVector vb, int max_num_div_vb) ... 尾崎スキーム`
- `void mul_drsmatrixt_dvec_oz(DVector ret, DRSMatrix a, int max_num_div_a, DVector vb, int max_num_div_vb) ... 転置、尾崎スキーム`
- `double dpower_rsmatrix(DVector evec, DRSMatrix mat, double reps, double aeaps, long int maxtimes) ... べき乗法`
- `int smul_dvector(DVector ret, double c, DVector vec)`
- `long int absmax_index_dvector(double * absmax, DVector vec)`
- `void subst_drsmatrix(DRSMatrix ret, DRSMatrix a) ...  $ret := a$`
- `void add_drsmatrix(DRSMatrix ret, DRSMatrix a, DRSMatrix b) ...  $ret := a + b$`
- `void sub_drsmatrix(DRSMatrix ret, DRSMatrix a, DRSMatrix b) ...  $ret := a - b$`
- `void cmul_drsmatrix(DRSMatrix ret, double c, DRSMatrix a) ...  $ret := c \cdot a$`
- `double absmax_row_drsmatrix(long int * max_j, long int row, DRSMatrix mat)`
- `double absmax_col_drsmatrix(long int * max_i, long int col, DRSMatrix mat)`
- `double normf_drsmatrix(DRSMatrix mat) ... フロベニウスノルム`
- `void split_drsmatrix(DRSMatrix high, DRSMatrix low, DRSMatrix org) ... SplitMat_A`
- `int split_drsmatrix_drsmat(DRSMatrix[] ret, int num_div, DRSMatrix org) ... SplitMat_A (複数分割)`
- `int split_drsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, DRSMatrix org) ... SplitMat_B`
- `void iLU0_drsmatrix(DRSMatrix mat) ... 不完全 LU 分解 (iLU0)`
- `void solve_iLU0_drsmatrix(DVector ret, DRSMatrix ilu, DVector b) ... 次を解く  $(iLU)x = b$`
- `void solve_iLU0t_drsmatrix(DVector ret, DRSMatrix ilu, DVector b) ... 次を解く  $x^T(iLU) = b^T$`

6.1.2 CDRSMatrix — 複素倍精度疎行列

`cdrsmatrix` 構造体は、複素疎行列を実数の `DRSMatrix` 部の対として格納します。そのポインタ型は `CDRSMatrix` です。メンバは `DRSMatrix re` (実部)、`DRSMatrix im` (虚部) です。

- `CDRSMatrix init_cdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_cdrsmatrix(CDRSMatrix mat)`
- `void print_cdrsmatrix(CDRSMatrix mat)`
- `void set_cdmatrix_cdrsmatrix(CDMatrix ret, CDRSMatrix spmat)`
- `double normf_cdrsmatrix(CDRSMatrix mat)` ... フロベニウスノルム
- `CDRSMatrix init_set_cdrsmatrix_cdmatrix(CDMatrix org)`
- `CDRSMatrix init_set_cdrsmatrix(CDRSMatrix org)`
- `double _Complex get_cdrsmatrix_ij(CDRSMatrix mat, long int i, long int j)`
- `void set_cdrsmatrix_ij(CDRSMatrix mat, long int i, long int j, double _Complex val)`
- `int mul_cdrsmatrix_cdvec(CDVector ret, CDRSMatrix mat, CDVector vec)`
- `int mul_cdrsmatrixt_cdvec(CDVector ret, CDRSMatrix mat, CDVector vec)` ... 転置
- `int mul_cdrsmatrixs_cdvec(CDVector ret, CDRSMatrix mat, CDVector vec)` ... 共役転置
- `void iLU0_cdrsmatrix(CDRSMatrix mat)`
- `void solve_iLU0_cdrsmatrix(CDVector ret, CDRSMatrix ilu, CDVector b)`
- `void solve_iLU0t_cdrsmatrix(CDVector ret, CDRSMatrix ilu, CDVector b)`
- `void solve_iLU0s_cdrsmatrix(CDVector ret, CDRSMatrix ilu, CDVector b)`

6.1.3 DDRSMatrix および CDDRSMatrix — double-double 疎行列

`ddrsmatrix` 構造体は double-double 疎行列を格納します。ポインタ型は `DDRSMatrix` です。要素は `double *element[DDSIZE]` として格納されます。複素数版 `CDDRSMatrix` は `DDRSMatrix` 部の対を保持します。

- `DDRSMatrix init_ddrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_ddrsmatrix(DDRSMatrix mat)`
- `void set_nzero_row_dim_dd(DDRSMatrix mat)`
- `void print_ddrsmatrix(DDRSMatrix mat)`
- `void normf_ddrsmatrix(double ret[DDSIZE], DDRSMatrix mat)`
- `DDRSMatrix init_set_ddrsmatrix_ddmatrix(DDMatrix org)`
- `void get_ddrsmatrix_ij(double ret[DDSIZE], DDRSMatrix mat, long int i, long int j)`
- `void set_ddrsmatrix_ij(DDRSMatrix mat, long int i, long int j, double val[DDSIZE])`
- `DDRSMatrix init_set_ddrsmatrix(DDRSMatrix org)`
- `void set0_ddrsmatrix(DDRSMatrix spmat)`

- void subst_ddrsmatrix(DDRSMMatrix ret, DDRSMMatrix a)
- DRSMMatrix init_set_drsmatrix_ddrsmat(DDRSMMatrix org)
- int get_vars_ddrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_ddrsmatrix_ddvec(DDVector ret, DDRSMMatrix mat, DDVector vec)
- int mul_ddrsmatrixt_ddvec(DDVector ret, DDRSMMatrix mat, DDVector vec)
- int mul_drsmatrix_ddvec(DDVector ret, DRSMMatrix mat, DDVector vec)
- int mul_drsmatrixt_ddvec(DDVector ret, DRSMMatrix mat, DDVector vec)
- void ddpower_sp(double max_eig[DDSIZE], DDVector evec, DDRSMMatrix mat, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int absmax_index_ddvector(double ret[DDSIZE], DDVector vec)
- void subst_ddrsmatrix_drsmat(DDRSMMatrix c, DRSMMatrix a) $\cdots c := (DD)a$
- void subst_drsmatrix_ddrsmat(DRSMMatrix c, DDRSMMatrix a) $\cdots c := (D)a$
- void absmax_row_ddrsmatrix(double mu[DDSIZE], long int * max_j, long int row, DDRSMMatrix mat)
- void absmax_col_ddrsmatrix(double mu[DDSIZE], long int * max_i, long int col, DDRSMMatrix mat)
- void add_ddrsmatrix_drsmat(DDRSMMatrix c, DDRSMMatrix a, DRSMMatrix b)
- void sub_ddrsmatrix_drsmat(DDRSMMatrix c, DDRSMMatrix a, DRSMMatrix b)
- int split_ddrsmatrix_drsmat(DRSMMatrix[] ret, int num_div, DDRSMMatrix org)
- int split_ddrsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, DDRSMMatrix org)
- void mul_ddrsmatrix_ddvec_oz(DDVector ret, DDRSMMatrix a, int max_num_div_a, DDVector vb, int max_num_div_vb)
- void mul_ddrsmatrixt_ddvec_oz(DDVector ret, DDRSMMatrix a, int max_num_div_a, DDVector vb, int max_num_div_vb)
- void iLU0_ddrsmatrix(DDRSMMatrix mat)
- void solve_iLU0_ddrsmatrix(DDVector ret, DDRSMMatrix ilu, DDVector b)
- void solve_iLU0t_ddrsmatrix(DDVector ret, DDRSMMatrix ilu, DDVector b)
- void solve_iLU0_drsmatrix_ddvec(DDVector ret, DRSMMatrix ilu, DDVector b)
- void solve_iLU0t_drsmatrix_ddvec(DDVector ret, DRSMMatrix ilu, DDVector b)

複素 DD 疎行列 (CDDRSMMatrix) :

- CDDRSMMatrix init_cddrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_cddrsmatrix(CDDRSMMatrix mat)
- void print_cddrsmatrix(CDDRSMMatrix mat)
- void normf_cddrsmatrix(double ret[DDSIZE], CDDRSMMatrix mat)
- CDDRSMMatrix init_set_cddrsmatrix(CDDRSMMatrix org)
- int mul_cddrsmatrix_cddvec(CDDVector ret, CDDRSMMatrix mat, CDDVector vec)
- int mul_cddrsmatrixt_cddvec(CDDVector ret, CDDRSMMatrix mat, CDDVector vec)

- int mul_cddrsmatrixs_cddvec(CDDVector ret, CDDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrixt_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrixs_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- void iLU0_cddrsmatrix(CDDRSMatrix mat)
- void solve_iLU0_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0t_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0s_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)
- void solve_iLU0t_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)
- void solve_iLU0s_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)

6.1.4 TDRSMatrix および CTDRSMatrix — triple-double 疎行列

tdrsmatrix 構造体は triple-double 疎行列を格納します。ポインタ型は TDRSMatrix です。要素は double *element[TDSIZE] として格納されます。

- TDRSMatrix init_tdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_tdrsmatrix(TDRSMatrix mat)
- void set_nzero_row_dim_td(TDRSMatrix mat)
- void print_tdrsmatrix(TDRSMatrix mat)
- void get_tdrsmatrix_ij(double ret[TDSIZE], TDRSMatrix mat, long int i, long int j)
- void set_tdrsmatrix_ij(TDRSMatrix mat, long int i, long int j, double val[TDSIZE])
- TDRSMatrix init_set_tdrsmatrix(TDRSMatrix org)
- void set0_tdrsmatrix(TDRSMatrix spmat)
- void subst_tdrsmatrix(TDRSMatrix ret, TDRSMatrix a)
- DRSMMatrix init_set_drsmatrix_tdrsmat(TDRSMatrix org)
- TDRSMatrix init_set_tdrsmatrix_tdmatrix(TDMatrix org)
- int get_vars_tdrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_tdrsmatrix_tdvec(TDVector ret, TDRSMatrix mat, TDVector vec)
- int mul_tdrsmatrixt_tdvec(TDVector ret, TDRSMatrix mat, TDVector vec)
- int mul_drsmatrix_tdvec(TDVector ret, DRSMMatrix mat, TDVector vec)
- int mul_drsmatrixt_tdvec(TDVector ret, DRSMMatrix mat, TDVector vec)
- void tdpower_sp(double max_eig[TDSIZE], TDVector evec, TDRSMatrix mat, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int absmax_index_tdvector(double ret[TDSIZE], TDVector vec)
- void subst_tdrsmatrix_drsmat(TDRSMatrix c, DRSMMatrix a)
- void subst_drsmatrix_tdrsmat(DRSMMatrix c, TDRSMatrix a)

- void absmax_row_tdrsmatrix(double mu[TDSIZE], long int * max_j, long int row, TDRSMatrix mat)
- void absmax_col_tdrsmatrix(double mu[TDSIZE], long int * max_i, long int col, TDRSMatrix mat)
- void add_tdrsmatrix_drsmat(TDRSMatrix c, TDRSMatrix a, DRSMatrix b)
- void sub_tdrsmatrix_drsmat(TDRSMatrix c, TDRSMatrix a, DRSMatrix b)
- int split_tdrsmatrix_drsmat(DRSMatrix[] ret, int num_div, TDRSMatrix org)
- int split_tdrsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, TDRSMatrix org)
- void mul_tdrsmatrix_tdvec_oz(TDVector ret, TDRSMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void mul_tdrsmatrixt_tdvec_oz(TDVector ret, TDRSMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void iLU0_tdrsmatrix(TDRSMatrix mat)
- void solve_iLU0_tdrsmatrix(TDVector ret, TDRSMatrix ilu, TDVector b)
- void solve_iLU0t_tdrsmatrix(TDVector ret, TDRSMatrix ilu, TDVector b)
- void solve_iLU0_drsmatrix_tdvec(TDVector ret, DRSMatrix ilu, TDVector b)
- void solve_iLU0t_drsmatrix_tdvec(TDVector ret, DRSMatrix ilu, TDVector b)

複素 TD 疎行列 (CTDRSMatrix) :

- CTDRSMatrix init_ctdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_ctdrsmatrix(CTDRSMatrix mat)
- void print_ctdrsmatrix(CTDRSMatrix mat)
- CTDRSMatrix init_set_ctdrsmatrix(CTDRSMatrix org)
- int mul_ctdrsmatrix_ctdvec(CTDVector ret, CTDRSMatrix mat, CTDVector vec)
- int mul_ctdrsmatrixt_ctdvec(CTDVector ret, CTDRSMatrix mat, CTDVector vec)
- int mul_ctdrsmatrixs_ctdvec(CTDVector ret, CTDRSMatrix mat, CTDVector vec)
- int mul_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix mat, CTDVector vec)
- int mul_cdrsmatrixt_ctdvec(CTDVector ret, CDRSMatrix mat, CTDVector vec)
- int mul_cdrsmatrixs_ctdvec(CTDVector ret, CDRSMatrix mat, CTDVector vec)
- void iLU0_ctdrsmatrix(CTDRSMatrix mat)
- void solve_iLU0_ctdrsmatrix(CTDVector ret, CTDRSMatrix ilu, CTDVector b)
- void solve_iLU0t_ctdrsmatrix(CTDVector ret, CTDRSMatrix ilu, CTDVector b)
- void solve_iLU0s_ctdrsmatrix(CTDVector ret, CTDRSMatrix ilu, CTDVector b)
- void solve_iLU0_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix ilu, CTDVector b)
- void solve_iLU0t_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix ilu, CTDVector b)
- void solve_iLU0s_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix ilu, CTDVector b)

6.1.5 QDRSMatrix および CQDRSMatrix — quadruple-double 疎行列

qdrsmatrix 構造体は quadruple-double 疎行列を格納します。ポインタ型は QDRSMatrix です。要素は `double *element[QDSIZE]` として格納されます。

- `QDRSMatrix init_qdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_qdrsmatrix(QDRSMatrix mat)`
- `void set_nzero_row_dim_qd(QDRSMatrix mat)`
- `void print_qdrsmatrix(QDRSMatrix mat)`
- `QDRSMatrix init_set_qdrsmatrix_qdmatrx(QDMatrix org)`
- `void get_qdrsmatrix_ij(double ret[QDSIZE], QDRSMatrix mat, long int i, long int j)`
- `void set_qdrsmatrix_ij(QDRSMatrix mat, long int i, long int j, double val[QDSIZE])`
- `QDRSMatrix init_set_qdrsmatrix(QDRSMatrix org)`
- `void set0_qdrsmatrix(QDRSMatrix spmat)`
- `void subst_qdrsmatrix(QDRSMatrix ret, QDRSMatrix a)`
- `DRSMatrix init_set_drsmatrix_qdrsmat(QDRSMatrix org)`
- `int get_vars_qdrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)`
- `int mul_qdrsmatrix_qdvec(QDVector ret, QDRSMatrix mat, QDVector vec)`
- `int mul_qdrsmatrixt_qdvec(QDVector ret, QDRSMatrix mat, QDVector vec)`
- `int mul_drsmatrix_qdvec(QDVector ret, DRSMatrix mat, QDVector vec)`
- `int mul_drsmatrixt_qdvec(QDVector ret, DRSMatrix mat, QDVector vec)`
- `void qdpower_sp(double max_eig[QDSIZE], QDVector evec, QDRSMatrix mat, double reps[QDSIZE], double aeaps[QDSIZE], long int maxtimes)`
- `long int absmax_index_qdvector(double ret[QDSIZE], QDVector vec)`
- `void subst_qdrsmatrix_drsmat(QDRSMatrix c, DRSMatrix a)`
- `void subst_drsmatrix_qdrsmat(DRSMatrix c, QDRSMatrix a)`
- `void absmax_row_qdrsmatrix(double mu[QDSIZE], long int * max_j, long int row, QDRSMatrix mat)`
- `void absmax_col_qdrsmatrix(double mu[QDSIZE], long int * max_i, long int col, QDRSMatrix mat)`
- `void add_qdrsmatrix_drsmat(QDRSMatrix c, QDRSMatrix a, DRSMatrix b)`
- `void sub_qdrsmatrix_drsmat(QDRSMatrix c, QDRSMatrix a, DRSMatrix b)`
- `int split_qdrsmatrix_drsmat(DRSMatrix[] ret, int num_div, QDRSMatrix org)`
- `int split_qdrsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, QDRSMatrix org)`
- `void mul_qdrsmatrix_qdvec_oz(QDVector ret, QDRSMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)`
- `void mul_qdrsmatrixt_qdvec_oz(QDVector ret, QDRSMatrix a, int max_num_div_a, QDVector`

- vb, int max_num_div_vb)
- void iLU0_qdrsmatrix(QDRSMMatrix mat)
- void solve_iLU0_qdrsmatrix(QDVector ret, QDRSMMatrix ilu, QDVector b)
- void solve_iLU0t_qdrsmatrix(QDVector ret, QDRSMMatrix ilu, QDVector b)
- void solve_iLU0_drsmatrix_qdvec(QDVector ret, DRSMMatrix ilu, QDVector b)
- void solve_iLU0t_drsmatrix_qdvec(QDVector ret, DRSMMatrix ilu, QDVector b)

複素 QD 疎行列 (CQDRSMMatrix) :

- CQDRSMMatrix init_cqdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_cqdrsmatrix(CQDRSMMatrix mat)
- void print_cqdrsmatrix(CQDRSMMatrix mat)
- CQDRSMMatrix init_set_cqdrsmatrix(CQDRSMMatrix org)
- int mul_cqdrsmatrix_cqdvec(CQDVector ret, CQDRSMMatrix mat, CQDVector vec)
- int mul_cqdrsmatrixt_cqdvec(CQDVector ret, CQDRSMMatrix mat, CQDVector vec)
- int mul_cqdrsmatrixs_cqdvec(CQDVector ret, CQDRSMMatrix mat, CQDVector vec)
- int mul_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix mat, CQDVector vec)
- int mul_cdrsmatrixt_cqdvec(CQDVector ret, CDRSMMatrix mat, CQDVector vec)
- int mul_cdrsmatrixs_cqdvec(CQDVector ret, CDRSMMatrix mat, CQDVector vec)
- void iLU0_cqdrsmatrix(CQDRSMMatrix mat)
- void solve_iLU0_cqdrsmatrix(CQDVector ret, CQDRSMMatrix ilu, CQDVector b)
- void solve_iLU0t_cqdrsmatrix(CQDVector ret, CQDRSMMatrix ilu, CQDVector b)
- void solve_iLU0s_cqdrsmatrix(CQDVector ret, CQDRSMMatrix ilu, CQDVector b)
- void solve_iLU0_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix ilu, CQDVector b)
- void solve_iLU0t_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix ilu, CQDVector b)
- void solve_iLU0s_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix ilu, CQDVector b)

6.1.6 MPFRSMMatrix および CMPFRSMMatrix — 多倍長精度疎行列 (USE_GMP が必要)

mpfrsmatrix 構造体は GMP の多倍長精度疎行列を格納します。ポインタ型は MPFRSMMatrix です。要素は mpf_t *element として格納されます。複素数版 CMPFRSMMatrix は mpc_t *element を用います。

- MPFRSMMatrix init_mpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- MPFRSMMatrix init2_mpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num, unsigned long prec)
- void free_mpfrsmatrix(MPFRSMMatrix mat)
- void set_nzero_row_dim_mpf(MPFRSMMatrix mat)
- void print_mpfrsmatrix(MPFRSMMatrix mat)
- void get_mpfrsmatrix_ij(mpf_t ret, MPFRSMMatrix mat, long int i, long int j)

- void set_mpfrsmatrix_ij(MPFRSMatrix mat, long int i, long int j, mpf_t val)
- MPFRSMatrix init_set_mpfrsmatrix(MPFRSMatrix org)
- void set0_mpfrsmatrix(MPFRSMatrix spmat)
- void set_mpfmatrix_mpfrsmat(MPFMatrix ret, MPFRSMatrix spmat)
- MPFRSMatrix init_set_mpfrsmatrix_mpfmatrix(MPFMatrix org)
- int get_vars_mpfrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int fread_urilinkdat_fname_mpf(MPFRSMatrix mat, const char * fname)
- long int absmax_index_mpfvector(mpf_t absmax, MPFVector vec)
- int mul_mpfrsmatrix_mpfvec(MPFVector ret, MPFRSMatrix mat, MPFVector vec)
- int mul_mpfrsmatrixt_mpfvec(MPFVector ret, MPFRSMatrix mat, MPFVector vec)
- int mul_drsmatrix_mpfvec(MPFVector ret, DRSMatrix mat, MPFVector vec)
- int mul_drsmatrixt_mpfvec(MPFVector ret, DRSMatrix mat, MPFVector vec)
- void mpfpower_rsmatrix(mpf_t max_eig, MPFVector evec, MPFRSMatrix mat, mpf_t reps, mpf_t aeaps, long int maxtimes)
- int smul_mpfvector(MPFVector ret, mpf_t c, MPFVector vec)
- void iLU0_mpfrsmatrix(MPFRSMatrix mat)
- void solve_iLU0_mpfrsmatrix(MPFVector ret, MPFRSMatrix ilu, MPFVector b)
- void solve_iLU0t_mpfrsmatrix(MPFVector ret, MPFRSMatrix ilu, MPFVector b)
- void solve_iLU0_drsmatrix_mpfvec(MPFVector ret, DRSMatrix ilu, MPFVector b)
- void solve_iLU0t_drsmatrix_mpfvec(MPFVector ret, DRSMatrix ilu, MPFVector b)

複素 MPF 疎行列 (CMPFRSMatrix) :

- CMPFRSMatrix init_cmpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- CMPFRSMatrix init2_cmpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num, unsigned long prec)
- void free_cmpfrsmatrix(CMPFRSMatrix mat)
- void set_nzero_row_dim_cmpf(CMPFRSMatrix mat)
- void print_cmpfrsmatrix(CMPFRSMatrix mat)
- void normf_cmpfrsmatrix(mpf_t ret, CMPFRSMatrix mat)
- mpc_ptr get_cmpfrsmatrix_ij(CMPFRSMatrix mat, long int i, long int j)
- void set_cmpfrsmatrix_ij(CMPFRSMatrix mat, long int i, long int j, mpc_t val)
- CMPFRSMatrix init_set_cmpfrsmatrix(CMPFRSMatrix org)
- void set_cmpfmatrix_cmpfrsmat(CMPFMatrix ret, CMPFRSMatrix spmat)
- CMPFRSMatrix init_set_cmpfrsmatrix_cmpfmatrix(CMPFMatrix org)
- int get_vars_cmpfrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_cmpfrsmatrix_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mul_cmpfrsmatrixt_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)

- `int mul_cmpfrsmatrixs_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)`
- `int mul_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)`
- `int mul_cdrsmatrixt_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)`
- `int mul_cdrsmatrixs_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)`
- `void iLU0_cmpfrsmatrix(CMPFRSMatrix mat)`
- `void solve_iLU0_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)`
- `void solve_iLU0t_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)`
- `void solve_iLU0s_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)`
- `void solve_iLU0_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)`
- `void solve_iLU0t_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)`
- `void solve_iLU0s_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)`

MPFRSMatrix から他の型への変換関数 (USE_GMP が必要) :

- `DRSMatrix init_set_drsmatrix_mpfrsmatrix(MPFRSMatrix org)`
- `CDRSMatrix init_set_cdrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)`
- `DDRSMatrix init_set_ddrsmatrix_mpfrsmatrix(MPFRSMatrix org)`
- `CDDRSMatrix init_set_cddrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)`
- `TDRSMatrix init_set_tdrsmatrix_mpfrsmatrix(MPFRSMatrix org)`
- `CTDRSMatrix init_set_ctdrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)`
- `QDRSMatrix init_set_qdrsmatrix_mpfrsmatrix(MPFRSMatrix org)`
- `CQDRSMatrix init_set_cqdrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)`

6.2 クリロフ部分空間反復解法

6.2.1 共役勾配法 (FCG, DCG, DDCG, TDCG, QDCG, MPFCG)

- `long int FCG(FVector ans, FMatrix a, FVector b, float reps, float aeps, long int maxtimes)`
- `long int DCG_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)`
- `long int bnc_DCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)`
- `long int CDC0CG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)`
- `long int CDC0CG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)`

double-double CG :

- `long int DDCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)`
- `long int DDCG_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double`

- aeps[DDSIZE], long int maxtimes)
- long int DDCG(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCOCG_sp(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCOCG_sp_d(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCOCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

triple-double CG :

- long int TDCG_sp(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCG_sp_d(TDVector ans, DRMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCOCG_sp(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCOCG_sp_d(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCOCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

quadruple-double CG :

- long int QDCG_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCG_sp_d(QDVector ans, DRMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCOCG_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCOCG_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCOCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

多倍長精度 CG (USE_GMP が必要) :

- long int MPFCG_sp(MPFVector ans, MPFRMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long

- int maxtimes)
- long int MPFCG_sp_d(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCOCG_sp(CMPFVector ans, CMPFRSMMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCOCG_sp_d(CMPFVector ans, CDRSMMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCOCG(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)

6.2.2 BiCG, CGS, BiCGSTAB, GPBiCG 法（倍精度）

各ソルバは、疎（_sp）、前処理付き疎（_sp_iLU0）、および密の各版で提供されます。

- long int DBiCG_sp(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DCGS_sp(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCGSTAB_sp(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DGPBiCG_sp(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCG_sp_iLU0(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DCGS_sp_iLU0(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DBiCGSTAB_sp_iLU0(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DGPBiCG_sp_iLU0(DVector ans, DRSMMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DBiCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DCGS(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCGSTAB(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DGPBiCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCG_sp(CDVector ans, CDRSMMatrix a, CDVector b, double reps, double aeps, long int maxtimes)

- long int CDCGS_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCGSTAB_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDGPBiCG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCG_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDCGS_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDBiCGSTAB_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDGPBiCG_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDBiCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDCGS(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCGSTAB(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDGPBiCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)

6.2.3 BiCG, CGS, BiCGSTAB, GPBiCG 法 (double-double)

- long int DDBiCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCG_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDCGS_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDBiCGSTAB_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)

- long int DDGPBiCG_sp_iLU0(DDVector ans, DDRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMMatrix ilu, double * norm2_res_history)
- long int DDBiCG_sp_d(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS_sp_d(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB_sp_d(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG_sp_d(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCG_sp_d_iLU0(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DDCGS_sp_d_iLU0(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DDBiCGSTAB_sp_d_iLU0(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DDGPBiCG_sp_d_iLU0(DDVector ans, DRSMMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int DDBiCG(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCG_sp(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS_sp(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCGSTAB_sp(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG_sp(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCG_sp_iLU0(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMMatrix ilu, double * norm2_res_history)
- long int CDDCGS_sp_iLU0(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMMatrix ilu, double * norm2_res_history)
- long int CDDBiCGSTAB_sp_iLU0(CDDVector ans, CDDRSMMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMMatrix ilu, double *

- norm2_res_history)
- long int CDDGPBiCG_sp_iLU0(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDMatrix ilu, double * norm2_res_history)
- long int CDDBiCG_sp_d(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS_sp_d(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCGSTAB_sp_d(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG_sp_d(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCG_sp_d_iLU0(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDMatrix ilu, double * norm2_res_history)
- long int CDDCGS_sp_d_iLU0(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDMatrix ilu, double * norm2_res_history)
- long int CDDBiCGSTAB_sp_d_iLU0(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDMatrix ilu, double * norm2_res_history)
- long int CDDGPBiCG_sp_d_iLU0(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDMatrix ilu, double * norm2_res_history)
- long int CDDBiCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCGSTAB(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

6.2.4 BiCG, CGS, BiCGSTAB, GPBiCG 法 (triple-double)

- long int TDBiCG_sp(TDVector ans, TDDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS_sp(TDVector ans, TDDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB_sp(TDVector ans, TDDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG_sp(TDVector ans, TDDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

- long int TDBiCG_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDCGS_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCGSTAB_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDGPBiCG_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCG_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCG_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDCGS_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCGSTAB_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDGPBiCG_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS_sp(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB_sp(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG_sp(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

- long int CTDBiCG_sp_iLU0(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMMatrix ilu, double * norm2_res_history)
- long int CTDCGS_sp_iLU0(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCGSTAB_sp_iLU0(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMMatrix ilu, double * norm2_res_history)
- long int CTDGPBiCG_sp_iLU0(CTDVector ans, CTDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCG_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDCGS_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCGSTAB_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDGPBiCG_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

6.2.5 BiCG, CGS, BiCGSTAB, GPBiCG 法 (quadruple-double)

- long int QDBiCG_sp(QDVector ans, QDRSMMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

- long int QDCGS_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCG_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDCGS_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDBiCGSTAB_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDGPBiCG_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDBiCG_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCGS_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCG_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDCGS_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDBiCGSTAB_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDGPBiCG_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDBiCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCGS(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCG_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

- long int CQDCGS_sp(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB_sp(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG_sp(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCG_sp_iLU0(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMMatrix ilu, double * norm2_res_history)
- long int CQDCGS_sp_iLU0(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMMatrix ilu, double * norm2_res_history)
- long int CQDBiCGSTAB_sp_iLU0(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMMatrix ilu, double * norm2_res_history)
- long int CQDGPBiCG_sp_iLU0(CQDVector ans, CQDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMMatrix ilu, double * norm2_res_history)
- long int CQDBiCG_sp_d(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCGS_sp_d(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB_sp_d(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG_sp_d(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCG_sp_d_iLU0(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CQDCGS_sp_d_iLU0(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CQDBiCGSTAB_sp_d_iLU0(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CQDGPBiCG_sp_d_iLU0(CQDVector ans, CDRSMMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CQDBiCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCGS(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

6.2.6 BiCG, CGS, BiCGSTAB, GPBiCG 法 (多倍長精度。USE_GMP が必要)

- long int MPFBiCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCG_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFCGS_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCGSTAB_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCG_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCG_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFCGS_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCGSTAB_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long

- int maxtimes)
- long int MPFGPBiCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCG_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCGS_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCGSTAB_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCG_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFCGS_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCGSTAB_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCG_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCGS_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCGSTAB_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCG_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFCGS_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCGSTAB_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCG(CMPFVector ans, MPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCGS(CMPFVector ans, MPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCGSTAB(CMPFVector ans, MPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps,

- long int maxtimes)
- long int CMPFGPBiCG(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)

第 7 章

単精度系および複素数系の精度ファミリ

第 2 章から第 6 章では、`double` に基づく実数型 `D`, `DD`, `TD`, `QD` および任意精度型 `MPF` について説明しました。BNCmatmul は、これらとまったく同じ演算群を共有する固定精度型のグループをさらに二つ提供します。すなわち `float` に基づいて構築された**単精度系**ファミリと**複素数系**ファミリです。本章では、これらのデータ型をまとめます。前章までのすべてのベクトル、行列、乗算、LU 分解、疎行列ルーチンは、関数名の型接頭辞を置き換えることにより、これらの型に対しても利用できます。

7.1 単精度系マルチコンポーネント型 (DS, TS, QS)

単精度系ファミリは、数値を二つ、三つ、または四つの `float` 値の重なりのない和として格納します (`float` に適用した「マルチコンポーネント」方式です)。これは `rds.h` および線形演算ヘッダ `dslinear.h`, `tslinear.h`, `qslinear.h` で宣言されています。

type	components	precision (bits, $\approx$)	decimal digits ($\approx$)
DS (double-single)	<code>float</code> ×2	48	14
TS (triple-single)	<code>float</code> ×3	72	21
QS (quadruple-single)	<code>float</code> ×4	96	28

表 7.1 単精度系マルチコンポーネント型。DSSIZE= 2, TSSIZE= 3, QSSIZE= 4.

要素レベルの算術には、マクロ群 `rds_*`, `rts_*`, `rqs_*` (第 2.9 節の `rdd_*` に相当します) を用います。例えば `rds_add`, `rts_mul`, `rqs_div`, `rqs_sqrt` などです。ベクトルおよび行列オブジェクトとその演算は、第 3 章とまったく同じ構成です。

- `DSVector init_dsvector(long int dim) ...` および `init_tsvector`, `init_qsvector`, `init_dsmatrix`, ...
- `void add_dsvector(DSVector c, DSVector a, DSVector b) ...` ベクトルの $\pm$. `cmul_dsvector`, `ip_dsvector`, `norm2_dsvector` など。
- `void mul_dsmatrix(DSMatrix C, DSMatrix A, DSMatrix B) ...` $C := AB$. `mul_dsmatrix_block`, `mul_dsmatrix_strassen_even`, `mul_dsmatrix_dsvect`, `mul_dsmatrixt_dsvect` もあります。
- `int DSLUdecomp(DSMatrix A) ...` LU 分解。DSLUdecompP (部分ピボット選択), `SolveDSL`, `SolveDSLSP` および TS/QS 版もあります。

要素のアクセサは値構造体 dsfloat / tsfloat / qsfloat (DSSIZE 個などの float 配列) です. get_dsmatrix_ij_dsfloat で読み出し, その .val ポインタを使用する前に, 返された値をローカル変数に格納してください.

7.2 複素数型 (CD およびマルチコンポーネント複素数)

複素数は, 実部と虚部を別々に格納します. ネイティブな複素 double 型 CD は C99 の double _Complex 配列を保持します. マルチコンポーネント複素数型は, 実部と虚部に対応する実数型の二つのベクトル／行列として保持します.

type	real/imag element	header
CD	double _Complex	cdlinear.h, clinear.h
CDD	DD (double-double)	cddlinear.h
CTD	TD (triple-double)	ctdlinear.h
CQD	QD (quadruple-double)	cqdlinear.h
CDS	DS (double-single)	cdslinear.h
CTS	TS (triple-single)	ctslinear.h
CQS	QS (quadruple-single)	cqslinear.h

表 7.2 複素数型. マルチコンポーネント複素数の要素値は cXXfloat = {val_re[.], val_im[.]} です (rcdd.h, rcds.h を参照).

7.2.1 ネイティブ複素 double (CD)

- CDVector init_cdvector(long int dim) ... および init_cdmatrix.
- void set_cdvector_i(CDVector vec, long int index, double _Complex val) ... および get_cdvector_i, set_cdmatrix_ij, get_cdmatrix_ij.
- void mul_cdmatrix(CDMatrix C, CDMatrix A, CDMatrix B) ... $C := AB$. mul_cdmatrix_cdvec, mul_cdmatrixt_cdvec もあります.
- int CDLUdecomp(CDMatrix A) ... LU 分解. CDLUdecompP (部分ピボット選択) および CDLUdecompC (完全ピボット選択) もあります.

7.2.2 マルチコンポーネント複素数 (CDD, CTD, CQD, CDS, CTS, CQS)

ベクトル／行列構造体は, 基礎となる実数型の二つの部分を保持します.

```
typedef struct { XXVector re; XXVector im; } cXXvector; /* e.g. DDVector re, im */
typedef struct { XXMatrix re; XXMatrix im; } cXXmatrix;
```

- CDDVector init_cddvector(int dim) ... および init_cddmatrix, さらに CTD/CQD/CDS/CTS/CQS の同等品があります.
- void set_cddvector_i(CDDVector vec, long int index, cddfloat* val) ... cXXfloat 値を介した

要素の設定／取得 (`get_cddvector_i_cddfloat`, `get_cddmatrix_ij_cddfloat`, `set_cddmatrix_ij`).

- `void mul_cddmatrix_4m(CDDMatrix C, CDDMatrix A, CDDMatrix B)` ... 4 回の実数乗算 ($\text{Re} = A_r B_r - A_i B_i$, $\text{Im} = A_r B_i + A_i B_r$) を用いた複素行列乗算です. 3 回乗算 (Karatsuba) 版は `mul_cddmatrix_3m` です.
- `void mul_cddmatrix_cddvec_4m(CDDVector v, CDDMatrix A, CDDVector vb)` ... 行列・ベクトル積です. `mul_cddmatrixt_cddvec` ($A^T \mathbf{vb}$) もあります.
- `double normf_cddmatrix(CDDMatrix A)` ... フロベニウスノルムです. ブロック／Strassen 乗算は `mul_cddmatrix_block_4m`, `mul_cddmatrix_strassen_4m` として利用できます.

CTD, CQD, CDS, CTS, CQS のそれぞれは, 独自の接頭辞を持つ同一の関数群を提供します (例えば `mul_cqsmatrix_4m`, `init_ctsmatrix`). CD, CDD, CTD, CQD に対応する疎行列版 (CRS 格納形式と SpMV) は, 実数系疎行列型とともに第 6 章で説明します (`sparse_cd.c`, `sparse_cdd.c`, `sparse_ctd.c`, `sparse_cqd.c`).

7.3 利用可能なファミリの一覧

group	types	where
real, double-based	D, DD, TD, QD, MPF	Chap. 3–6
real, float-based	F, DS, TS, QS	this chapter
complex	CD, CDD, CTD, CQD, CDS, CTS, CQS, CMPF	this chapter, Chap. 6
GPU (CUDA)	g/gc/cg variants of all of the above	Chap. 8

表 7.3 BNCmatmul が提供する精度ファミリ.

第 8 章

CUDA による GPU 計算

本章では、基本的な線形計算、行列乗算、LU 分解、および疎行列ベクトル乗算を NVIDIA GPU 上で実行する BNCmatmul の CUDA 実装について説明します。本実装は **gdtq-0.0.2**（デバイス用多成分浮動小数点ヘッダ層）の上に構築されており、任意精度については **mpc_cuda-0.0.1**（MPFR/MPC のデバイス移植版）を用います。基準とする対象は NVIDIA GB10（sm_121, CUDA 13）です。

すべての GPU 関数は静的ライブラリ `libbncmatmul-0.24_cuda.a` にまとめられています。単一のスクリプト `bench/build_cuda_full.sh` は、すべての GPU モジュールをこのライブラリにコンパイルし、検証スイート全体（各精度の行列乗算、行列ベクトル積、LU、SpMV、および CUDA MPFR/MPC 行列乗算）を実行します。

8.1 命名規則とデータ型

すべての GPU 型および関数には先頭に `g`（“GPU”）が付きます。要素型は CPU 側と同じ多成分方式に従い、`double` に基づく `d` 系列と、`float` に基づく `s` 系列があります。

GPU prefix	element type	bits ($\approx$)	CPU counterpart
gf	float (float)	24	FVector / FMatrix
gd	double (double)	53	DVector / DMatrix
gds	double-single (float2)	48	DSVector / DSMatrix
gts	triple-single (float3)	72	TSVector / TSMatrix
gqs	quadruple-single (float4)	96	QSVector / QSMatrix
gdd	double-double (double2)	106	DDVector / DDMatrix
gtd	triple-double (double3)	159	TDVector / TDMatrix
gqd	quadruple-double (double4)	212	QDVector / QDMatrix

表 8.1 実数値の GPU 要素型。 $g\tau$ は $\tau \in \{f, d, ds, ts, qs, dd, td, qd\}$ とする。

複素数には 2 つの系列が用意されています。ネイティブ複素型 `gcd`（複素 `double`）および `gcf`（複素 `float`）は、実部と虚部を 2 つの別々のスカラ配列として格納します。多成分複素型 `cgdd`, `cgtd`, `cgqd`, `cgds`, `cgts`, `cgqs` は、実部と虚部を対応する $g\tau$ 要素の 2 つの別々の配列として格納します。

各系列はベクトルオブジェクトと行列オブジェクトを定義します。実数型 $g\tau$ の場合は次のとおりです。

```
typedef struct { long int dim;                gXX_real *element; } gXXvector;
typedef struct { long int row_dim, col_dim;   gXX_real *element; } gXXmatrix;
```

複素型の場合は、単一の `element` ポインタが `re`, `im` の対に置き換えられます。構造体へのポインタは `GXXVector` / `GXXMatrix` (大文字) と名付けられます。例えば `GDDVector`, `GCDMatrix`, `CGTSVector` などです。

行列は行優先で、ストライドは `col_dim` に等しい形で格納されます (GPU 側は CPU 側の SIMD 列パディングを**使用しません**)。2 つのブロック／グリッド引数 `int num_blocks_per_grid` および `int num_threads_per_block` が、カーネルを起動するすべての関数に渡されます。典型的な値はそれぞれ 32～128 です。

8.2 デバイスメモリとホスト・デバイス間転送

以下のリストにおいて、`g $\tau$` は実数 GPU 型のいずれかを表し、`XX` はその CPU 側の対応物を表します (例: `gdd $\leftrightarrow$ DD`)。

- `GXXVector init_gXXvector(long int dim)` ... ホストメモリにベクトルを確保します。
- `GXXVector init_gXXvector_dev(long int dim)` ... デバイス (GPU) メモリにベクトルを確保します。
- `void free_gXXvector(GXXVector vec)`
- `void free_gXXvector_dev(GXXVector vec)`
- `GXXMatrix init_gXXmatrix(long int row_dim, long int col_dim)`
- `GXXMatrix init_gXXmatrix_dev(long int row_dim, long int col_dim)`
- `void free_gXXmatrix_dev(GXXMatrix mat)`
- `void subst_gXXvector_dev_XXvec(GXXVector dev, XXVector src)` ... CPU のベクトルを GPU へコピーします (ホスト → デバイス)。
- `void subst_XXvector_gXXvec_dev(XXVector dst, GXXVector dev)` ... GPU のベクトルを CPU へ戻します (デバイス → ホスト)。
- `void subst_gXXmatrix_dev_XXmat(GXXMatrix dev, XXMatrix src)`
- `void subst_XXmatrix_gXXmat_dev(XXMatrix dst, GXXMatrix dev)`
- `void print_gXXvector_dev(GXXVector dev)`
- `void print_gXXmatrix_dev(GXXMatrix dev)`

ネイティブ複素型 `gcd/gcf` の CPU 側対応物は、複素 `double` の `CDVector`/`CDMatrix` です。転送関数は `subst_gcdvector_dev_cdvec` および `subst_cdvector_gcdvec_dev` (および行列版) です。多成分複素型 `cg $\tau$` の CPU 側対応物は、複素多成分の `CXXVector`/`CXXMatrix` (例: `CDDMatrix`) であり、`subst_cgddvector_dev_cddvec` などを用います。

8.3 ベクトルと行列の演算

以下のルーチンはすべて GPU 上で動作し、2つの起動引数 `int nbg`（ブロック数）と `int ntb`（スレッド数）を取ります。簡潔さのため、これらは引数リストから省略しています。

- `void add_gXXvector_dev(GXXVector c, GXXVector a, GXXVector b) ...  $c := a + b$ .`
- `void sub_gXXvector_dev(GXXVector c, GXXVector a, GXXVector b) ...  $c := a - b$ .`
- `void cmul_gXXvector_dev(GXXVector c, gXX_real val, GXXVector a) ...  $c := val \cdot a$ .`
- `void subst_gXXvector_dev(GXXVector c, GXXVector a) ...  $c := a$ .`
- `void set0_gXXvector_dev(GXXVector c) ...  $c := 0$ .`
- `void ip_gXXvector_dev(gXX_real* ret_dev, GXXVector a, GXXVector b) ...` 内積 (a, b) 。結果はデバイススカラ `ret_dev` に書き込まれます。
- `void norm2_gXXvector_dev(gXX_real* ret_dev, GXXVector a) ...  $\|a\|_2$ .`
- `void norm1_gXXvector_dev(gXX_real* ret_dev, GXXVector a) ...  $\|a\|_1$ .`
- `void normi_gXXvector_dev(gXX_real* ret_dev, GXXVector a) ...  $\|a\|_\infty$ .`
- `void add_gXXmatrix_dev(GXXMatrix C, GXXMatrix A, GXXMatrix B) ...  $C := A + B$ .`
- `void sub_gXXmatrix_dev(GXXMatrix C, GXXMatrix A, GXXMatrix B) ...  $C := A - B$ .`
- `void cmul_gXXmatrix_dev(GXXMatrix C, gXX_real sc, GXXMatrix A) ...  $C := sc \cdot A$ .`
- `void transpose_gXXmatrix_dev(GXXMatrix C, GXXMatrix A) ...  $C := A^T$ .`
- `void subst_gXXmatrix_dev(GXXMatrix C, GXXMatrix A) ...  $C := A$ .`
- `void set0_gXXmatrix_dev(GXXMatrix C) ...  $C := O$ .`
- `void setI_gXXmatrix_dev(GXXMatrix C) ...  $C := I$ .`
- `void normf_gXXmatrix_dev(gXX_real* ret_dev, GXXMatrix A) ...` フロベニウスノルム $\|A\|_F$ 。

複素系列はさらに `conjtrans_gcdmatrix_dev` / `conjtrans_cgXXmatrix_dev` ($C := A^H$, 共役転置) を提供します。また複素スカラ乗算は実部と虚部を2つの引数として取ります。例：`cmul_gcdvector_dev` `GCDVector c, double val_re, double val_im, GCDVector a`。

8.4 行列乗算と行列ベクトル積

- `void mul_gXXmatrix_dev(GXXMatrix C, GXXMatrix A, GXXMatrix B) ...  $C := AB$` （出力行ごとに1スレッド，グリッドストライド）。
- `void mul_gXXmatrix_gXXvec(GXXVector v, GXXMatrix A, GXXVector vb) ...  $v := Avb$ .`
- `void mul_gXXmatrixt_gXXvec(GXXVector v, GXXMatrix A, GXXVector vb) ...  $v := A^T vb$ .`

複素系列にも同じ3つのルーチンが存在します。例：`mul_gcdmatrix_dev`, `mul_cgddmatrix_dev`, `mul_cgddmatrix_cgddvec`, `mul_cgddmatrixt_cgddvec`。複素乗算は4回の実数乗算による公式 $\text{Re}(AB) = A_r B_r - A_i B_i$, $\text{Im}(AB) = A_r B_i + A_i B_r$ を用います。

多成分型に対する Strassen 行列乗算が `src/matmul_strassen_general_g{dd,td,qd,ds,ts,qs}.cu` に提供されています。

GB10 上で対応する CPU ルーチンと比較して検証した $C := AB$ の精度は次のとおりです。gf および 4 倍単精度の cgqs はビット単位で一致 (0)。gd は $\sim 10^{-16}$, gcd は $\sim 10^{-19}$, gdd は $\sim 10^{-31}$, gds は $\sim 10^{-14}$, gts は $\sim 10^{-22}$ です。

8.5 GPU 上の LU 分解と線形ソルバ

部分ピボット選択を伴う右向き (right-looking) LU 分解と、前進／後退代入ソルバが、すべての実数型およびネイティブ複素型に対して提供されています。分解と求解は 1 つのドライバにまとめられています。

- `int gXX_lu_solve_dev(GXXMatrix A, GXXVector b, GXXVector x, long int ch[], int blocks, int threads) ...` A をその LU 因子で上書きし, $Ax = b$ を解いて x に格納します (すべてデバイスオブジェクト)。各ステップで選択されたピボット行は `ch[]` (ホスト, 長さ n , NULL でもよい) に記録されます。成功時に 0 を返します。

XX は d, f, dd, td, qd, ds, ts, qs (実数。例: `gqs_lu_solve_dev`), または cd, cf (ネイティブ複素。`gcd_lu_solve_dev` / `gcf_lu_solve_dev`) のいずれかです。10 個のドライバはすべて `extern "C"` として宣言されています。既知の解に対して検証した最大相対誤差 (次元 100) は次のとおりです。gd は 1.2×10^{-12} , gf は 9.5×10^{-7} , gcd は 2.8×10^{-15} , gdd/gtd/gds は 0, gts は 7.7×10^{-22} , gqs は 2.8×10^{-29} です。

8.6 疎行列ベクトル乗算 (SpMV)

GPU の疎行列は, CPU の “DRS” 形式とは独立に, 標準的な CSR (Compressed Sparse Row) 形式で格納されます。構造体は次元, 非ゼロ要素数, および 3 つのデバイス配列を保持します。

```
typedef struct {
    long int row_dim, col_dim, nnz;
    long int *row_ptr;    /* length row_dim + 1 */
    long int *col_idx;    /* length nnz */
    gXX_real *val;        /* length nnz */
} gXXspmatrix;           /* GXXSPMatrix = gXXspmatrix * */
```

- `GXXSPMatrix init_gXXspmatrix_dev(long int row_dim, long int col_dim, long int nnz, const long int* row_ptr, const long int* col_idx, const gXX_real* val) ...` ホストの CSR 配列からデバイスの CSR 行列を構築します。
- `void free_gXXspmatrix_dev(GXXSPMatrix mat)`
- `void mul_gXXspmatrix_gXXvec(GXXVector ret, GXXSPMatrix A, GXXVector x, int nbg, int ntb) ...` $ret := Ax$ (行ごとに 1 スレッド)。

ネイティブの gd/gf SpMV は, 上記のとおり GDVector/GFVector の密ベクトルを使用します。多成分実数型 (gdd...gqs) は, x と ret に対して生のデバイス `gXX_real *` 配列を取り, `mul_gXXspmatrixgXX_real*` y , `GXXSPMatrix A`, `const gXX_real* x`, ... を介して扱います。ネイティブ複素 `gcd/gcf` および多成分複素 `cgr` の SpMV は, 実部／虚部の値配列を分離して保持します。例: `mul_gcdspmatrixdouble* yre,`

`double* yim, GCDSPMatrix A, const double* xre, const double* xim, ...`。転置積 $A^T \mathbf{x}$ は、ネイティブの `mul_gdspmatrixt_gdvec`（アトミックスキュッタ）によって、または多成分型の場合は A^T の CSR をホスト上で構築して $A\mathbf{x}$ カーネルを再利用することによって得られます。

8.7 任意精度行列乗算（CUDA MPFR / MPC）

デバイス移植版 `mpc_cuda` を用いると、行列乗算または行列ベクトル積の内積を、コンパイル時に指定した任意精度（既定 1024 ビット）で GPU 上に累算できます。行列とベクトルは、単純な行優先の `double` 配列として渡されます（複素版は実部／虚部を分離した `double` 配列を使用します）。

- `int cuda_mul_mpfmatrix(double* C, const double* A, const double* B, int n, int lblocks, int lthreads) ...  $C := AB$  ( $n \times n$ )。各要素を PREC ビットの MPFR で累算します。`
- `int cuda_mul_mpfmatrix_vec(double* y, const double* A, const double* x, int n, int lblocks, int lthreads) ...  $y := Ax$ 。`
- `int cuda_mul_cmpfmatrix(double* Cr, double* Ci, const double* Ar, const double* Ai, const double* Br, const double* Bi, int n, int lblocks, int lthreads) ... 複素  $C := AB$ 。MPC で累算します。`
- `int cuda_mul_cmpfmatrix_vec(double* yr, double* yi, const double* Ar, const double* Ai, const double* xr, const double* xi, int n, int lblocks, int lthreads)`

これらのルーチンは静的ライブラリ `/usr/local/lib/libmpc_cuda.a` をリンクし、デバイスのスタック／ヒープ上限を拡大する必要があります（ドライバは内部で `cudaLimitStackSize` とデバイスヒープを設定します）。GB10 上で検証した結果、MPFR ($n = 96$) と MPC ($n = 80$) の行列乗算および行列ベクトル積は、いずれも 1024 ビットで CPU の結果と 0 まで一致しました。

8.8 GPU スイートのビルドと実行

```
sh bench/build_cuda_full.sh
```

は、上記でリストしたすべてのモジュールを `libbncmatmul-0.24_cuda.a` にコンパイルし、完全な検証を実行します。デバイス算術演算子は `gdtq` の集約モジュールによって供給されます。すなわち `d` 系列には `gqd.cu` (`double2/3/4`)、`s` 系列には `gqs.cu` (`float2/3/4`) です。`gdd/gds` オブジェクトはすでに演算子をインラインで保持していますが、`gtd/gqd/gts/gqs` は対応する集約オブジェクトに対してデバイスリンクされます。任意精度モジュールは、`/usr/local/include/mpc_cuda` 以下の `mpc_cuda` ヘッダと、`/usr/local/share/mpc_cuda/mpfr_cuda_defs.txt` 内の定義を必要とします。

第 9 章

Python からの BNCmatmul の利用

本章では、2.8 節の関数エクスポート API (`src/rdtq_func.c` / `include/rdtq_func.h`) と 2.7 節の要素毎の一括関数を使って、BNCmatmul の DD/TD/QD (および DS/TS/QS) 演算・FMA・初等関数を Python 標準の `ctypes` モジュールから呼び出す方法を示します。本章のコード断片はすべてライブラリに同梱の `python/example_rdtq.py` から取ったもので、そのまま実行できます。対応する回帰テストは `python/test_rdtq.py` です。

9.1 共有ライブラリのビルド

エクスポートされるシンボルは、`rdtq_func.c` (スカラー関数 99 個) と `elem_vector.c` (一括関数 12 個) からビルドされる小さな共有ライブラリ `python/librdtq.so` に収められます。レガシー `make` ターゲットでビルドします:

```
$ cd bncmatmul-0.24/src
$ make -f Makefile.legacy librdtq          # generic build
$ make -f Makefile.legacy librdtq_neon    # Arm NEON build
```

あるいはコンパイラを直接使う場合:

```
$ gcc -O3 -ffp-contract=off -fPIC -shared -Iinclude \
    src/rdtq_func.c src/elem_vector.c -o python/librdtq.so -lm
```

(BNCmatmul の他の部分と同様、`-ffp-contract=off` は必須です。) フルライブラリの共有オブジェクト `libbncmatmul-0.24*.so` にも同じシンボルに加えて線形計算 API 全体が含まれます。`librdtq.so` はスクリプト用途向けの最小自己完結サブセットです。

9.2 ライブラリのロードとフル精度表示

DD/TD/QD 値は 2 / 3 / 4 個の `double` の未評価和なので、`ctypes` では小さな `c_double` 配列で表すのが自然です。各 limb は厳密な 2 進値なので、limb ごとに `decimal.Decimal` へ変換して合計すればフル精度の値が厳密に再現されます (Python の `float` 演算では 53 ビットに丸め戻されてしまいます):

```
import ctypes as ct
```

```

from decimal import Decimal, getcontext

lib = ct.CDLL("./librdtq.so")

DD = ct.c_double * 2    # double-double (~32 decimal digits)
TD = ct.c_double * 3    # triple-double (~48 decimal digits)
QD = ct.c_double * 4    # quad-double (~63 decimal digits)

getcontext().prec = 70
def to_decimal(limbs):
    return sum((Decimal(x) for x in limbs), Decimal(0))

x = DD(1.5, 0.0)        # x = 1.5 (high limb, low limb)
r = DD()
lib.rdd_exp(r, x)        # result first, as in rdd.h
print("dd exp(1.5) =", to_decimal(r))
# dd exp(1.5) = 4.48168907033806482260205546011928...

```

引数順に注意してください。rdd.h の慣習どおり、結果が先頭です。float 基のファミリは `ct.c_float * 2/3/4` 配列と `rds_ / rts_ / rqs_` 接頭辞を使います。

9.3 算術演算・FMA・初等関数

次の断片では、quad-double 平方根、証明付き branch-free FMA（厳密な残差検査 $\sqrt{2}^2 - 2$ として）、FMA 駆動 triple-double 除算、 $\sin / \cos$ の同時計算を使います：

```

# quad-double: sqrt(2) to ~63 digits
two = QD(2.0, 0.0, 0.0, 0.0)
s = QD()
lib.rqd_sqrt(s, two)
print("qd sqrt(2) =", to_decimal(s))
# 1.41421356237309504880168872420969807856967187537694807317667973...

# residual sqrt(2)^2 - 2 via the certified FMA: fma(s, s, -2)
m2 = QD(-2.0, 0.0, 0.0, 0.0)
resid = QD()
lib.rqd_fma(resid, s, s, m2)
print("qd sqrt(2)^2 - 2 =", to_decimal(resid))    # ~0 (qd eps level)

# FMA-driven division: 1/3 in triple-double
a = TD(1.0, 0.0, 0.0); b = TD(3.0, 0.0, 0.0); q = TD()

```



```
print("max rel.err vs numpy =", err)    # ~2.2e-16 (binary64 comparison floor)
```

同じパターンが `bnc_dd_log_array / sin / cos` に、そして `limb` 配列を 3 本または 4 本にすれば `bnc_td*_array / bnc_qd*_array` にも使えます。配列レイアウトはライブラリの `ddvector/tdvector/qdvector` 型 (`vec->element`) と同一なので、C 側との結果のやり取りに変換は不要です。

9.5 注意事項

- 結果引数が先頭です (`rdd.h` の慣習)。C 側の `c_dd_*` 関数は逆 (入力先頭) です。取り違えると静かに誤った結果になるため、Python からは `rXX_*` API を使ってください。
- スカラー引数は明示的にラップしてください (`ct.c_double(...)` / `ct.c_float(...)`)。例えば `rXX_fma_safe` のスカラー被乗数です。素の Python float は `ctypes` の既定変換により呼び出しを壊します。
- 入力は正規化された `limb` 組を想定しています。Python float から `(x, 0, ...)` の形で作った値は常に安全で、ライブラリが返す結果は正規化されています。
- `to_decimal` 以上のフル精度入出力には、`mpfr_dtq_sd.c` の MPFR ベース変換関数 (例: `rdd_get_str`) がフルライブラリ `libbncmatmul-0.24*.so` および `librdd.so` からエクスポートされています (`python/rdd.py` を参照)。

参考文献

- [1] D.H. Bailey. QD. <https://www.davidhbailey.com/dhbsoftware/>.
- [2] T. Granlaud and GMP development team. The GNU Multiple Precision arithmetic library. <https://gmplib.org/>.
- [3] M. Jolders, J.-M. Muller, V. Popescu, and W. Tucker. Campary: Cuda mutiple precision arithmetic library and applications. *5th ICMS*, 2016.
- [4] Tomonori Kouya. BNCpack. <https://na-inet.jp/na/bnc/>.
- [5] LAPACK. <http://www.netlib.org/lapack/>.
- [6] MPLAPACK/MPBLAS. Multiple precision arithmetic LAPACK and BLAS. <https://github.com/nakatamaho/mplapack>.
- [7] Daichi Mukunoki, Katsuhisa Ozaki, Takeshi Ogita, and Toshiyuki Imamura. Accurate matrix multiplication on binary128 format accelerated by ozaki scheme. In *50th International Conference on Parallel Processing, ICPP 2021, New York, NY, USA, 2021*. Association for Computing Machinery.
- [8] MPFR Project. The MPFR library. <https://www.mpfr.org/>.
- [9] S. M. Rump, T. Ogita, and S. Oishi. Accurate floating-point summation part I: Faithful rounding. *SIAM Journal on Scientific Computing*, Vol. 31, No. 1, pp. 189–224, 2008.
- [10] S. M. Rump, T. Ogita, and S. Oishi. Accurate floating-point summation part II: Sign, k-fold faithful and rounding to nearest. *SIAM Journal on Scientific Computing*, Vol. 31, No. 2, pp. 1269–1302, 2008.
- [11] Taiga Utsugiri and Tomonori Kouya. Acceleration of multiple precision matrix multiplication using ozaki scheme, 2023.